

User Guide

Using SD Core for Windows: SD BASIC, TCL, the dictionaries and the file system.

SD CORE FOR WINDOWS

W1.0-0

Copyright and licence

Who wrote this set, how it is licensed, and what the licence lets you do with it.

This page carries the copyright and the licence for **the whole User set**. Every other page in the set names them in one line at the foot; the full terms are here, once, so that they are stated in one place and cannot come to disagree with each other.

RELEASED WITH	SD Core for Windows W1.0-0
COPYRIGHT	Copyright © 2026 Donald Montaine
LICENCE	Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)

You are free to **share** this document - copy and redistribute it in any medium or format - and to **adapt** it - remix, transform and build upon it - for any purpose, including commercially.

Two conditions apply. **Attribution:** you must give appropriate credit, provide a link to the licence, and indicate if changes were made. **ShareAlike:** if you remix, transform or build upon this document, you must distribute what you produce under the same licence. You may not add legal terms or technological measures that restrict others from doing anything the licence permits.

This is a summary and not a substitute for the licence itself. The complete text is at <https://creativecommons.org/licenses/by-sa/4.0/>.

The other two sets

The documentation is three separate sets, and each is handed out on its own: **GettingStarted**, **User** and **Administrator**. Each carries its own copy of this page, under the same copyright and the same licence.

The set this page belongs to

Read SD Core - Introduction and Getting Started for what the User set covers and how the two references inside it are arranged.

SD Core - Introduction and Getting Started

What a multivalued database is, what SD is, the four components, and your first session.

This page orients you to SD Core for Windows: what a multivalued database is, where SD came from, what the pieces are, and how to take your first steps. It is the only page in this set that assumes nothing.

What is a multivalued database?

A multivalued database stores data in records made of **fields**, where each field can hold **more than one value** — and each value can hold **more than one subvalue**. A single field in a customer record can therefore carry every phone number the customer has, without a separate table or a join.

The model was designed by Dick Pick in the 1970s as the Pick Operating System. It has been through PI/open, UniVerse, Unidata, D3, jBASE, QM, and ScarletDME — and SD is one of its direct descendants.

The three delimiters that make it work are **field marks**, **value marks** and **subvalue marks** — control characters that separate the levels inside a single string. A dynamic array in SDBasic is a string that carries these marks, and `extract`, `insert`, `delete` and `replace` work on them directly.

What SD is

SD Core for Windows is a version of SD, with elements found in the main SD version and in ScarletDME. ScarletDME was a fork of the original GPL release of OpenQM 2.6.6.

That lineage matters when you go looking for documentation. Not all the features of the *commercial* OpenQM 2.6.6 were in the GPL release, and no documentation specific to the GPL version was ever released. The OpenQM 2.6.6 documents can be used as a reference, but SD Core has additions, changes and deletions — of features, of structure, of security and of commands. This documentation set covers those changes.

If you have used OpenQM, or SD on Linux, much of SD Core will still be familiar: the same data model, the same query processor, the same BASIC.

SD Core for Windows is Windows only. There are no `#ifdef` branches keeping Linux alive in this source — Linux SD is a separate project and this is not a build of it.

SD Core is free software under the GNU General Public Licence v3. `config_gpl` displays the licence and `config_contrib` the list of contributors. The installer carries compiled binaries; the source is a separate download.

The four components

The command processor (TCL)	reads what you type at the <code>:</code> prompt and dispatches it to a verb, a program, a paragraph or a query
The query processor	runs <code>list</code> , <code>select</code> , <code>count</code> , <code>sort</code> and the rest — the reporting language
SDBasic	the programming language: a compiled BASIC with dynamic arrays, file I/O, and the multivalue string functions
The SDClient API	a C client library (<code>sdc11lib.dll</code>) that lets an external application connect to SD, read and write records, execute commands and call subroutines

Signing in

```
sd
```

You land in **the SD account with your own name**. Nothing asks for a password — Windows has already authenticated you. SD asks Windows who you are.

If `sd` answers *Account ... not in register*, you are in the wrong account or your group membership has not taken effect yet. If it answers *not registered for SD use*, you are not in the `sdusers` group.

You must sign out and back in after being added to `sdusers`. Windows fixes group membership when you sign in. Until you get a new logon token you cannot read the data tree at all, and the symptom looks like a broken install.

SD is already running. It is a Windows service — **String Database (SD)** — and Windows starts it at every boot. You do not type `sd -start`.

Your first file and record

```
create.file customers
ed customers 1001
```

`ed` is the line editor, and it needs nothing installed. In `ed`: `i` to insert, type your lines, a full stop on its own line to stop inserting, then `fi` to file and exit.

A programmer account can also use `edit` (Microsoft Edit, a full-screen editor) or `micro` (a full-screen editor with syntax highlighting). Both need `OS.EXECUTE` permission — see the administrator documentation.

```
list customers
count customers
```

Commands are lower case now. Typing `LIST` still works — SD tries what you typed, then lower case, then upper, and finally with any hyphens changed to dots, so `clear-select` reaches `clear.select` too.

Writing a program

A program lives in a `bp` file — a directory file, which is an ordinary Windows folder with one file per program. You can write it in `ed`, in `edit`, in `micro`, or in any text editor you like (Notepad, VS Code, etc.) — the folder is on disk at:

```
C:\ProgramData\SD\user_accounts\<account>\bp
```

Compile and catalogue it from inside SD:

```
basic bp myprog
catalog bp myprog
```

Then run it by name:

```
myprog
```

Becoming an administrator

```
logto sdsys
```

You will get a UAC consent prompt unless the session is already elevated. That is the gate — there is no SDSYS password, and there is deliberately no second shared secret held by every administrator.

IF YOU ARE OVER `ssh`, THIS MAY NOT WORK. A UAC prompt has no interactive desktop there. Start an elevated terminal at the machine instead.

What is not in SD Core

The following were in OpenQM, in ScarletDME, or in SD on Linux, and are not in SD Core for Windows:

GONE	WHY
QMNet (remote files)	Removed; the API is the supported way to reach another SD server
Embedded Python	Dropped; the intended use is as a back end data store reached through the API
<code>sdlnxd</code> daemon	Linux-only; the Windows service replaces it
<code>ENCRYPT.FIELD</code> verb	Removed; <code>sdencrypt()</code> and <code>sddecrypt()</code> in SDBasic are the supported route
<code>sed</code> , <code>update.record</code> , <code>modify</code> editors	Gone; use <code>edit</code> , <code>micro</code> or <code>ed</code>
PROC language	Removed; use paragraphs instead
NLS, <code>SET.LANGUAGE</code>	Removed; SD Core is English only
Silent install	Refused deliberately; the installer asks questions that cannot be defaulted
Multi-user Remote Desktop	Not supported; accounts SD creates are denied the console and RDP

Document conventions

bold	a word typed as it stands
-------------	---------------------------

<i>italics</i>	something you supply
----------------	----------------------

braces { }	an optional part
------------	------------------

<code>code</code>	a command, a function name, or something you type
-------------------	---

SD Basic - Program Structure

Programs, subroutines and functions; variables, matrices, common blocks and how a program is called.

This page covers the shape of an SD BASIC program: how it is declared, how it is called, what its variables are, and the declarations that have to appear before anything else.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The three kinds of program

```
program {name}  
subroutine name {(argument, ...)} {var.args}  
function name {(argument, ...)} {var.args}
```

program	runs on its own. Started by <code>run</code> , by its catalogued name, or from another program with <code>execute</code>
----------------	--

subroutine	called by <code>call</code> , and returns nothing. Its arguments carry values back
-------------------	--

function	called inside an expression, and returns a value with <code>return</code>
-----------------	---

A source record with no declaration at all is a program. `end` closes it.

```
program hello  
  print 'hello'  
end
```

`var.args` allows the caller to pass fewer arguments than are declared; `arg.count()` then says how many actually arrived, and reading one that was not passed is an unassigned-variable error rather than a silent empty string.

LOCAL SUBROUTINES AND FUNCTIONS

```
local subroutine name {(arguments)}  
local function name {(arguments)}
```

These live inside the program that uses them rather than in the catalogue, and they can declare `private` variables of their own — which is how you get a routine whose working variables do not collide with the rest of the program.

They are not called the way external ones are. A local subroutine is reached with `gosub name(...)` and not `call`; a local function must be declared with `defun ... local` before it is used; and both need an explicit `return`, because falling into the closing `end` stops the program. All of that, with the scope rules and the errors each mistake produces, is on SD Basic - Modern Program Structure.

INTERNAL SUBROUTINES

```
gosub label  
...  
label:  
...  
return
```

`gosub` is not a call to another program — it jumps within the current one, and shares all its variables. The nesting limit is **256 levels**.

Calling

```
call name {(argument, ...)}  
call @variable {(argument, ...)}  
subr(name {, argument, ...})
```

<code>call name</code>	the name is fixed at compile time
------------------------	-----------------------------------

<code>call @variable</code>	the name is taken from the variable at run time
-----------------------------	---

<code>subr()</code>	the function form — use it where you want the result in an expression
---------------------	---

Arguments are passed by reference, not by value. A subroutine that assigns to one of its parameters changes the caller's variable. That is how a subroutine returns anything at all, and it is also how a subroutine that uses a parameter as scratch space quietly corrupts its caller's data. **If you do not intend to return a value through an argument, copy it into a local variable first.**

Passing an expression rather than a variable — `call sub(a + 0)` — forces a copy, because there is no variable to write back to.

```
catalogued(name)
```

reports whether a name can be called. `catalogued('ZZNOSUCH')` is `0`. Use it before a `call @variable` whose name came from data, because an uncatalogued name is a runtime abort rather than a testable failure.

Variables

Variables are not declared and have no type. A variable holds whatever was last assigned to it, and its type follows the value.

```
assigned(variable)
unassigned(variable)
vartype(variable)
```

EXPRESSION	RESULT
<code>assigned(neverset)</code>	<code>0</code>
<code>assigned(zz)</code> after <code>zz = 1</code>	<code>1</code>
<code>unassigned(neverset)</code>	<code>1</code>

The compiler warns about a variable that is never assigned — *"WARNING: NEVERSET is not assigned a value"* — but it still compiles, and using the variable is an error only when the line is reached.

VARTYPE()

Returns the internal type code:

VALUE	CODE
unassigned	<code>0</code>

VALUE	CODE
an integer	<code>2</code>
a floating-point number	<code>3</code>
a string	<code>5</code>

The remaining codes exist for things a program does not usually inspect: `1` address, `4` subroutine reference, `6` file variable, `7` array, `8` common block, `9` image, `11` select list, `13` socket, `15` object.

`vartype()` tells you how the value is held, not what it means. `'12'` read from a file is a string (`5`) even though it is a number everywhere it is used; the same value after arithmetic is an integer (`2`). Use `num()` to ask whether something is numeric — see SD Basic - Math Functions — and keep `vartype()` for telling an unassigned variable from an empty one.

`set.unassigned` returns a variable to the unassigned state, which is how you make `assigned()` false again after using it.

Constants

```
equate name to value
equ name to value
```

`equate` is resolved at compile time and costs nothing at run time. It can name a literal, another variable, or a matrix element:

```
equate true to 1
equate esc to char(27)
equate cust.name to record<3>
```

An `equate` is a substitution, not a variable. You cannot assign to one, and it is replaced wherever the name appears. `equate cust.name to record<3>` means `cust.name` reads field 3 of whatever `record` holds at that moment.

```
$include file.name record.name
include file.name record.name
```

pulls in another source record at compile time. The system definitions live in `syscom` — `$include syscom`
`keys.h` for the key numbers used by `fileinfo()`, `selectinfo()` and the rest.

Matrices

```
dim name(rows {, columns})  
dimension name(rows {, columns})  
mat name = expression  
mat name = mat other.name
```

A matrix is a fixed set of numbered variables, not a dynamic array. `dim` must appear before any use, and the bounds are fixed at that point.

```
dim m(3)  
mat m = 7          ;* every element becomes 7
```

After `mat m = 7`, `m(1)`, `m(2)` and `m(3)` all read `7`.

A matrix has a zero element and it is not counted in the dimension. `dim m(3)` gives `m(0)` through `m(3)`. `m(0)` is where `matparse` puts anything left over, and it is easy to forget it exists.

BETWEEN A MATRIX AND A STRING

```
matbuild variable from matrix {, start {, end}}  
matparse matrix from string, delimiter  
inmat({matrix})
```

<code>matbuild</code>	joins the elements into one string
<code>matparse</code>	splits a string across the elements
<code>inmat()</code>	after <code>matparse</code> , how many elements were filled

With `m` holding `7`, `7`, `7`:

`matbuild bs from m`the three elements joined
with **field marks**

`matbuild bs from m, 1, 2`

the first two only

`matparse m from 'x' : @vm : 'y' : @vm : 'z', @vm then m(1) ... m(3)``x y z`

`inmat()` after that`3`

The `using` clause of `matbuild` **does not work, in either case**. The documented syntax `matbuild var from mat using delimiter` compiles the keyword as a **variable name** and the program aborts at run time with *"Unassigned variable USING"*, with both `using` and `USING`.

This is inherited, not introduced by this port — the `st.matbuild` block in `BCOMP` is byte-identical to the one in the upstream `sdb64` tree. **The delimiter is always a field mark**. If you need another one, use `matbuild` and then `change()`, or build the string with a `for` loop.

`matparse` splits on the delimiter you give and stops when the matrix is full; anything left over goes into element zero.

Common blocks

```
common {/name/} variable, ...
clearcommon
delete.common name
```

Variables in a common block survive `chain` and are shared between programs that declare the same block. An **unnamed** common block — `common // a, b, c` — belongs to the program run and is cleared when it ends; a **named** one persists for the session.

`//` **is the unnamed common marker, which is why it is not an integer-divide operator**. See SD Basic - Math Functions.

`clearcommon` sets every unnamed common variable to zero. `delete.common` discards a named block entirely, which is the only way to reclaim it within a session.

A common block is matched by name and position, not by variable name. Two programs declaring the same block with different variable lists will read each other's data at the wrong offsets, silently. **Put the declaration in an include record and `$include` it everywhere**, so there is one copy.

Class modules and objects

SD BASIC has class modules with member variables, methods, properties, constructors, destructors and inheritance. **They have their own page**, because they are a different way of writing a program rather than a feature of this one: SD Basic - Modern Program Structure.

The short form:

```
class name {inherits base}
private variable, ...      public variable, ...
public subroutine name(...) public function name(...)
get name                  set name(value)
end
```

```
o = object('CLASSNAME')
o->method(args)
o->property = value
```

SD's own client library is a class module. `gpl.bp/SDCLIENT` is 1,040 lines with 33 members and both lifecycle hooks — connecting to another SD server from BASIC means instantiating it. *(An earlier version of this page said nothing in SD used class modules. That was wrong.)*

Arguments

```
arg.count()
arg(n)
set.arg n, value
```

These read the arguments of the **command line** that started the program, not the arguments of a subroutine call.

`arg(1)` is the first word after the verb.

Discarding a result

```
void expression
```

Calls a function and throws the answer away. It exists because SD BASIC has no statement form for a function call — `void kernel(...)` is the idiom you will see throughout `gp1.bp`.

What is not here

Nothing in the program-structure group has been removed from this port. The `package` statements — `enter.package`, `exit.package` and `package` — went before SD Core, and `varset` is present but is an internal debugging aid rather than something application code uses.

See also

SD Basic - Program Control · SD Basic - Dynamic Arrays · SD Basic - File Handling · SD Basic - Math Functions.

SD Basic - Program Control

Conditions, loops, jumps, and handing control to another program or to the command processor.

This page covers the statements that decide what runs next: conditional execution, loops, jumps, and the several ways one program starts another.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Conditions

```
if expression then statement {else statement}
```

```
if expression then
    statements
end else
    statements
end
```

The single-line and block forms can be mixed — `then` on one line and `end else` opening a block is legal — but a block that starts on one line must end with `end` before the `else`.

An empty branch still needs a statement. Use `null`:

```
read rec from f, id then null else stop 'missing'
```

`and` **and** `or` **have the same precedence and are applied left to right.** `a or b and c` means `(a or b) and c`, not `a or (b and c)`. This is not what C, Python, Java or SQL do, and it is the single most common way a condition in ported code changes meaning. **Parenthesise anything that mixes them.** The full precedence table is in SD Basic - Math Functions.

BEGIN CASE

```
begin case
  case expression
    statements
  case expression
    statements
  case 1
    statements
end case
```

The first matching case runs and the rest are skipped — with two identical `case n = 2` branches, only the first executes. There is no fall-through and no `break`.

`case 1` is the conventional default, because the constant 1 is always true. If no case matches and there is no default, nothing runs.

Loops

COUNTED

```
for variable = start to limit {step increment}
  statements
next variable
```

LOOP

VALUES TAKEN

`for i = 1 to 10 step 3`

`1 4 7 10`

`for i = 1 to 2 step 0.5`

`1 1.5 2`

`for i = 3 to 1`

none — the body never runs

The limit is tested before the first pass, so a loop whose start is already past its limit does nothing rather than running once. A fractional `step` is allowed, but a fractional step that cannot represent the limit exactly may stop one iteration early — count with integers and scale if it matters.

`step` may be negative, in which case the loop runs while the variable is greater than or equal to the limit.

CONDITIONAL

```
loop
  statements
while expression
  statements
repeat
```

`while` continues the loop when the expression is true; `until` continues when it is false. Either may appear anywhere in the body, more than once, and a `loop` with neither runs until something in the body leaves it.

`loop / k += 1 / r := k / while k < 3 / repeat` produces `1 2 3` — the test happens where it is written, so the body before it always runs at least once.

```
continue
exit
```

`continue` starts the next pass; `exit` leaves the loop. Both apply to the innermost loop only.

Jumps

```
goto label
go to label
gosub label
return
return to label
```

Labels end with a colon at their definition and are written without one in the jump. A label must be in the same program.

`ON ... GOTO` AND `ON ... GOSUB`

```
on expression goto label1, label2, label3
on expression gosub label1, label2, label3
```

Jumps to the n -th label, counting from 1.

An out-of-range value does not fall through — it is clamped to the nearest end. This is the opposite of what most MultiValue documentation says. On this port, with two labels:

EXPRESSION	WHERE IT WENT
<code>on 0 goto a1, a2</code>	label 1
<code>on 3 goto a1, a2</code>	label 2
<code>on -1 goto a1, a2</code>	label 1

So there is no "none of the above" branch. A value of zero — which is what an empty or non-numeric variable evaluates to — silently runs the first case. **Range-check the expression yourself before the statement**, or use `begin case`, which has a real default.

Stopping

```
stop {message}
abort {message}
quit
```

`stop` ends the program and returns to whatever called it

`abort` ends the program **and everything that called it**, returning to the command processor

`quit` leaves the current command level

`set.exit.status value` sets the value a caller can read, which is how a program run from a script reports success or failure.

Running something else

```
execute expression {capturing variable} {returning variable} {passlist} {rtnlist variable}
perform expression
```

```
chain expression
enter name {(arguments)}
```

execute runs a command **at a new level**; the current program resumes afterwards

perform as **execute**, but the command shares the caller's level — it can see and change the caller's select lists and files

chain replaces the current program; **nothing comes back**

enter transfers to another program, also without return, but keeps the current level

execute 'WHO' capturing **cap** puts the command's output in **cap** rather than on the screen, one field per line — **dcount(cap, @fm)** is **1** for a single-session **WHO**.

A command that does not exist is not an error you can catch. **execute** 'ZZNOSUCHVERB' capturing **out** completes, and **@system.return.code** reads **-1**. The program carries on. **Test** **@system.return.code** **after any** **execute** **whose command name came from data.**

CLAUSE

capturing *var* the command's output goes to *var* instead of the terminal

returning *var* error messages go to *var*

passlist the caller's default select list is handed to the command

rtnlist *var* the command's select list comes back

chain **discards unnamed common by default.** The **chain.keep.common** option preserves it — **option** **chain.keep.common** — which is the setting a ported application usually needs.

Waiting

```
sleep {seconds}
nap milliseconds
pause {timeout}
```



```
wake user.number  
rqm {time}
```

<code>sleep</code>	wait a number of seconds, or until a time of day
<code>nap</code>	wait a number of milliseconds
<code>pause</code>	wait until another process calls <code>wake</code> , or until the timeout expires
<code>wake</code>	release a process that is paused
<code>rqm</code>	release the processor briefly — the historical "release quantum"

`pause` and `wake` are how two SD sessions coordinate without polling. The user number `wake` needs is the one `@user.no` reports in the waiting process.

Interrupting

```
break {key} {on | off}  
break {key} clear  
set.break.handler name  
remove.break.handler
```

`break off` prevents the user interrupting the program; `break on` restores it. **Every `break off` needs a matching `break on`** — they nest, and a program that aborts between the two leaves the session with interrupts disabled.

`set.break.handler` names a subroutine to run instead of entering the debugger, which is how a program cleans up rather than dying at an arbitrary point.

`break off` around a whole program is a trap, not a safety measure. It also disables the interrupt during any `execute` the program performs, so a runaway query started from inside it cannot be stopped either. Turn it off around the critical section — the write, the transaction — and back on immediately.

Doing nothing

```
null
rem text
remark text
* text
```

`null` is a statement that does nothing, for branches that must be present but empty. `rem`, `remark` and a leading `*` are comments; so is anything after `;*` on a line.

`rem` is also a function. `rem(a, b)` is the remainder — see SD Basic - Math Functions. The compiler tells them apart by the bracket: a `rem` immediately followed by `(` is the function, anything else is a comment. `rem (a + b)` is wrong is a comment, and `x = rem (a, b)` is a syntax error, because the space changes what it is.

What is not here

These existed in OpenQM or earlier SD releases and **are not in SD Core for Windows**. A program using one will not compile.

<code>aborte</code> · <code>abortm</code>	abort with a message number or text
<code>stope</code> · <code>stopm</code>	stop with a message number or text
<code>rsd</code>	
<code>enter.package</code> · <code>exit.package</code> · <code>package</code>	
<code>trace</code>	
<code>errmsg</code>	see below

`errmsg` is in the compiler's statement table and still does not compile. `errmsg 1000` is rejected with "Unrecognised statement". The name survives in `BCOMP`'s list but its opcode was removed, so the table is not a reliable guide for this one — **being listed as a statement does not mean a statement exists**.

`procread` and `procwrite` **do** compile, despite appearing on the SD 0.8.0 removed list; `procread` was restored afterwards. Both were checked, not assumed.

See also

[SD Basic - Program Structure](#) · [SD Basic - Math Functions](#) · [SD Basic - File Handling](#) · [SD Basic - Select Lists](#).

SD Basic - Math Functions

Arithmetic, division and remainders, rounding, powers, trigonometry and random numbers.

This page covers the numeric side of SD BASIC: the arithmetic operators, the functions that divide and round, powers and logarithms, trigonometry, the functions that reduce a dynamic array to a number, bit manipulation, and the control over how many decimal places a number shows when it becomes a string.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. Braces mark optional parts of a syntax line; in the tables, *italics* mark something you supply and **bold** marks a word typed as it stands.

Where this port behaves differently from the older OpenQM and SD documentation, the difference is called out rather than left for you to discover.

Numbers in SD BASIC

SD holds a number as either an **integer** or a **floating-point** value and moves between them as needed. You do not declare which; an expression that can stay integral does, and one that cannot becomes floating point.

Everything in SD is ultimately a string, so a variable holding "12" is a number wherever a number is wanted. A string that is not numeric evaluates as zero in an arithmetic context, and num() below is how you test before relying on that.

ARITHMETIC OPERATORS

OPERATOR	MEANING
+	add
-	subtract, and unary minus
*	multiply
/	divide — the result may be fractional
** · ^	raise to a power. The two spellings are the same operator

There are also compound assignment operators — `+=`, `-=`, `*=`, `/=` and `:=` for concatenation — so `total += n` is `total = total + n`.

There is no integer-divide operator. `/` always divides fully:

```
print 7 / 2      ;* prints 3.5
```

For a whole-number result use `div()`, `idiv()` or `rdiv()`, described below. `//` is **not** integer division — it is the marker for an unnamed common block, as in `common // a, b, c`.

PRECEDENCE

Operators lower in this table are applied first. Anything in parentheses is evaluated before the expression around it.

OPERATORS

- | | |
|---|---|
| 1 | <code>**</code> <code>^</code> |
| 2 | <code>*</code> <code>/</code> |
| 3 | <code>+</code> <code>-</code> |
| 4 | the <code>fmt</code> operator |
| 5 | <code>:</code> concatenation |
| 6 | <code><</code> <code>></code> <code>=</code> <code>#</code> <code><=</code> <code>>=</code> <code><></code> <code>matches</code> , and their word forms <code>lt</code> <code>gt</code> <code>eq</code> <code>ne</code> <code>le</code> <code>ge</code> |
| 7 | <code>and</code> <code>&</code> <code>or</code> <code>!</code> |

`and` and `or` have the same precedence, which is not what most languages do. They are applied left to right as they are met, so `a or b and c` is `(a or b) and c` — not `a or (b and c)`. **Parenthesise any condition that mixes them.** This has been true since OpenQM and is not a change made by this port, but it surprises people arriving from C, Python or SQL, where `and` binds tighter.

DIVIDING BY ZERO

Dividing by zero stops the program with a runtime error. That can be turned into a warning that yields zero and carries on:

```
option div.zero.warning
```

The option applies to `/`, `div()`, `idiv()` and `rdiv()` alike.

Dividing to a whole number

Three functions divide and return an integer, and **they do not round the same way**. Picking the wrong one is the commonest numeric bug in MultiValue code, so the differences are set out in full.

FORMAT

```
div(dividend, divisor)
idiv(dividend, divisor)
rdiv(dividend, divisor)
```

FUNCTION RULE

<code>div()</code>	divides and discards the fraction, always towards zero
<code>idiv()</code>	towards zero when both arguments are integers, downwards when either is floating point — see the warning below
<code>rdiv()</code>	divides and rounds to the nearest integer, with a half rounded away from zero

THE FOUR SIDE BY SIDE

EXPRESSION	RESULT	EXPRESSION	RESULT
<code>div(7, 2)</code>	3	<code>rdiv(7, 2)</code>	4
<code>div(-7, 2)</code>	-3	<code>rdiv(-7, 2)</code>	-4
<code>div(7, -2)</code>	-3	<code>rdiv(7, -2)</code>	-4
<code>div(-7.5, 2)</code>	-3	<code>rdiv(5, 2)</code>	3
<code>idiv(7, 2)</code>	3	<code>rdiv(-5, 2)</code>	-3
<code>idiv(-7, 2)</code>	-3		

`idiv()` **changes its rounding when a floating-point number is involved, and nothing in the call shows it.** With two integers it truncates towards zero; if either argument is floating point it takes the floor, which for a negative result is a different number:

```
idiv(-7, 2) ;* -3 both arguments are integers idiv(-7.0, 2) ;* -4 one argument is floating point
```

The two differ only for negative results — for positive ones truncation and floor agree. **If the sign can be negative, use `div()`**, which truncates towards zero whatever the argument types are. This behaviour is inherited from OpenQM and has been kept for compatibility; it is not new in this port.

`div()` returns a whole number even when the arguments are fractional: `div(7.5, 2)` is `3`.

Remainders

```
mod(dividend, divisor)
rem(dividend, divisor)
```

Both give the remainder of a division and **they differ on the sign**. `mod()` takes the sign of the *divisor*; `rem()` takes the sign of the *dividend*.

EXPRESSION	MOD	REM
<code>(7, 3)</code>	<code>1</code>	<code>1</code>
<code>(-7, 3)</code>	<code>2</code>	<code>-1</code>
<code>(7, -3)</code>	<code>-2</code>	<code>1</code>
<code>(-7, -3)</code>	<code>-1</code>	<code>-1</code>

`mod()` is the mathematical modulus — the result always lies between zero and the divisor — which is what you want for wrapping a value into a range, such as an index into a cyclic table. `rem()` is the remainder C and most other languages give, and is what you want when you are reconstructing a value from a quotient and a remainder.

Neither raises a divide-by-zero error. A divisor of zero returns the dividend unchanged: `mod(7, 0)` and `rem(7, 0)` are both `7`. A loop that relies on a remainder shrinking will not terminate if the divisor can reach zero, and nothing will tell you.

Truncation, sign and magnitude

```
int(expr)
abs(expr)
neg(expr)
```

`int()` discards the fractional part, **towards zero**. `int(3.7)` is `3` and `int(-3.7)` is `-3`. It does not round: use `rdiv(n, 1)` or add `0.5` if you want rounding

`abs()` the magnitude, with any sign removed. `abs(-3.5)` is `3.5`

`neg()` the value with its sign reversed. `neg(3)` is `-3`

`int()` is affected by a configuration parameter. `INTPREC` sets how many decimal places are considered before truncating, which stops a value that is `2.999999999` only because of floating-point representation from truncating to `2`. It defaults to `13` and may be set from `0` to `14`. `config('INTPREC')` reports the value in force, and `config_gpl` is not where to look — the whole parameter list is shown by the `config` verb.

Powers, roots and logarithms

```
pwr(base, exponent)
sqrt(expr)
exp(expr)
ln(expr)
```

`pwr()` *base* raised to *exponent*. Identical to the `**` and `^` operators, so `pwr(2, 10)`, `2 ** 10` and `2 ^ 10` all give `1024`. A fractional exponent works: `pwr(2, 0.5)` is `1.4142`

`sqrt()` the square root. `sqrt(2)` is `1.4142`

`exp()` *e* raised to the given power. `exp(1)` is `2.7183`

`ln()` the natural logarithm, base *e*. `ln(10)` is `2.3026`

There is no base-10 logarithm function. Divide by the natural log of ten: `ln(x) / ln(10)`.

`sqrt()` and `ln()` **stop the program on a negative argument** — *"SQRT() attempted for negative value"* and *"LN() attempted for negative value"*. Neither returns an error code you can test, so guard the argument yourself if it can go negative.

The four decimal places shown above are not the precision of the calculation — they are the default precision applied when the number is turned into a string for printing. See *Controlling how numbers print*.

Trigonometry

```
sin(expr)    asin(expr)
cos(expr)    acos(expr)
tan(expr)    atan(expr)
```

The arguments and results are in degrees, not radians. This catches people out constantly, because almost every other language's maths library uses radians. There is no radian variant.

EXPRESSION	RESULT	EXPRESSION	RESULT
<code>sin(30)</code>	<code>0.5</code>	<code>asin(0.5)</code>	<code>30</code>
<code>cos(60)</code>	<code>0.5</code>	<code>acos(0.5)</code>	<code>60</code>
<code>tan(45)</code>	<code>1</code>	<code>atan(1)</code>	<code>45</code>

To work in radians, convert: multiply by `180 / 3.14159265358979` on the way in, and divide on the way out.

Random numbers

```
rnd(limit)
randomize {seed}
```

`rnd()` returns a whole number from `0` to *limit* minus one — so `rnd(10)` gives `0` through `9`, confirmed over 500 draws. A negative *limit* gives a negative result in the mirrored range: `rnd(-10)` returns `0` down to `-9`.

`randomize` seeds the generator. With a *seed* the sequence repeats identically every run — `randomize 42` followed by three `rnd(1000)` calls gives the same three numbers every time — which is what you want for a reproducible test. With no argument it seeds from the clock, which is what you want in production.

```
randomize 42
for i = 1 to 5
  print rnd(100)
next i
```

This is not a cryptographic random number generator and must not be used to make passwords, tokens or keys. It is the C library's `rand()`, seeded from a 32-bit value. For anything security-bearing use `sdcrypt()` and the facilities described under *Security* rather than building your own.

Continued in

SD Basic - Numeric Tests and Formatting — reducing an array to a number, numeric tests, bit operations, checksums and how numbers print.

SD Basic - Numeric Tests and Formatting

Reducing arrays to a number, testing what is numeric, bit operations, checksums and print formatting.

This page continues SD Basic - Math Functions.

Reducing a dynamic array to a number

These take a dynamic array and give a single value, or a shallower array.

```
sum(array)
summation(array)
maximum(array)
minimum(array)
```

sum()	adds the elements at the lowest mark level present and returns an array one level shallower. Subvalues become values, values become fields
summation()	adds every element at every level and returns one number
maximum()	the largest numeric element
minimum()	the smallest numeric element

WORKED THROUGH

With **a** holding **1**, **2**, **3** as values, and **b** holding a field of **10**^s**20** and **30**, where ^s is a subvalue mark:

EXPRESSION	RESULT
sum(a)	6
sum(b)	30 and 30 as two values — the subvalues collapsed, the value did not
summation(b)	60

EXPRESSION	RESULT
<code>maximum(a)</code>	3
<code>minimum(a)</code>	1

All four silently skip anything that is not a number. With 1, two, 3 in an array, `sum()` returns 4 and `maximum()` returns 3. Nothing is reported. **If a non-numeric element means your data is wrong, you have to test for it yourself** — walk the array with `num()` before summing.

`maximum()` and `minimum()` return the **null string**, not zero, when there is no numeric element at all. Test for that with `if result = '' then`, because a numeric test would read the null string as zero and cannot tell it from a genuine maximum of zero.

While developing, `option non.numeric.warning` makes SD report a non-numeric value used where a number was expected, which turns a class of silent wrong answers into something you can see. It is off by default. `option` on its own lists every option and its current state; `option all off` clears them.

Negative values are handled correctly: `maximum()` of -5 and -3 is -3.

COMPARING TWO VALUES

```
max(expr1, expr2)
min(expr1, expr2)
```

These take **two values, not an array**, and return the larger or smaller. Unlike the four above they are not restricted to numbers: they use SD's general comparison, so `max('abc', 'abd')` returns `abd`. `max(3, 7)` returns 7.

Testing whether a value is a number

```
num(expr)
```

Returns true if *expr* would be accepted as a number, false otherwise.

WHAT IT ACCEPTS

EXPRESSION	RESULT	
<code>num('12')</code>	true	
<code>num('12.5')</code>	true	
<code>num('.5')</code>	true	a leading decimal point is fine
<code>num('+3')</code>	true	a leading sign is fine
<code>num('')</code>	true	the null string is numeric
<code>num(' 12')</code>	false	no leading space
<code>num('12 ')</code>	false	no trailing space
<code>num('3-')</code>	false	a trailing sign is not accepted
<code>num('1e3')</code>	false	no exponent notation
<code>num('123456789012345')</code>	true	15 digits before the point
<code>num('1234567890123456')</code>	false	16 is too many

`num('')` is true, and it is the single most common source of a wrong answer from this function. An empty field passes the test and then evaluates as zero, so a validation written as `if num(value) then ...` accepts a blank. **Test for the null string first:**

```
if value # '' and num(value) then ... end
```

The rule is consistent with the rest of SD, where an empty value is zero in a numeric context — but the function reads like a validator and is not one.

`nums()` applies the same test to every element of a dynamic array. See SD Basic - Dynamic Arrays for the whole family of `s` functions.

Bit operations

These work on **32-bit signed integers**. A value outside that range is truncated to fit.

<code>bitand(expr1, expr2)</code>	<code>bitset(expr, bit)</code>
<code>bitor(expr1, expr2)</code>	<code>bitreset(expr, bit)</code>
<code>bitxor(expr1, expr2)</code>	<code>bittest(expr, bit)</code>
<code>bitnot(expr)</code>	<code>shift(expr, distance)</code>

`bitand()` bitwise AND. `bitand(12, 10)` is `8`

`bitor()` bitwise OR. `bitor(12, 10)` is `14`

`bitxor()` bitwise exclusive OR. `bitxor(12, 10)` is `6`

`bitnot()` bitwise NOT — every bit inverted. `bitnot(0)` is `-1`

`bitset()` returns *expr* with the given bit set. `bitset(0, 3)` is `8`

`bitreset()` returns *expr* with the given bit cleared. `bitreset(15, 0)` is `14`

`bittest()` true if the given bit is set. `bittest(8, 3)` is true, `bittest(8, 2)` is false

`shift()` shifts the bits. **A negative distance shifts left, a positive distance shifts right**

Bits are numbered from zero, and zero is the least significant bit. So bit 3 has the value 8, and bit 31 is the sign bit. **There is no range check on the bit number** — a value outside 0 to 31 gives an unpredictable result rather than an error, so validate it yourself if it is calculated.

`shift()` 's direction is the reverse of what the name suggests, and its right shift does not preserve the sign. `shift(1, -4)` is `16` — shifted left. `shift(256, 4)` is `16` — shifted right. And `shift(-1, 1)` is `2147483647`, not `-1`: the value is treated as unsigned, so a right shift brings in zeros rather than copying the sign bit. **Do not use `shift()` to divide a signed number by a power of two** — use `div()`.

Checksums

`checksum(expr)`

Returns a 32-bit integer derived from the bytes of *expr*. It is cheap and it changes when the data changes — `checksum('hello')` is `1199` and `checksum('hellp')` is `1200` — which makes it useful for spotting that a record has been altered since you last read it.

It is not a cryptographic hash and must not be used as one. It is a rotate-and-exclusive-or over the bytes, and it is straightforward to construct a different string with the same result. It will not detect deliberate tampering, and it must never be used to store or compare a password. `checksum('')` is `0`, so an empty value and a failed read are indistinguishable by checksum alone.

For real cryptography this port provides `sdencrypt()` and `sddecrypt()`. The older `encrypt()` and `decrypt()` functions **have been removed** and programs using them will not compile.

Controlling how numbers print

`precision n`

Sets how many decimal places are used when a floating-point number is converted to a string — by `print`, by concatenation, by writing to a file. It does not change the accuracy of the arithmetic itself, only what you see and what gets stored.

The default is 4. *n* may be 0 to 14; a value outside that range is pulled to the nearest end rather than refused.

<code>1 / 3</code> at the default	<code>0.3333</code>
<code>1 / 3</code> after <code>precision 8</code>	<code>0.33333333</code>
<code>1 / 3</code> after <code>precision 0</code>	<code>0</code>
<code>1 / 3</code> after <code>precision 20</code>	<code>0.33333333333333</code> — clamped to 14, not refused

Precision is per program and it does not travel with a call. Every program starts at 4, including one you `call`, and setting it in a caller has no effect inside the subroutine. It is restored to the caller's value when the subroutine returns. **A subroutine that formats numbers must set its own `precision`** — a common cause of a report where the totals show four decimal places and the line items show something else.

Where you need a specific layout rather than a number of decimals, use `fmt()` and the output conversion codes instead; they are covered in SD Basic - Data Conversion.

COMPARING FLOATING-POINT NUMBERS

Two floating-point values are treated as equal if they differ by less than the `FLTDIFF` configuration parameter, which defaults to a very small number (`0.000000000291`).

This is why `0.1 + 0.2 = 0.3` is true in SD BASIC, where the same test is false in C, Java, Python and JavaScript. It is a deliberate convenience and not an accident of this port. `config('FLTDIFF')` reports the value in force.

It also means two values that differ in the fourteenth decimal place compare as equal, so a comparison is not a safe way to detect a tiny drift. If you need to know that two numbers really are bit-identical, compare their string forms after `precision 14`.

Applying a function to every element

Several of the functions above have a partner ending in `s` that applies the same operation to every element of a dynamic array instead of to a single value: `abss()`, `negs()`, `mods()`, `nums()`.

```
abss(-1 : @vm : 2 : @vm : -3)    ;* gives 1, 2, 3 as three values
```

The whole family — including the string and comparison members — is described in SD Basic - Dynamic Arrays.

What is not here

These existed in OpenQM and in earlier SD releases and **are not in SD Core for Windows**. A program using one will not compile.

<code>encrypt()</code> · <code>decrypt()</code>	replaced by <code>sdencrypt()</code> and <code>sddecrypt()</code>
---	---

Nothing else in the mathematical and logical group has been removed: every function the OpenQM 2.6.6 documentation lists under that heading is present in this port, and this page covers the numeric ones. The comparison and logical members of the group — `ands()`, `ors()`, `nots()`, `eqs()`, `nes()`, `gts()`, `ges()`, `lts()`, `les()`, `ifs()`, `not()`, `reuse()` — are documented in SD Basic - Dynamic Arrays, where their multivalue behaviour is the point.

See also

SD Basic - Dynamic Arrays · SD Basic - Data Conversion · SD Basic - String Functions · the `config` verb, for `INTPREC` and `FLTDIFF`.

SD Basic - String Functions

Measuring, finding, extracting, trimming, building and changing the case of text.

This page covers the functions that work on a string as text. The functions that work on a string as a **structure** — fields, values and subvalues held together by marks — are in SD Basic - Dynamic Arrays, and the ones that convert between internal and external form are in SD Basic - Data Conversion. The line between the three is not always obvious, so each page points at the others where they meet.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Strings, and what counts as a character

Everything in SD is a string. There is no separate character type and no length declaration — a variable holds as much as you put in it.

The mark characters are ordinary characters to every function on this page. `@fm`, `@vm` and `@sm` occupy one byte each and are counted, indexed, extracted and replaced like any other:

```
len(1 : @vm : 2)      ;* 3 - the two digits and the mark between them
len(@fm)              ;* 1
```

That is convenient when you want it and a trap when you do not. A `len()` used to size a display field will include the marks; an `index()` searching for a character may find one inside a mark-delimited structure you did not mean to look at. Where you want structure rather than text, use the dynamic-array functions.

Case conversion leaves marks alone: `upcase(@vm)` is still `@vm`.

Measuring and counting

```
len(string)
count(string, substring)
dcount(string, delimiter)
```

`len()` the number of characters. `len('hello')` is 5, `len('')` is 0

`count()` how many times *substring* occurs

`dcount()` how many **fields** *delimiter* divides the string into — normally one more than `count()`

THE DIFFERENCE IN PRACTICE

EXPRESSION	RESULT	
<code>count('a,b,c', ',')</code>	2	two delimiters
<code>dcount('a,b,c', ',')</code>	3	three fields
<code>dcount('a,,c', ',')</code>	3	the empty middle field still counts
<code>dcount('a', ',')</code>	1	no delimiter, one field
<code>dcount('', ',')</code>	0	not 1
<code>count('aaaa', 'aa')</code>	2	matches do not overlap

`dcount()` **of an empty string is zero, not one.** Everywhere else an empty string behaves like a single empty field, so `for i = 1 to dcount(x, @fm)` is the correct idiom precisely **because** it does nothing when `x` is empty. Code that assumes at least one field and indexes `field(x, ',', 1)` without checking will read an empty value rather than fail.

`count()` **does not count overlapping matches.** `count('aaaa', 'aa')` is 2, not 3: once a match is taken, scanning resumes after it. This matters when the substring can overlap itself.

Delimiters may be more than one character — `dcount('a::b', '::')` is 2.

Finding a position

`index(string, substring, occurrence)`

Returns the character position at which the *occurrence*-th appearance of *substring* starts, or 0 if there is no such occurrence. **The occurrence argument is not optional.**

EXPRESSION	RESULT
<code>index('banana', 'an', 1)</code>	2
<code>index('banana', 'an', 2)</code>	4
<code>index('banana', 'an', 3)</code>	0 — there is no third
<code>index('banana', 'z', 1)</code>	0
<code>index('banana', 'an', 0)</code>	0

Because a miss and a zeroth occurrence both return 0, and position 0 is not a valid position, a simple `if index(...) then` is a safe test for "was it found".

`COL1()` AND `COL2()`

```
col1()
col2()
```

After a `field()` call, these give the positions **either side** of the field that was extracted: `col1()` is the position of the delimiter before it and `col2()` the position of the delimiter after it.

For `field('alpha,beta,gamma', ',', 2)`, which returns `beta`:

<code>col1()</code>	6 — the comma before <code>beta</code>
<code>col2()</code>	11 — the comma after it

For the first field `col1()` is 0, since nothing precedes it. **When the field is not found, both are 0** — which is also what they read before any `field()` has been called, so they cannot distinguish "not found" from "not asked". Test the result of `field()` itself.

`col1()` and `col2()` belong to the running program, and every called program starts with both at zero. A subroutine that calls `field()` changes its own pair, not the caller's.

Extracting part of a string

BY POSITION

The substring operator takes a start and a length:

```
string[start, length]
```

EXPRESSION ON 'ABCDEFGH'	RESULT	
s[2, 3]	bcd	
s[2, 99]	bcdefgh	a length past the end is not an error
s[9, 2]	empty	a start past the end gives nothing
s[0, 3]	abc	a start below 1 is treated as 1
s[-3, 2]	ab	so is a negative start

The one-argument form is the rightmost characters, not the character at that position.

```
s[3]      ;* fgh - the last three characters of 'abcdefgh'
```

This reads like an array subscript and is not one. It is the commonest misreading of SD BASIC by someone arriving from another language.

The substring form can also be assigned to, replacing that stretch in place:

```
z = 'a,b,c'  
z[2,1] = 'Z'      ;* z is now 'aZb,c'
```

BY DELIMITER

```
field(string, delimiter, occurrence {, count})  
fieldstore(string, delimiter, start, count, replacement)
```

<code>field()</code>	returns the <i>occurrence</i> -th delimited field. With <i>count</i> , returns that many fields including their delimiters
<code>fieldstore()</code>	replaces <i>count</i> fields starting at <i>start</i> . A count of zero inserts instead of replacing

EXPRESSION	RESULT
<code>field('a,b,c', ',', 2)</code>	<code>b</code>
<code>field('a,b,c', ',', 2, 2)</code>	<code>b,c</code>
<code>field('a,b,c', ',', 9)</code>	<i>empty</i>
<code>fieldstore('a,b,c', ',', 2, 1, 'X')</code>	<code>a,X,c</code> — replaced
<code>fieldstore('a,b,c', ',', 2, 0, 'X')</code>	<code>a,X,b,c</code> — inserted

An *occurrence* below 1 is treated as 1.

`substrings(array, start, length)` applies the substring operator to **every element** of a dynamic array:
`substrings('abcd' : @vm : 'efgh', 2, 3)` gives `bcd` and `fgh`.

Trimming

```
trim(string)
trimb(string)
trimf(string)
trim(string, character {, mode})
```

The one-argument forms work on spaces:

EXPRESSION ON <code>' A B '</code>	RESULT
<code>trim()</code>	<code>a b</code> — leading, trailing and repeated inner spaces
<code>trimb()</code>	<code>a b</code> — trailing only
<code>trimf()</code>	<code>a b</code> — leading only

The three-argument form works on any character and takes a mode letter. **All nine, on 'xxaxxbxx' trimming** `x`:

MODE	MEANING	RESULT
a	remove every occurrence	<code>ab</code>
b	leading and trailing only	<code>axxb</code>
c	compress repeats, leave the ends	<code>xaxbx</code>
d	as <code>trim()</code> — spaces, both ends and compressed inside	—
e	as <code>trimb()</code> — trailing spaces	—
f	as <code>trimf()</code> — leading spaces	—
l	leading only	<code>axxbxx</code>
t	trailing only	<code>xxaxxb</code>
r	both ends and compress repeats inside	<code>axb</code>

Omitting the mode gives `r`, not `b`: `trim(t, 'x')` is `axb`. If you only want the ends stripped, you must say `'b'` — this is the difference between `a b` and `a b` on real data and it is easy to miss.

Modes `d`, `e` and `f` ignore the character argument entirely and act on spaces, so the character you pass with them is never read.

For every other mode the character must not be empty. A null second argument stops the program with "Null string as second argument to TRIM()" — so a `trim(x, sep)` where `sep` came from data needs checking before the call, not after.

`CROP()`

```
crop(string)
```

Removes **trailing empty fields, values and subvalues** — the debris left by deleting the last element of a dynamic array. `crop()` of a value list `a`, `b`, empty, empty gives just `a`, `b`. It does not touch spaces.

Building strings

```
space(count)
str(string, count)
```

`space()` *count* spaces. `space(0)` is empty

`str()` *string* **repeated** *count* times

`str()` **repeats, it does not pad.** `str('ab', 3)` is `ababab`, six characters, not `ab` padded to three. To pad, concatenate `space()` and take a substring, or use `fmt()` — see SD Basic - Data Conversion.
`str('ab', 0)` is empty.

Strings are joined with the `:` operator, and `:=` appends in place:

```
line = 'a' : ',' : 'b'
line := ',c'
```

`cats(array1, array2)` joins two dynamic arrays **element by element** rather than end to end, and `splice(array1, string, array2)` does the same with a separator between each pair:

EXPRESSION ON `A`, `B` AND `1`, `2`

RESULT

`cats(p, q)`

`a1` and `b2`

`splice(p, '-', q)`

`a-1` and `b-2`

Changing case

```
upcase(string)
downcase(string)
swapcase(string)
```

EXPRESSION ON	'ABC1'	RESULT
<code>upcase()</code>		ABC1
<code>downcase()</code>		abc1
<code>swapcase()</code>		AbC1

Digits, punctuation and mark characters are left alone.

`fold()` **is not a case function.** Despite the name it wraps text into lines — see *Wrapping text into lines* below. The case-insensitive comparison people reach for `fold` expecting is `compare()` or, for whole files, the no-case file option described under SD Basic - File Handling.

Continued in

SD Basic - String Matching and Replacing — replacing, wrapping, comparing, pattern matching, quoting and phonetic matching.

SD Basic - String Matching and Replacing

Replacing, wrapping, comparing, pattern matching, quoting and phonetic matching.

This page continues SD Basic - String Functions.

Replacing

```
change(string, old, new {, count {, start{}})
swap(string, old, new {, count {, start{}})
substitute(array, old.list, new.list {, delimiter{}})
convert(from.characters, to.characters, string)
convert from.characters to to.characters in variable
```

`change()` and `swap()` are the same function — the compiler emits the same operation for both. Use whichever reads better; there is no behavioural difference to choose between.

The fourth and fifth arguments are frequently misread:

<i>count</i>	how many occurrences to change , not which one
<i>start</i>	which occurrence to start at

EXPRESSION ON <code>'BANANA'</code>	RESULT	
<code>change('banana', 'a', 'X')</code>	<code>bXnXnX</code>	all of them
<code>change('banana', 'a', 'X', 2)</code>	<code>bXnXna</code>	the first two
<code>change('banana', 'a', 'X', 1, 2)</code>	<code>banXna</code>	one, starting at the second
<code>change('banana', 'a', '')</code>	<code>bnn</code>	an empty replacement deletes

`CONVERT()` WORKS ON SINGLE CHARACTERS, AND DELETES

`convert()` maps each character of *from.characters* to the character in the same position of *to.characters*.

EXPRESSION	RESULT	
<code>convert('abc', 'xyz', 'aabbcc')</code>	<code>xyyyzz</code>	$a \rightarrow x, b \rightarrow y, c \rightarrow z$
<code>convert('abc', 'x', 'aabbcc')</code>	<code>xx</code>	b and c were deleted

A character with no opposite number is removed, not left alone. If the two lists are not the same length, every character in the excess of *from.characters* is deleted from the string. This is the documented Pick behaviour and it is almost never what someone writing `convert(',', '', text)` expects to happen to the rest of their data — though it is exactly what they want when deleting is the intent.

`convert` also exists as a **statement**, which modifies a variable in place rather than returning a value:

```
convert ',', ' to ';' in line
```

SUBSTITUTE()

`substitute(array, old.list, new.list {, delimiter})` replaces whole **elements** of a dynamic array rather than characters within them, matching each element against *old.list*. Substituting `b` with `Z` in the values `a`, `b`, `c` gives `a`, `Z`, `c`.

Wrapping text into lines

```
fold(string, width {, delimiter})
```

Breaks *string* into lines of at most *width* characters, **breaking at spaces where it can** and hard-breaking where a single word is too long.

```
fold('the quick brown fox jumps over the lazy dog', 12, '|')
```

gives

```
the quick|brown fox|jumps over|the lazy dog
```

The two-argument and three-argument forms use different default separators, which is not an obvious thing to guess.

CALL	SEPARATOR
<code>fold(s, 12)</code>	field marks — four fields
<code>fold(s, 12, '')</code>	value marks — four values
<code>fold(s, 12, sep)</code>	whatever <i>sep</i> holds — its first character only

So passing an explicit empty delimiter is **not** the same as omitting it. If the mark level matters, give the delimiter explicitly.

The width may itself be a dynamic array, giving successive lines different lengths and reusing the last value once the list runs out:

```
fold(s, 5 : @vm : 20, '|')
```

gives `the|quick brown fox|jumps over the lazy|dog` — a five-character first line, then twenty. This is how you wrap a paragraph around an indent.

Comparing and testing

```
compare(string1, string2 {, justification})
alpha(string)
```

`compare()` returns `-1` if *string1* sorts first, `1` if *string2* does, and `0` if they are equal.

EXPRESSION	RESULT
<code>compare('abc', 'abd')</code>	<code>-1</code>
<code>compare('abd', 'abc')</code>	<code>1</code>
<code>compare('abc', 'abc')</code>	<code>0</code>
<code>compare('B', 'a')</code>	<code>-1</code> upper case sorts before lower — byte order

EXPRESSION	RESULT
<code>compare('a2', 'a10')</code>	<code>1</code> plain text order: <code>2</code> after <code>1</code>
<code>compare('a2', 'a10', 'r')</code>	<code>-1</code> right-justified: the numbers compare as numbers

The `r` justification is how you sort `item2` before `item10`. Without it, embedded numbers compare character by character and `10` sorts before `9`. `1` is the explicit left-justified form and is the default.

`alpha()` is true if every character is a letter.

EXPRESSION	RESULT
<code>alpha('abc')</code>	true
<code>alpha('ab1')</code>	false
<code>alpha('')</code>	false

`alpha('')` is false and `num('')` is true. The two tests disagree about the empty string, so a validation that uses both needs to say which answer it wants for an empty field rather than assuming they behave alike. `num()` is covered in SD Basic - Math Functions.

Pattern matching

```
string matches pattern
matchfield(string, pattern, element)
```

`matches` is an operator and returns true or false. `matchfield()` returns the part of the string that matched a given element of the pattern.

THE PATTERN LANGUAGE

ELEMENT	MATCHES
<code>na*</code>	exactly <i>n</i> alphabetic characters
<code>nn*</code>	exactly <i>n</i> digits
<code>nx*</code>	exactly <i>n</i> characters of any kind

ELEMENT	MATCHES
0a · 0n · 0x	any number, including none
$n - ma \cdot n - mn \cdot n - mx^*$	between n and m characters
"literal"	the quoted text, as it stands
$\sim$ <i>element</i>	anything the element would not match

Elements are written one after another with no separator.

HOW THE CODES COMBINE

TEST	RESULT
'abc' matches '3a'	true
'ab12' matches '2a2n'	true
'abc' matches '0a'	true
' ' matches '0a'	true
'ab' matches '3x'	false
'12' matches '2-4n'	true
'1' matches '2-4n'	false
'12345' matches '2-4n'	false
'abc' matches '~3n'	true
'123' matches '~3n'	false
'ab-12' matches '2a"- "2n'	true
'ab12' matches '0a0n'	true
'a1b2' matches '0a0n'	false

A pattern may be a dynamic array, and the values are alternatives. 'ab' matches ('2a' : @vm : '2n') is **true** — the string need only match one of them. This is how you accept several formats without writing the test three times:

```
valid = code matches ('3n' : @vm : '2a4n' : @vm : '"X"5n')
```

PICKING OUT THE PARTS

`matchfield(string, pattern, element)` returns the text that matched the *element*-th part of the pattern. On 'ab12' against '2a2n':

<code>matchfield('ab12', '2a2n', 1)</code>	<code>ab</code>
--	-----------------

<code>matchfield('ab12', '2a2n', 2)</code>	<code>12</code>
--	-----------------

That makes a pattern a parser as well as a test — validate with `matches`, then pull the pieces out with `matchfield()` rather than counting character positions by hand.

Quoting

```
quote(expr)
dquote(expr)
squote(expr)
```

<code>quote()</code> · <code>dquote()</code>	wrap in double quotes — <code>"a"</code>
--	--

<code>squote()</code>	wrap in single quotes — <code>'a'</code>
-----------------------	--

`quote()` and `dquote()` are the same function — the compiler emits the same operation for both, so there is nothing to choose between them.

For CSV output do not build quoting by hand: `csvdq()` and the CSV statements handle embedded quotes and separators to RFC 4180, and are covered in SD Basic - CSV Files.

Phonetic matching

```
soundex(string)
```

Returns a four-character code that is the same for words that sound alike, for finding a name someone has spelled by ear.

EXPRESSION	RESULT
<code>soundex('Smith')</code>	<code>S530</code>
<code>soundex('Smyth')</code>	<code>S530</code>
<code>soundex('Robert')</code>	<code>R163</code>
<code>soundex('Rupert')</code>	<code>R163</code>
<code>soundex('O Brien 123')</code>	<code>0165</code> — non-letters are ignored
<code>soundex('')</code>	<code>0000</code>

`soundex('')` returns `0000`, not an empty string. A blank name therefore gets a code, and every blank name gets the *same* code — so a search keyed on soundex will match all of them together unless you exclude empty input first.

Soundex is tuned for English and does poorly on names of other origins. It is a way to narrow a search, not to decide a match.

Applying a function to every element

Most functions here have a partner ending in `s` that applies the same operation to every element of a dynamic array:

<code>lens('one' : @vm : 'three')</code>	<code>3</code> and <code>5</code>
<code>counts('a-b' : @vm : 'c-d-e', '-')</code>	<code>1</code> and <code>2</code>
<code>fields('a-b' : @vm : 'c-d-e', '-', 1)</code>	<code>a</code> and <code>c</code>
<code>trims(' x ' : @vm : ' y ')</code>	<code>x</code> and <code>y</code>
<code>indexs('Ab' : @vm : 'cD', 'b', 1)</code>	<code>2</code> and <code>0</code>
<code>soundexs('Smith' : @vm : 'Smyth')</code>	<code>S530</code> and <code>S530</code>

`spaces(2 : @vm : 3)`two spaces and three

The full family, and the rules for how they pair elements when given two arrays, is in SD Basic - Dynamic Arrays.

What is not here

`encrypt()` · `decrypt()`**removed** — replaced by `sdencrypt()` and `sddecrypt()`

Nothing else in the string-handling group has been removed. `dparse` and `dparse.csv` are statements rather than functions and are covered in SD Basic - Data Conversion and SD Basic - CSV Files; `iconv()`, `oconv()`, `fmt()`, `char()`, `seq()`, `ascii()` and `ebcdic()` are in SD Basic - Data Conversion; `extract()`, `insert()`, `delete()`, `replace()`, `locate`, `remove`, `raise()`, `lower()`, `vslice()` and `dcount()` used against marks are in SD Basic - Dynamic Arrays.

See also

SD Basic - Dynamic Arrays · SD Basic - Data Conversion · SD Basic - Math Functions · SD Basic - CSV Files.

SD Basic - Dynamic Arrays

Fields, values and subvalues - reading, changing and searching the structure that holds every SD record.

A dynamic array is a string with structure in it. Three mark characters divide it into **fields**, **values** and **subvalues**, and every record SD reads or writes is one. This page covers the statements and functions that work on that structure.

The functions that treat the same string as plain text are in SD Basic - String Functions; the two pages overlap where a function is used both ways, and each says so.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

In the examples below, `^` stands for a field mark, `|` for a value mark and `\` for a subvalue mark.

The marks

	NAME	CONSTANT	CODE
<code>^</code>	field mark	<code>@fm</code>	254
<code> </code>	value mark	<code>@vm</code>	253
<code>\</code>	subvalue mark	<code>@sm</code>	252

They nest: a field contains values, a value contains subvalues. There is a fourth, `@tm` (text mark, 251), used by `fmt()` and by some conversions, and `@im` (item mark, 255), which SD uses internally.

```
a = 'f1' : @fm : 'v1' : @vm : 'v2' : @fm : 'f3'
```

gives `f1^v1|v2^f3` — three fields, the second of which holds two values.

The marks are ordinary characters and they are counted. `len(a)` is `11` for the array above — eight data characters and three marks. Everything in SD Basic - String Functions sees them.

```
dcount(string, delimiter)
```

counts the pieces a delimiter divides a string into. On the array above, `dcount(a, @fm)` is 3 and `dcount(a<2>, @vm)` is 2.

`dcount()` of an empty string is 0, not 1. That is what makes `for i = 1 to dcount(rec, @fm)` correct — an empty record does nothing.

Reading a piece

```
array<field {, value {, subvalue}}>  
extract(array, field {, value {, subvalue}})
```

The angle-bracket form and `extract()` are the same operation.

EXPRESSION ON	F1^V1\ V2^F3	RESULT
a<2>		v1\ v2 — the whole field, marks and all
a<2,2>		v2
a<2,1,1>		v1
extract(a, 2, 1)		v1
a<9>		empty

Reading past the end is not an error. `a<9>` on a three-field array returns the null string, exactly as a genuinely empty field would. **There is no way to tell "field 9 is empty" from "there is no field 9"** — use `dcount()` if the difference matters.

Asking for a value of a field that has none returns the whole field: `a<1,1>` is `f1`.

Changing a piece

```
array<field {, value {, subvalue}}> = expression
replace(array, field, value, subvalue, expression)
insert(array, field, value, subvalue, expression)
delete(array, field {, value {, subvalue}})
```

EXPRESSION ON	RESULT
<code>b<2,2> = 'NEW'</code>	<code>f1^v1\ NEW^f3</code>
<code>replace(a, 2, 2, 0, 'R')</code>	<code>f1^v1\ R^f3</code>
<code>insert(a, 2, 2, 0, 'I')</code>	<code>f1^v1\ I\ v2^f3</code> — pushes the rest along
<code>delete(a, 2, 1)</code>	<code>f1^v2^f3</code> — removes one value
<code>delete(a, 2)</code>	<code>f1^f3</code> — removes the whole field

The assignment form is what almost all code uses. `replace()` and `insert()` exist for when you need the result as an expression rather than assigned back.

`replace()` and `insert()` count their arguments greedily, and the short forms need a semicolon.

`replace(a, 2, 2, 'R')` **does not compile** — the parser takes `2` as the value, `'R'` as the subvalue, and then demands the replacement string it never got. Either give all five arguments, or separate the final one with a semicolon.

CALL	RESULT
<code>replace(a, 2, 2, 0, 'R')</code>	<code>f1^v1\ R^f3</code>
<code>replace(a, 2; 'S')</code>	<code>f1^S^f3</code> — replaces the whole field
<code>replace(a, 2, 2; 'T')</code>	<code>f1^v1\ T^f3</code>
<code>replace(a, 2, 2, 'R')</code>	compile error

APPENDING

A subscript of `-1` appends:

RESULT

`b<-1> = 'APP'`

`f1^v1\|v2^f3^APP` — a new field at the end

`b<2, -1> = 'AV'`

`f1^v1\|v2\|AV^f3` — a new value in field 2

Writing past the end pads with empty fields rather than failing. `insert(a, 9, 0, 0, 'FAR')` on a three-field array gives `f1^v1|v2^f3^^^^^FAR` — five empty fields appear to fill the gap. A loop with an off-by-one index therefore grows the record silently instead of raising anything.

THE STATEMENT FORMS

```
del array<field {, value {, subvalue}}>
ins string before array<field {, value {, subvalue}}>
```

`del` and `ins` change the variable in place. `del b<2>` gives `f1^f3`, and `ins 'X' before b<2>` gives `f1^X^v1|v2^f3`.

Searching

LOCATE

```
locate expression in array<field {, value {, subvalue}}> {by order} setting variable
  then statements
else statements

locate(expression, array, field {, value {, subvalue}})
```

`locate` finds a whole element and reports its position. When it does not find one, `setting` receives **the position where it would be inserted** to keep the order — which is what makes it the standard way to add to a sorted list.

The number of subscripts chooses the level searched. this is the thing people get wrong. With `vals` holding `apple|cherry|damson` as three values in one field:

STATEMENT

RESULT

`locate 'cherry' in vals<1> setting p`

miss, insert at 2

`locate 'cherry' in vals<1,1> setting p`

found at 2

`array<f>` searches the **fields**, starting at field *f*. `array<f,v>` searches the **values of field f**, starting at value *v*. `array<f,v,s>` searches subvalues. So a single subscript on an array of values finds nothing, silently, and reports an insertion point — which looks like a working search right up until the data has two fields.

The statement form requires the angle brackets. `locate 'x' in vals setting p` does not compile: *"Field reference not found where expected"*.

The starting subscript really does start there. With `vals<1,2>`, `locate 'apple'` — which is value 1 — **misses**, and reports an insertion point of 4.

The function form takes the position as arguments instead, and returns the position or zero:

CALL	RESULT
<code>locate('cherry', vals, 1)</code>	<code>0</code> — searched fields
<code>locate('cherry', vals, 1, 1)</code>	<code>2</code> — searched values
<code>locate('zebra', vals, 1, 1)</code>	<code>0</code>

ORDER CODES

Without `by`, `locate` scans the whole list and the insertion point is the end. With `by`, it assumes the list is already sorted that way, stops as soon as it passes the place the value would go, and reports that place.

CODE	ORDER
al	ascending, left-justified — ordinary text order
ar	ascending, right-justified — numbers compare as numbers
dl	descending, left-justified
dr	descending, right-justified

For `apple|cherry|damson`:

SEARCH	INSERT AT
<code>'aardvark' by 'al'</code>	<code>1</code>
<code>'banana' by 'al'</code>	<code>2</code>
<code>'zebra' by 'al'</code>	<code>4</code>

And the justification is not a detail. For the values `2`, `10`, `30`, looking for `9`:

INSERT AT

by 'a1'	4 — as text, 9 sorts after 30
by 'ar'	2 — as numbers, between 2 and 10

Use `ar` for anything numeric. `a1` on numbers produces a list that looks sorted in short test data and comes apart at ten items.

`by` on a list that is *not* actually sorted that way stops early and reports a miss for a value that is present.

FIND AND FINDSTR

```
find string in array {, occurrence} setting field {, value {, subvalue}}
findstr string in array {, occurrence} setting field {, value {, subvalue}}
```

`find` matches a **whole element**; `findstr` matches **any element containing the string**. Both report the position as separate field, value and subvalue numbers rather than one index.

For `p^q|r`:

	RESULT
find 'r' in d setting f, v, s	field 2, value 2, subvalue 1
findstr 'q' in d setting f, v	field 2, value 1

Continued in

SD Basic - Operating on Whole Arrays — walking an array, mark levels, the s functions, logical functions, sorting and mark mapping.

SD Basic - Operating on Whole Arrays

Walking an array, changing the mark level, the s functions, sorting and mark mapping.

This page continues SD Basic - Dynamic Arrays.

Walking an array

```
remove variable from array setting delimiter
```

`remove` takes the next element off the front of *array* and reports which mark followed it. It is much faster than repeated `extract()` on a long record, because it does not rescan from the start each time.

The delimiter code is the mark's level, and zero means the end. For `p^q|r\s`:

CODE	MEANING
<code>0</code>	end of the string — this was the last element
<code>2</code>	a field mark followed
<code>3</code>	a value mark followed
<code>4</code>	a subvalue mark followed

```
s = record
loop
  remove element from s setting mark
  ...
while mark
repeat
```

`remove` **consumes the variable it reads.** Work on a copy — `s = record` — or the record is gone by the end of the loop. The functions that follow depend on an internal pointer into that variable:

`getrem(string)`how far through the string the pointer has reached

`setrem offset on string`move the pointer

After one `remove` from `aaa^bbb`, `getrem()` is `4`.

Changing the string resets the pointer. A loop that both `remove`s from a string and assigns to it will restart from the beginning, silently.

Changing the mark level

```
raise(string)
lower(string)
```

`raise` promotes every mark one level — subvalues become values, values become fields. `lower` demotes them. `lower('x' : @fm : 'y')` has one field and two values; `raise('x' : @vm : 'y')` has two fields.

These are how you move a nested structure between levels without walking it — for example turning a multivalued field into a record.

Working across an array

`VSLICE`

```
vslice(array, value.number)
```

Takes the n -th value of **every** field and returns them as fields. On `a1|a2^b1|b2`, `vslice(h, 2)` gives `a2^b2`.

That is the idiom for pulling one column out of a set of associated multivalued fields.

`SUBSTRINGS`, `FIELDSTORE` AND `SPLICE`

`substrings(array, start, length)`the substring operator applied to every element

```
fieldstore(string, delimiter, start, count, replacement)
```

replace or insert delimited fields — a *count* of zero inserts

```
splice(array1, string, array2)
```

joins the arrays element by element with *string* between

```
splice('a' : @vm : 'b', '-', '1' : @vm : '2') gives a-1|b-2.
```

SUBSTITUTE

```
substitute(array, old.list, new.list {, delimiter})
```

Replaces whole **elements** that match. Substituting `b` with `z` in the values `a`, `b`, `c` gives `a|z|c`.

REUSE

```
reuse(expression)
```

Makes a single value behave as though it repeated for every element of the other operand. **Without it, arithmetic only reaches the first element.**

EXPRESSION

RESULT

```
(1\|2\|3) + 10
```

```
11\|2\|3
```

```
(1\|2\|3) + reuse(10)
```

```
11\|12\|13
```

The first line is almost never what was intended, and it produces a plausible number rather than an error.

The `s` functions

Most string and arithmetic functions have a partner ending in `s` that applies the operation to every element instead of to the whole string: `abss()`, `negs()`, `mods()`, `nums()`, `lens()`, `counts()`, `fields()`, `trims()`, `trimbs()`, `trimfs()`, `indexs()`, `soundexs()`, `spaces()`, `strs()`, `fmts()`, `iconvs()`, `oconvs()`, `folds()`, `cats()`.

When two Arrays are different lengths, the shorter is treated as empty, not truncated. With `1|2|3` and `10|20`:

CALL	RESULT	
<code>cats(p, q)</code>	<code>110\ 220\ 3</code>	the third pairs with nothing
<code>mods(p, q)</code>	<code>1\ 2\ 3</code>	<code>mod(3, 0)</code> is <code>3</code> — no error

So a mismatch produces a full-length answer with quietly wrong elements in the tail. **Compare** `dcount()` **on both operands before relying on the result.**

Logical functions

These take true/false values element by element. In SD, zero and the null string are false and anything else is true.

<code>not(expression)</code>	logical negation of a single value
<code>nots(array)</code>	of every element
<code>ands(a, b)</code> · <code>ors(a, b)</code>	element-by-element AND and OR
<code>eqs</code> · <code>nes</code> · <code>gts</code> · <code>ges</code> · <code>lts</code> · <code>les</code>	element-by-element comparison, giving 1 or 0
<code>ifs(control, true.values, false.values)</code>	picks from one array or the other per element

CALL	RESULT
<code>not(0)</code> , <code>not(1)</code> , <code>not('')</code>	<code>1</code> , <code>0</code> , <code>1</code>
<code>nots(1\ 0\ '')</code>	<code>0\ 1\ 1</code>
<code>ands(1\ 0\ 1, 1\ 1\ 0)</code>	<code>1\ 0\ 0</code>
<code>ors(1\ 0\ 0, 0\ 1\ 0)</code>	<code>1\ 1\ 0</code>
<code>eqs(1\ 2\ 3, 1\ 20\ 3)</code>	<code>1\ 0\ 1</code>
<code>ifs(1\ 0, 'yes'\ 'yes', 'no'\ 'no')</code>	<code>yes\ no</code>

Sorting

```
sortinit {order}  
sortadd key, data  
sortclear
```

`sortinit` starts a sort, `sortadd` feeds it a key and a record, and reading it back drains it. The order codes are the same `a1` / `ar` / `d1` / `dr` as `locate`. For sorting a whole file, `sselect` is faster — see SD Basic - Select Lists.

Mark mapping

```
mark.mapping file.variable, on | off | expression
```

Controls whether SD translates mark characters when reading and writing a **directory** file, whose records are ordinary operating-system files. With mapping on, field marks become newlines on the way out and back again on the way in; with it off, the bytes pass through unchanged.

It has no effect on a dynamic file, where records are stored in SD's own format.

What is not here

Nothing in the dynamic-array group has been removed from this port.

`dcount()`, `fieldstore()`, `substrings()`, `splice()`, `substitute()` and the whole `s` family are also described in SD Basic - String Functions, where they are used on text rather than on structure.

See also

SD Basic - String Functions · SD Basic - File Handling · SD Basic - Select Lists · SD Basic - Math Functions.

SD Basic - Data Conversion

Dates, times, numbers and text between the form SD stores and the form people read.

SD stores a date as a day number and a time as a second count. Neither is readable, and neither is what a user types. `iconv()` turns what a person wrote into what SD stores; `oconv()` turns it back. `fmt()` lays a value out in a fixed width. This page covers all three, and the character-level conversions beside them.

SD folds case, so a program may be written in either case. Keywords and conversion codes are shown here in lower case; **the codes themselves are not case sensitive**. In the tables, *italics* mark something you supply.

The two directions

```
iconv(expression, conversion)
oconv(expression, conversion)
```

<code>iconv()</code>	in — external form to internal. Use it on input
----------------------	--

<code>oconv()</code>	out — internal to external. Use it on output
----------------------	---

`iconvs()` and `oconvs()` do the same to every element of a dynamic array.

A failed `iconv()` returns the null string and sets `status()`. `iconv('nonsense', 'D')` returns empty and `status()` afterwards is 1.

```
d = iconv(typed.date, 'D')
if status() then
  print 'That is not a date.'
end
```

This is the validation idiom, and the return value alone is not enough — a user who typed nothing also gets an empty result, so testing the result cannot tell an empty input from a bad one. `status()` can.

Dates

```
iconv(text, 'D')
oconv(daynumber, 'D{n}{separator}{format}')
```

Day zero is 31 December 1967. `oconv(0, 'D4/')` is `12/31/1967`. Dates before that are negative. `26 AUG 2026` is day **21423**.

CONVERSION	RESULT FOR DAY 21423
<code>oconv(d, 'D')</code>	<code>26 AUG 2026</code>
<code>oconv(d, 'D2/')</code>	<code>08/26/26</code>
<code>oconv(d, 'D4-')</code>	<code>08-26-2026</code>
<code>oconv(d, 'DMA')</code>	<code>AUGUST</code>
<code>oconv(d, 'DW')</code>	<code>3</code> — day of the week
<code>oconv(d, 'DWA')</code>	<code>WEDNESDAY</code>
<code>oconv(d, 'DQ')</code>	<code>3</code> — quarter
<code>oconv(d, 'DY')</code>	<code>2026</code>

The digit after `D` is how many digits of year to show; the character after that is the separator.

The default order is month, day, year. `D2/` gives `08/26/26` for the 26th of August. **E** puts it in **day-month-year order**: `D2/E` gives `26/08/26` and `D4/E` gives `26/08/2026`. A site that expects day-first must say **E** on **every** conversion; a report that mixes the two is a formatting bug that reads as a data bug.

`iconv()` accepts a wide range of typed forms — `26 AUG 2026`, `08/26/2026`, `26-8-26` — and resolves a two-digit year against a sliding window. **Always check** `status()`: `13/13/26` is not a date and will come back empty.

Times

```
iconv(text, 'MT')
```

```
oconv(seconds, 'MT{H}{S}')
```

A time is the number of seconds since midnight. `14:30:05` is **52205**.

CONVERSION	RESULT
<code>oconv(t, 'MT')</code>	<code>14:30</code>
<code>oconv(t, 'MTS')</code>	<code>14:30:05</code>
<code>oconv(t, 'MTH')</code>	<code>02:30pm</code>
<code>oconv(t, 'MTHS')</code>	<code>02:30:05pm</code>

`s` adds seconds, `h` switches to a 12-hour clock. `date()` and `time()` give today's day number and the current second count; `timedate()` gives both, already formatted.

Numbers

SCALING AND SEPARATORS

```
oconv(number, 'MD{n}{separator}')
```

`md` n moves the decimal point n places **left**, so the stored value is an integer and the displayed one has decimals.

CONVERSION	RESULT
<code>oconv(1234567, 'MD2')</code>	<code>12345.67</code>
<code>oconv(1234567, 'MD2,')</code>	<code>12,345.67</code>
<code>oconv(1234567, 'MD0,')</code>	<code>1,234,567</code>
<code>oconv(-42, 'MD2')</code>	<code>-0.42</code>

MD2 divides by 100 — it does not round to two places. `oconv(-42, 'MD2')` is `-0.42`, not `-42.00`. It is for money held as whole pence or cents. **To show two decimals of a value that is already scaled, use `fmt()` with a `2` after the justification.**

OTHER BASES

CONVERSION	RESULT
<code>oconv(255, 'MX')</code>	<code>FF</code>
<code>iconv('FF', 'MX')</code>	<code>255</code>
<code>oconv(5, 'MB')</code>	<code>101</code>
<code>oconv(8, 'MO')</code>	<code>10</code>

`mx` hexadecimal, `mb` binary, `mo` octal. `xtd()` and `dtx()` do the same for hex without going through `oconv()` — see *Characters and codes* below.

SELECTING CHARACTERS

CONVERSION	RESULT
<code>oconv('abc', 'MCU')</code>	<code>ABC</code>
<code>oconv('ABC', 'MCL')</code>	<code>abc</code>
<code>oconv('hello world', 'MCT')</code>	<code>Hello World</code>
<code>oconv('a1b2', 'MCN')</code>	<code>12</code> — digits only
<code>oconv('a1b2', 'MCA')</code>	<code>ab</code> — letters only

`mcn` and `mca` **discard** everything else rather than replacing it, which makes them a quick way to strip formatting out of a typed reference.

Laying a value out: `fmt()`

```
fmt(expression, specification)
fmts(array, specification)
```

The specification is a width, a justification, and optional extras.

CALL	RESULT
<code>fmt('ab', '10L')</code>	<code>ab_</code> — left, padded to 10

CALL	RESULT
<code>fmt('ab', '10R')</code>	<code>_____ab</code> — right
<code>fmt('ab', '10*R')</code>	<code>*****ab</code> — padded with <code>*</code>
<code>fmt(1234.5, '12R2')</code>	<code>_____1234.50</code> — two decimals, right

The character before the justification letter is the fill character; a digit after it is the number of decimal places.

A value longer than the width is wrapped, not truncated. `fmt('abcdefgh', '4L')` returns `abcd` and `efgh` separated by a **text mark**, not `abcd`. A field written straight to a screen or a file therefore gains a stray character rather than being cut short. **To truncate, take a substring first** — `fmt(s[1,4], '4L')`.

THE FORMAT QUALIFIER

The same thing can be written by placing the specification **straight after the value, with nothing between them**:

```
print total '10r2'
```

With `t` holding `1234.5`, `t '10R2'` gives `_____1234.50`. **There is no `fmt` keyword in this form** — writing `t fmt '10R2'` compiles `fmt` as a variable name and aborts at run time with *"Unassigned variable FMT"*. The qualifier is recognised by juxtaposition alone, which is why it is allowed only where an expression cannot be followed by something else.

Characters and codes

CALL	RESULT
<code>char(65)</code>	<code>A</code>
<code>seq('A')</code>	<code>65</code>
<code>seq('')</code>	<code>0</code>
<code>xta('FF')</code>	<code>255</code>
<code>dtx(255)</code>	<code>ff</code>
<code>dtx(255, 4)</code>	<code>00ff</code>

<code>char()</code>	the character with the given code
<code>seq()</code>	the code of the first character. <code>seq('')</code> is <code>0</code> , the same as <code>seq(char(0))</code>
<code>xtd()</code>	hexadecimal text to a number
<code>dtx()</code>	a number to hexadecimal text, in lower case , optionally zero-padded to a width
<code>ascii()</code> · <code>ebcdic()</code>	translate a string between the two character sets

`ascii(char(193))` is `A` — EBCDIC 193 is the letter A.

`char()` **is how you write a mark character or a control code as a constant**, and `equate` is the place to put it:

```
equate esc to char(27)
equate fm to char(254)
```

Splitting a delimited string

```
dparse string, delimiter, var1, var2, ...
```

Splits *string* on *delimiter* and assigns the pieces to the named variables in order. It is the fixed-shape alternative to a `remove` loop: use it when you know how many pieces there are and want them in named variables rather than an array.

Missing pieces leave their variables empty; extra pieces are discarded.

`dparse.csv` does the same for a CSV line, honouring quoting — see SD Basic - CSV Files.

Dictionary conversions

```
itype(dictionary.record)
```

Evaluates a dictionary item — an I-type expression — against the record currently being processed, and returns what it computes. It is how a program uses the same derived field a query would, instead of duplicating the formula.

`trans()`, `rtrans()` and `xlate()` fetch a field from another file and can apply that file's own conversion; they are covered in SD Basic - File Handling because they open a file to do it.

National settings

```
getnls(key)
setnls key, value
```

Report and set the national-language settings — currency symbol, thousands and decimal separators, date order. `setnls` **changes them for the session**, so a program that alters them and then aborts leaves the session changed. Read the old value, set the new, and restore it on the way out.

What is not here

Nothing in the conversion group has been removed from this port.

`fmt()`, `char()`, `seq()`, `ascii()`, `ebcdic()` and the `convert` statement are also mentioned in SD Basic - String Functions, where they are used on text rather than for conversion. `precision`, which decides how many decimal places a floating-point number shows when it becomes a string, is in SD Basic - Math Functions.

See also

SD Basic - String Functions · SD Basic - Math Functions · SD Basic - File Handling · SD Basic - CSV Files.

SD Basic - File Handling

Opening files, reading and writing records, locking, and transactions.

This page covers the statements that reach the database: opening a file, reading and writing whole records or single fields, the locking that keeps two sessions out of each other's way, and transactions.

Sequential files — ordinary operating-system files read a line at a time — are in SD Basic - Sequential Files. Select lists are in SD Basic - Select Lists.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Opening

```
open {dict,} file.name to file.variable {readonly}
  then statements
else statements

openpath pathname to file.variable {readonly}
  then statements
else statements
```

`open` looks the name up in the account's VOC. `openpath` takes an operating system path and bypasses the VOC entirely.

The `else` branch is where a missing file arrives, and it is not optional. A program that omits it does not compile. Never leave it empty:

```
open 'CUSTOMERS' to f.cust else
  stop 'Cannot open CUSTOMERS'
end
```

`dict` opens the dictionary rather than the data portion — `open 'dict', 'CUSTOMERS' to f.dict else ...`.

`close file.variable` releases it. A file left open is closed when the program ends, so `close` matters mainly in a long-running program that opens files in a loop.

Reading

```
read variable from file.variable, record.id then ... else ...
readv variable from file.variable, record.id, field then ... else ...
matread matrix from file.variable, record.id then ... else ...
```

<code>read</code>	the whole record into one dynamic array
<code>readv</code>	a single field
<code>matread</code>	the record spread across a dimensioned matrix, one field per element

For a record holding `alpha` and `beta|gamma`:

RESULT

<code>read got from f, 'R1'</code>	<code>then</code> branch; <code>dcount(got, @fm)</code> is <code>2</code>
<code>read got from f, 'NOSUCH'</code>	<code>else</code> branch
<code>readv fv from f, 'R1', 2</code>	<code>beta\ gamma</code>

A missing record is the `else` branch, not an error. And `status()` after one reads **3006**, which is how you tell "no such record" from a real failure such as a permission problem.

`matread` **puts everything past the end of the matrix into element zero.** A three-field record read into `dim mm(2)` gives `mm(1) = f1`, `mm(2) = f2` and `mm(0) = f3`. Nothing is lost and nothing is reported.

`inmat()` **does not tell you it happened** — it reads `0` after that `matread`, not `3`. It reports the element count after `matparse`, not after `matread`. If a record may have more fields than your matrix, `read` it and use `dcount(rec, @fm)`, or size the matrix from the dictionary.

Writing

```
write variable to file.variable, record.id
writev variable to file.variable, record.id, field
matwrite matrix to file.variable, record.id
```

Writing creates the record if it does not exist and replaces it if it does. There is no "insert only" form — read first if that distinction matters, and hold a lock between the read and the write or two sessions will both find it absent.

`delete file.variable, record.id` removes a record. After `delete f, 'R2'`, reading `R2` takes the `else` branch.

`clearfile file.variable` empties the file but leaves it in place.

Locking

SD locks a **record**, not a row or a page, and the lock belongs to the session.

```
readu variable from file.variable, record.id {locked ...} then ... else ...
readvu variable from file.variable, record.id, field ...
matreadu matrix from file.variable, record.id ...
recordlocku file.variable, record.id
recordlockl file.variable, record.id
release {file.variable {, record.id}}
```

<code>readu</code>	read and take an update lock
<code>readl</code>	read and take a shared read lock
<code>recordlocku</code>	take an update lock without reading
<code>release</code>	give locks back

`write` releases the lock it finds. `release` with no arguments releases everything the session holds.

```
recordlocked(file.variable, record.id)
```

reports the state. After `readu`, `recordlocked(f, 'R1')` is **2**; an unlocked record reads **0**; after `release f, 'R1'` it is **0** again.

The `locked` clause is the only way not to wait. Without it, a `readu` against a record another session holds **blocks until that session lets go**. With it, control goes to the `locked` branch immediately:

```
readu rec from f, id locked
  print 'Someone else is editing that record.'
```

```
    return
end then
...
end else
...
end
```

A program without a `locked` clause is a program that can hang a user indefinitely with no message. **Any interactive read-for-update should have one.**

`filelock` and `fileunlock` lock a whole file, and `lock` / `unlock` take one of 64 numbered general-purpose semaphores — used to serialise something that is not a record at all, such as a report run.

The rest of this is its own page. What the other session sees, what every `recordlocked()` code means, how to find out **who** holds a lock, and how long a wait actually is, are all in SD Basic - Locks and Transactions, which covers what two sessions running at once see of each other.

Transactions

```
begin transaction
...
commit
...
rollback
end transaction
```

Everything written between `begin transaction` and `commit` either all lands or none of it does. `rollback` discards the lot.

Locks taken inside a transaction are held until it ends, whichever way it ends. That is what makes the transaction safe and also what makes a long one expensive — keep the span short, and never wait for user input inside one.

And inside a transaction you must already hold the lock on every record you write. The same `write` that works outside one fails inside it with *"Error 3023 (o/s 0) writing record (Possible full disk?)"*, which is a lock error wearing a disk error's message. SD Basic - Locks and Transactions has the detail, along with what `commit`, `rollback` and simply reaching `end transaction` each do.

Asking about a file

```
fileinfo(file.variable, key)
```

On a newly created dynamic file:

KEY	MEANING	RESULT
0	is this variable an open file?	1
1	the VOC name it was opened by	ZZWORK
2	the path on disk	/cygdrive/c/ProgramData/SD/user_accounts/don/ZZWORK
3	file type	3
5	modulus	
6	minimum modulus	1
7	group size	
11	current load percentage	
13	does the file have alternate key indexes?	
1001	read-only?	
1003	size in bytes, excluding indexes	

File types: 1 SH, 3 **DH — an ordinary dynamic file**, 4 directory, 5 sequential.

The PATH comes back in POSIX form, not as a Windows path — `/cygdrive/c/ProgramData/...`, not `C:\ProgramData\...`. **Handing that string to a Windows program does not work** — Windows reads it as a drive-relative path and either fails silently or reports that the parent directory does not exist. This is not a theoretical caution: it is what stopped the full-screen editors working the first time they were built for this port.

And the conversion function is not available to an ordinary program. The kernel can convert such a path, and `kernel()` is an internal-only intrinsic — a `kernel(...)` call in a user account does not compile, and the compiler's complaint is "*Matrix KERNEL is not referenced in a DIM statement*", reported at the last line of the program. See SD Basic - System and Environment. **So a path a Windows program is going to see should come from your own configuration, not from `fileinfo()`.**

There is no record-count key. Key `6` is the *minimum modulus* and reads `1` on a small file whatever it contains — `1` with two records present and `1` again after `clearfile`. To count records, `select` the file and read `selectinfo(list, 3)`, or count in a `readnext` loop.

`dir(pathname)` lists an operating-system directory as a dynamic array — name, size and modification time per entry.

Reaching another file's data

```
trans({dict} file.name, record.id, field, action)
rtrans({dict} file.name, record.id, field, action)
xlate({dict} file.name, record.id, field, action)
```

All three fetch one field of one record from a named file, opening it if needed. `trans` and `xlate` are the same function under two names; `rtrans` takes an already-open file variable instead of a name.

The *action* says what to do when the record is missing:

x	return the null string
v	return the null string and print an error
c	return the record id itself
n	return the record id itself

A *field* of `0` returns the record id.

These open a file on every call unless it is already open. In a loop over many records that is a real cost — `open` the file once and use `rtrans`, or read it directly.

Creating and configuring

```
create.file path {directory | dynamic}  
create file.variable then ... else ...  
configure.file file.variable, key, value
```

Most applications create files with the `create.file` **command** rather than the statement — from BASIC, `execute 'create.file ...'` is the usual route, and it is what the examples on this page use.

`set.trigger` attaches a program that runs on every write or delete to a file. A trigger runs inside the caller's transaction, so anything it does is part of the same all-or-nothing unit.

What is not here

Nothing in the file-handling group has been removed from this port, but two things about it changed:

VFS	the virtual file system layer has been removed from the C entirely . <code>fileinfo()</code> never reports a VFS type, and the type code is gone
------------	--

The data tree is private	<code>C:\ProgramData\SD</code> is protected by an access-control list. A file created by SD is reachable through SD, and not by an ordinary Windows user poking at the directory
---------------------------------	--

See also

SD Basic - Select Lists · SD Basic - Dynamic Arrays · SD Basic - Sequential Files · SD Basic - Alternate Key Indexes.

SD Basic - Select Lists

Building, walking, saving and combining lists of record ids.

A select list is a list of record ids held by the session. Building one and reading it back is how a program processes a whole file, and how it picks up the result of a query run at the command line.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The eleven lists

There are **eleven** select lists, numbered `0` to `10`. List `0` is the default: every statement that takes a list number uses it when you do not say otherwise, and it is the one a query at the command line leaves behind.

A list is session state, not program state. It survives the program that made it, and a program that leaves one behind changes what the next command does. **Clear a list you built for your own use** — see `clearselect` below.

Building a list

```
select file.variable {to list} {on error statements}
sselect file.variable {to list}
selectn file.variable {to list}
selectv file.variable to list.variable
selectindex index.name {, value} from file.variable {to list}
formlist array {to list}
```

<code>select</code>	every id in the file, in no useful order
<code>sselect</code>	every id, sorted
<code>selectn</code>	as <code>select</code> , but does not clear the list first

<code>selectv</code>	the list into a variable rather than a numbered list
<code>selectindex</code>	from an alternate key index — see SD Basic - Alternate Key Indexes
<code>formlist</code>	builds a list from a dynamic array you already have

On a file holding two records:

	RESULT
<code>select f</code> then a <code>readnext</code> loop	2 ids
<code>formlist 'X' : @fm : 'Y' : @fm : 'Z' to 7</code> then a loop	3 ids

`sselect` **sorts** and `select` **does not**, and `select` **is not in id order**. It returns ids in whatever order the file's hashing puts them, which changes as the file grows. **A report that must come out in order needs** `sselect`, or a query with a `by` clause; relying on `select` to be ordered produces output that is right on the developer's small test file and wrong in production.

Walking a list

```
readnext variable {from list} {on error statements}
  then statements
else statements

readnext variable, value.position, subvalue.position {from list}
```

`readnext` takes the next id and moves on. **The `else` branch is how the loop ends** — it fires when the list is exhausted.

```
select f
loop
  readnext id else exit
  read rec from f, id else continue
  ...
repeat
```

The exploded form — `readnext id, vpos, spos` — is for lists built from a multivalued index, where the same id appears once per matching value and the extra variables say which value matched.

A `readnext` loop and a `readu` inside it is the commonest deadlock. Two sessions walking the same file in different orders will each hold a record the other wants. Use `sselect` so both walk in the same order, and give the `readu` a `locked` clause so at least one of them gives up rather than waiting — see SD Basic - File Handling.

Asking about a list

```
selectinfo(list, key)
```

KEY	MEANING
1	is the list active?
3	how many entries it holds

After `select f to 3`, `selectinfo(3, 1)` is `1`; after `clearselect 3` it is `0`.

Key 3 is the only reliable way to count a file's records — `fileinfo()` has no record-count key. `select` the file and read `selectinfo(list, 3)`.

Clearing

```
clearselect {list}  
clearselect all
```

`clearselect` with no argument clears list 0. `clearselect all` clears all eleven.

Clear a list you no longer need, especially list 0: a program that returns to the command line leaving a list active makes the *next* command operate on that list instead of the whole file, which is a confusing thing to debug from the other end.

Saving a list

```
savelist name {from list} then ... else ...  
getlist name {to list} then ... else ...  
deletelist name
```

A saved list is a record in the account's `&SAVEDLISTS&` file and outlives the session.

A `savelist` of a two-id list followed by `getlist ... to 5` and a `readnext` loop on list 5 returned the same 2 ids.

`savelist` and `getlist` require a `then` or `else` clause, and the compiler's complaint does not say **so**. Written without one, the error is "*Expected THEN or ELSE*" reported against whatever follows — which reads like a problem with the next statement. `else null` is enough:

```
savelist list.name from 3 else null getlist list.name to 5 else stop 'no such list'
```

`formlist` and `clearselect` do **not** take a `then` / `else` clause, so the four statements are not consistent with each other.

Combining and comparing lists

```
selectleft index.name from file.variable  
selectright index.name from file.variable  
setleft index.name from file.variable  
setright index.name from file.variable
```

These walk an alternate key index in either direction from the current position, rather than building a whole list at once. `setleft` and `setright` position at the start or end without reading anything. They are covered with the rest of the index statements in SD Basic - Alternate Key Indexes.

```
listindex(list, delimiter, item)  
indices(file.variable {, index.name})
```

`listindex()` picks one entry out of a delimited list. `indices()` reports which alternate key indexes a file has, or the definition of one.

Lists as data

A list can be filtered in BASIC rather than by building a query string — which is usually clearer, and cannot be broken by a value containing a quote. `formlist` is the way back in.

```
select f to 2
keep = ''
loop
  readnext id from 2 else exit
  if id[1,3] = 'abc' then keep := id : @fm
repeat
keep = trim(keep, @fm, 'T')      ;* see the warning below
formlist keep to 3
```

`selectv` **does not give you a dynamic array**. It puts the list into a variable of its own kind: `vartype()` of a `selectv` target is **11**, a select list. Concatenating it or calling `dcount()` on it fails with *"Data cannot be converted to a string"*. **Read it with** `readnext variable from list.variable`, which returns the same three ids. `selectv` is for passing a list to a subroutine, not for inspecting one.

A trailing field mark becomes an extra, empty id. An array of three ids built with `keep := id : @fm` ends in a mark, and `formlist keep to 3` gives **four** entries — the fourth being the null string, which then reads a record that does not exist. **Strip the trailing mark**, or append the mark *before* each id except the first.

What is not here

Nothing in the select-list group has been removed from this port.

See also

SD Basic - File Handling · SD Basic - Alternate Key Indexes · SD Basic - Dynamic Arrays · SD Basic - Program Control.

SD Basic - Alternate Key Indexes

Building an index on a field, selecting through it, and the three steps that report success while doing nothing useful.

An alternate key index lets you find records by the contents of a field instead of by record id. This page covers creating one, keeping it, and reading it from BASIC.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Creating an index takes three steps, and skipping any of them is silent

This is the whole of this page's warning, so it is first. Each step reports success on its own, and a program that stops after the first or second gets an index that exists and finds nothing.

```
create.index file.name field.name          ;* at the command line
```

then **close the file in every session that has it open**, then

```
build.index file.name field.name
```

then reopen the file.

WHAT EACH STEP ACTUALLY DOES

A dynamic file with three records, `TOWN` being field 2, holding `LONDON`, `LEEDS` and `LONDON`:

<code>create.index ZZAK TOWN</code>	"Added index for TOWN", <code>@system.return.code</code> 0
<code>indices(f)</code> on the already-open file variable	empty
<code>fileinfo(f, 13)</code> on it	0 — no indexes
after <code>close f</code> and reopening	<code>indices(f2)</code> is <code>TOWN</code> , <code>fileinfo(f2, 13)</code> is 1

<code>selectindex 'TOWN', 'LONDON'</code> at that point	0 ids
<code>build.index</code> with the file still open	"Cannot gain exclusive access to file", <code>@system.return.code</code> 3021
<code>build.index</code> after <code>close</code>	"Building index 'TOWN'... 3 records processed / Populating index...", <code>@system.return.code</code> 0
<code>selectindex 'TOWN', 'LONDON'</code> after reopening	C1 and C3

An open file variable never learns about a new index. The handle carries the index list it had when it was opened. So a program that creates an index and then uses it in the same run finds nothing, and neither `indices()` nor `fileinfo()` gives it a hint — both report the state of the handle, not of the file. **Close and reopen.**

And `create.index` makes an empty index. It records the definition; it does not walk the existing records. Until `build.index` runs, the index is there, is reported by `indices()`, and matches nothing. A file that was indexed before it had data does not need building; one indexed afterwards always does.

`build.index` **needs exclusive access and your own session counts.** With the file open in the same program, it refuses with "Cannot gain exclusive access to file" and `@system.return.code` **3021**. **Test that return code** — the message goes to the capture variable, not to the screen, so a program that does not look at it carries on with an unbuilt index.

Once it is built, it maintains itself

After `build.index`, writing a fourth record with `TOWN` of `LONDON` made `selectindex 'TOWN', 'LONDON'` return **3** ids without any further action. Ordinary `write` and `delete` keep every index on the file up to date.

That is also the cost: each index makes every write a little more expensive. Index the fields you select on, not every field.

Selecting through an index

```
selectindex index.name {, value} from file.variable {to list}
```

With and without a value it returns completely different things, and neither form says so. On the file above:

```
selectindex 'TOWN', 'LONDON' from f to 4
```

the record ids — C1, C3

```
selectindex 'TOWN' from f to 4
```

the distinct index values — LEEDS, LONDON

So the no-value form is how you ask "*what towns are there?*" and the with-value form is how you ask "*who is in London?*". A program that drops the value argument by accident gets a list of values where it expected ids, reads them as record ids, and finds nothing — with no error at any point.

Walking an index in order

```
setleft index.name from file.variable
setright index.name from file.variable
selectleft index.name from file.variable {setting variable} {to list}
selectright index.name from file.variable {setting variable} {to list}
```

`setleft` positions before the first index value, `setright` after the last. `selectright` then moves to the next value and selects its ids; `selectleft` moves back.

`setting` **receives the index value the scan landed on**. After `setleft 'TOWN' from f`, a `selectright` returns **1 id** with `setting` holding `LEEDS` — the first value in order.

These four take no `then` or `else` clause. Writing one is a compile error — "*Unrecognised statement*" — because the parser expects `setting` or `to` and finds a keyword it has no use for. To detect the end of the index, read the select list they build: an empty one means there was nothing further.

Walking with `selectright` is how you produce a report in index order without sorting, and how you page through a large file a value at a time rather than selecting all of it.

Asking what indexes a file has

```
indices(file.variable)
indices(file.variable, index.name)
```

one argument	the index names, as a dynamic array
--------------	-------------------------------------

two arguments	the definition of one index
---------------	-----------------------------

`indices(f2, 'TOWN')` returns a **nine-field** definition whose field 2 is the **field number** the index is built on — `2` for a `TOWN` in field 2. Field 1 holds the dictionary-style definition the index was made from.

`fileinfo(file.variable, 13)` is the quick yes/no: `1` if the file has any index, `0` if not — subject to the open-handle caveat above.

Maintaining an index from BASIC

```
akread variable from file.variable, index.name, key then ... else ...
akwrite variable to file.variable, index.name, key
akdelete file.variable, index.name, key
akclear file.variable, index.name
akenable file.variable, index.name
akrelease file.variable, index.name
create.ak file.variable, index.name, ...
delete.ak file.variable, index.name
```

These reach into the index directly rather than letting SD maintain it. They exist for the case an ordinary index cannot express — an index whose key is computed by a program rather than taken from a field.

They are not the normal route and they will let you corrupt an index. Writing an index entry that does not match the record it points at produces a `selectindex` that returns ids whose records do not have the value you asked for. Nothing detects that except `build.index`, which rebuilds from the data and throws away whatever you put there. **Use the ordinary statements and let SD maintain the index unless you have a reason you can write down.**

`akenable` and `akrelease` turn maintenance off and on around a bulk load, which is the one common reason to touch these at all: loading a million records with three indexes live is far slower than loading them and building the indexes afterwards.

`listindex()`

```
listindex(list, delimiter, item)
```

Picks one entry out of a delimited list — it has nothing to do with alternate key indexes despite the name, and belongs with the list functions in SD Basic - Select Lists.

What is not here

Nothing in the index group has been removed from this port.

See also

[SD Basic - Select Lists](#) · [SD Basic - File Handling](#) · [SD Basic - Dynamic Arrays](#).

SD Basic - Sequential Files

Reading and writing ordinary operating-system files a line or a block at a time.

A sequential file is an ordinary file on disk, read and written a line at a time rather than a record at a time. It is how SD exchanges data with anything that is not SD — a log, an export, a file dropped in a folder by another system.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Opening

```
openseq file.name, record.id {append | overwrite | readonly} to file.variable
  then statements
else statements

openseq pathname {append | overwrite | readonly} to file.variable
  then ... else ...
```

The two-name form addresses a record inside a **directory** file — an SD file whose records are ordinary files in a folder. The one-name form takes an operating system path and reaches anything on the machine the account is allowed to read.

The `then` and `else` branches do not mean success and failure.

<code>then</code>	the file already existed
-------------------	---------------------------------

<code>else</code>	it did not exist, and has been created — or the open genuinely failed
-------------------	---

`openseq 'ZZDIR', 'LINES'` on a name that does not exist takes the `else` branch and gives a usable file variable.

So `else` is not an error branch, and a program that writes `stop 'cannot open'` there will refuse to create a file it was supposed to create. To tell "created" from "failed", test the variable — a failed open leaves it unusable and the next `writeseq` takes its `else` branch.

KEYWORD

append	position at the end, so writes add to what is there
---------------	---

overwrite	truncate on open
------------------	------------------

readonly	open without taking a write lock
-----------------	----------------------------------

Reading

```
readseq variable from file.variable {on error statements} then ... else ...
readblk variable from file.variable, bytes then ... else ...
```

<code>readseq</code>	one line, without its terminator
----------------------	---

<code>readblk</code>	a fixed number of bytes, terminators included
----------------------	---

`readseq`'s `else` branch is end of file. Three lines written and read back give `3` and the first as `line one`, and `status()` at end of file is **3006** — the same code a missing record gives in SD Basic - File Handling.

```
openseq 'EXPORT', 'DATA' to f else stop 'cannot open'
loop
  readseq line from f else exit
  ...
repeat
closeseq f
```

Writing

```
writeseq variable to file.variable then ... else ...
writeseqf variable to file.variable then ... else ...
writeblk variable to file.variable
weofseq file.variable
```

<code>writeseq</code>	write a line and a terminator
<code>writeseqf</code>	the same, then flush to disk
<code>writeblk</code>	write bytes with no terminator added
<code>weofseq</code>	truncate the file at the current position

`writeseq` **does not flush**. A program that writes a log and then aborts can lose the tail. `writeseqf` costs a disk write per line, which is why it is a separate statement — use it for anything that has to survive a crash, and `writeseq` plus a `flush` at the end for bulk output.

`weofseq` at the start of a file is how you empty it: `openseq` positions at the beginning, and truncating there discards everything.

THE LINE TERMINATOR IS CRLF

Two lines written with `writeseq` and read back with `readblk`, byte by byte:

bytes	65 66 13 10 67 68 13 10
meaning	A B CR LF C D CR LF

This port writes CRLF, not LF. That is deliberate — a file SD produces opens correctly in Notepad and every other Windows tool, which was not true of the Linux original. `readseq` accepts either, and strips whichever it finds, so a file written on Linux still reads correctly here.

It matters when you count bytes. A file of n lines is n bytes longer than the same file with LF endings, so an offset computed by adding up line lengths must allow two bytes per line, not one.

Position

```
seek file.variable {, offset {, relative.to}}
```

RELATIVE.TO

0	from the start of the file
1	from the current position

2	from the end
---	--------------

`seek sq, 0, 0` followed by `readseq` returns the first line; `seek sq, 5, 0` followed by `readblk sq, 4` returns the four bytes from offset 5.

`seek` with no offset returns to the start. Mixing `seek` with `readseq` is safe, but an offset that lands in the middle of a CRLF gives a `readseq` that starts with a stray terminator.

Buffering and timeouts

```
nobuf file.variable  
flush file.variable  
timeout file.variable, seconds
```

nobuf	turn buffering off for this file — every write goes straight out
-------	--

flush	push what is buffered now
-------	---------------------------

timeout	how long a read waits before giving up
---------	--

`nobuf` is for a file another process is watching. `timeout` matters on a named pipe or a device, where a read can otherwise block for ever; on an ordinary file it does nothing useful.

Asking about the file

```
status variable from file.variable then ... else ...
```

Fills *variable* with the operating system's view of the file: **21 fields** — size, timestamps, permissions and the rest, in the order the C library reports them.

`fileinfo(file.variable, 5)` reports a sequential file's type as **5**, and key `1006` gives the current position.

Deleting

```
deleteseq file.name, record.id then ... else ...  
deleteseq pathname then ... else ...
```

Removes the file. `closeseq` releases the variable without deleting anything; a sequential file left open is closed when the program ends, but the buffered tail is only guaranteed once `closeseq` has run.

Directory files, and where the marks go

A **directory** file is an SD file whose records are ordinary files. That is what makes sequential access to them possible, and it is the usual target for `openseq`.

Because the records are ordinary files, SD has to decide what to do with mark characters on the way in and out. `mark.mapping` controls it — see SD Basic - Dynamic Arrays. With mapping on, field marks become newlines and back again; with it off the bytes pass through untouched, which is what you want for a file another system produced.

What is not here

Nothing in the sequential-file group has been removed from this port. The **VFS** layer, which used to sit between these statements and the operating system, has been removed from the C entirely.

See also

SD Basic - File Handling · SD Basic - CSV Files · SD Basic - Data Conversion.

SD Basic - CSV Files

Reading and writing comma-separated data without hand-rolling the quoting.

SD has statements that read and write CSV directly, and they handle the two things that make hand-written CSV code wrong: a separator inside a field, and a quote inside a field. This page covers them.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Writing

```
writcsv value1, value2, ... to file.variable then ... else ...  
printcsv {on print.unit} value1, value2, ... {:}
```

`writcsv` writes one line to a **sequential** file — see SD Basic - Sequential Files for opening one. `printcsv` sends the same thing to the terminal or a print unit.

The quoting is done for you, and it is conformant. For three values, one containing a comma and one containing quotes:

```
writcsv 'a', 'b,c', 'say "hi"' to f
```

the file receives, byte for byte:

```
a,"b,c","say ""hi"""
```

A field is quoted only when it needs to be, an embedded quote is doubled, and the result is what RFC 4180 specifies. **Do not build this by hand** — the common mistakes are quoting nothing, quoting everything, and escaping a quote with a backslash, and each produces a file that opens wrongly in a spreadsheet rather than failing visibly.

A trailing colon on `printcsv` suppresses the line terminator, so several statements can build one row.

Reading

```
readcsv from file.variable to variable1, variable2, ... then ... else ...
inputcsv variable1, variable2, ...
matreadcsv matrix from file.variable then ... else ...
```

<code>readcsv</code>	one line from a sequential file into named variables
<code>inputcsv</code>	the same, from the terminal
<code>matreadcsv</code>	one line into a dimensioned matrix, one field per element

The line written above, read back with `readcsv from f to v1, v2, v3`, gives exactly

<code>v1</code>	<code>a</code>
<code>v2</code>	<code>b, c</code>
<code>v3</code>	<code>say "hi"</code>

— a clean round trip, with the quoting removed and the embedded comma and quotes restored.

`readcsv`'s `else` branch is end of file, as `readseq`'s is.

The number of variables is fixed at compile time and the data is not. A row with more fields than you named loses the extras silently; one with fewer leaves the trailing variables empty, which is indistinguishable from a row that genuinely had empty fields. If the width varies, use `csvdq()` below and count what came back.

Splitting a line you already have

```
csvdq(line {, delimiter})
dparse.csv line, delimiter, variable1, variable2, ...
```

`csvdq()` is a **de-quoter, not a quoter. the name reads the other way**. It takes one CSV line and returns its fields separated by **field marks**, honouring the quoting:

CALL	RESULT
<code>csvdq('plain')</code>	<code>plain</code> — one field, unchanged
<code>csvdq('has,comma')</code>	<code>has</code> and <code>comma</code> — two fields
<code>csvdq('has"quote')</code>	<code>hasquote</code> — the quote was consumed as quoting
<code>csvdq('a;b', ';')</code>	<code>a</code> and <code>b</code>

So it is the function to reach for when a line arrived from somewhere else and you want it as a dynamic array:

```
fields = csvdq(line)
n = dcount(fields, @fm)
for i = 1 to n
    ...
next i
```

There is no matching function that quotes a field. To produce CSV, use `writcsv` or `printcsv`, which do it as part of writing.

`dparse.csv` does the same split but assigns straight into named variables. `dparse.csv 'x,"y,z",w', ',', p1, p2, p3` gives `x`, `y,z` and `w` — the quoted comma correctly kept inside the second field.

The separator

Every statement here defaults to a comma. `csvdq()` and `dparse.csv` take the separator as an argument; a semicolon is common in locales where the comma is the decimal separator.

A file whose separator is a semicolon is still called CSV by the program that produced it. Look at a line before assuming the separator, and take it from configuration rather than hard-coding it — the same export from the same system changes separator when the machine's locale changes.

Line endings

CSV written by SD ends each line with **CRLF** — see SD Basic - Sequential Files. That is what RFC 4180 specifies and what Windows tools expect. Reading accepts either.

What is not here

Nothing in the CSV group has been removed from this port. The whole group is **new relative to the OpenQM 2.6.6 documentation** — none of these seven names appears in its by-type list, so the old help is no guide to them.

Two behaviours were fixed in this port and are worth knowing if you are moving files across from the Linux original:

reading	a conformant CRLF file used to lose its last field on every row, because the stray CR stayed attached to it
writing	output is now CRLF, so a file SD produced conforms to the RFC its own documentation claims

See also

SD Basic - Sequential Files · SD Basic - Data Conversion · SD Basic - String Functions.

SD Basic - Terminal Input and Output

Writing to the screen, reading from the keyboard, and positioning the cursor.

This page covers what a program says to the person running it and what it reads back: the output statements, the input statements, and the cursor and key handling underneath them.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Output

```
print {on print.unit} expression {:}  
crt expression {:}  
display expression {:}
```

All three write a line. The differences matter only once a print unit is involved:

<code>print</code>	goes to the current print unit — the terminal normally, the printer after <code>printer on</code>
<code>crt</code>	always goes to the terminal, even while <code>printer on</code> is in force
<code>display</code>	a synonym for <code>crt</code>

That is the whole reason `crt` exists. A report that prints its output and shows progress on screen uses `print` for the report and `crt` for the progress; using `print` for both puts "Processing record 400..." in the middle of the printed report.

A trailing colon suppresses the line ending, so the next statement continues on the same line:

```
crt 'Working' :  
for i = 1 to 10  
  crt '.' :  
next i  
crt ''
```

LAYING OUTPUT OUT

The format qualifier goes straight after the value, with nothing between them:

```
print total '10r2'
```

`42 'R#8'` gives `42` — right-justified in eight columns. The full specification language is in SD Basic - Data Conversion, along with the warning that a value longer than the width **wraps** rather than truncating.

The cursor

```
@(column, row)
@(code)
```

`@()` returns a string of terminal control characters. Printing it moves the cursor or changes the display; it is not a statement.

On this port it emits ansi escape sequences. `@(0,0)` is six bytes — `27 91 49 59 49 72`, which is `ESC [ 1 ; 1 H`, the standard cursor positioning sequence. `@(-1)`, clear screen, is also six bytes.

CODE

<code>@(-1)</code>	clear the screen and home the cursor
<code>@(-2)</code>	home the cursor without clearing
<code>@(-3)</code>	clear to end of screen
<code>@(-4)</code>	clear to end of line
<code>@(-5)</code> · <code>@(-6)</code>	cursor off, cursor on
<code>@(-13)</code> · <code>@(-14)</code>	reverse video on, off

The sequences come from the terminal definition, not from a fixed table. A session whose terminal type is wrong gets sequences the terminal does not understand and displays as literal text. `term` at the command line reports and sets the type.

Screen geometry

```
@crtwide
@crthigh
terminfo(key)
```

`@crtwide` and `@crthigh` are the current width and height. After `term 200,9999` they read **200** and **9999** — they follow whatever `term` last set, which is why a program must read them rather than assume 80 by 24.

`terminfo()` reports a named capability of the terminal definition. In a session with **no terminal**, `terminfo('name')` and `terminfo('cols')` both return the **null string** — so a program that formats a screen from `terminfo()` must cope with getting nothing back when it is run from a script or the API.

```
save.screen(column, row, width, height)
restore.screen(saved, column, row)
```

save and restore a rectangle, for a program that pops a window over what is already displayed.

Input

```
input variable {, length} {::} {with prompt} {format} {then ... else ...}
input @(column, row) {, length} : variable
inputfield variable, length {, ...}
```

`input` reads a line. The optional *length* limits what may be typed and returns as soon as that many characters arrive, without waiting for Enter.

CLAUSE

append	start with the variable's current value, so the user edits rather than retypes
overlay	as append, but typing replaces from the first character
edit	full line editing
hidden	do not echo — for a password
panning	scroll a field wider than the space it is shown in

CLAUSE

upcase fold what is typed to upper case

timeout n give up after n seconds and take the `else` branch

`hidden` **is the only correct way to read a password.** Turning the echo off by hand with `echo off` leaves it off if the program aborts between the two statements, and the user then types blind at the command prompt.

```
inputerr message
inputclear
cleardata
data expression
```

`inputerr` show a message on the status line without disturbing the screen

`inputclear` discard anything typed ahead

`data` **stack** a value for the next `input` to consume

`cleardata` discard what `data` stacked

`data` is how one program answers another's prompts:

```
data 'Y'
execute 'DELETE.FILE OLDSTUFF'
```

A `data` value that is not consumed stays stacked. The next `input` anywhere in the session takes it — including a prompt the user was meant to answer. `cleardata` **after anything that might not have consumed it.**

Keys

```
keyin()
keyinc()
keyinn()
```



```
keyready()  
keycode()
```

<code>keyin()</code>	wait for one keystroke and return it
<code>keyinc()</code>	as <code>keyin()</code> , but translate an escape sequence into a single key code
<code>keyinr()</code>	raw — no translation at all
<code>keyready()</code>	true if a keystroke is waiting; does not consume it
<code>keycode()</code>	the code of the key that ended the last <code>input</code>

```
keyedit code, key.sequence  
keyexit code, key.sequence  
keytrap code, key.sequence  
bindkey(key.string, action)
```

These attach a key to an editing action, an exit from `input`, or a trap. `keycode()` then tells the program which one fired, so a form can act on F3 without reading raw escape sequences.

Prompts and headings

```
prompt character  
heading {no.eject} {on print.unit} text  
footing {on print.unit} text  
page {on print.unit} {page.number}
```

`prompt` sets the character SD shows when it wants input — a colon by default; `prompt ''` suppresses it.

`heading` and `footing` are covered with the rest of the paging machinery in SD Basic - Printing, because they apply to the terminal and the printer in the same way.

Echo and silence

```
echo on | off | expression  
hush on | off | expression
```

echo	whether typed characters appear
------	---------------------------------

hush	whether output appears
------	-------------------------------

`hush on` suppresses output entirely — used around an `execute` whose chatter you do not want. Both nest badly: they are a session setting, not a stack, so a program that aborts between `hush on` and `hush off` leaves the session silent and the user with no way to know why.

Reading the command line

```
tclread variable
```

Returns the command line that started the program, so a verb written in BASIC can parse its own arguments. `@sentence` holds the same thing.

The port's terminal, in one paragraph

SD Core for Windows turns on the console's ANSI processing itself rather than inheriting it, so `@()` sequences work in an ordinary PowerShell window without the user configuring anything. Over ssh the session is a console session, so the same sequences reach the client terminal. **SD is a terminal system throughout; the API is the only way to reach it without one**, and there a program is reading a command's output rather than driving a screen. **Such a session has no geometry and no capabilities**, which is why `terminfo()` returns the null string above rather than guessing.

What is not here

`ttyget()` · `ttyset` **removed** — they exposed a POSIX terminal structure this port does not have

`connect.port()` **removed**

See also

[SD Basic - Printing](#) · [SD Basic - Data Conversion](#) · [SD Basic - Program Control](#).

SD Basic - Printing

Print units, headings and page breaks, and sending output somewhere other than the screen.

Printing in SD goes through a **print unit** — a numbered destination that can be the terminal, a printer, a file or a spooler. This page covers directing output to one, and the paging that goes with it.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Print units

There are numbered print units. **Unit 0 is the terminal.** Any statement that produces output takes an `on` clause to choose a different one:

```
print on 1 'this goes to unit 1'
heading on 1 'so does this heading'
```

Without an `on` clause, output goes to the **current** unit — which is the terminal until `printer on` redirects it.

```
printer on
printer off
```

`printer on` sends `print` to the print unit; `printer off` sends it back to the terminal. `crt` ignores both and always writes to the screen — see SD Basic - Terminal Input and Output.

Choosing a destination

```
printer file {on print.unit} file.name, record.name {then ... else ...}
printer name {on print.unit} printer.name
printer display {on print.unit}
printer close {on print.unit}
printer reset
```

file	write to a record in a directory file
name	send to a named Windows printer
display	send to the terminal
close	finish the job and release the unit
reset	put every unit back to its default

The unit number goes after `on`, not `first`. `printer file on 1 'REPORTS', 'DAILY'` is right; `printer file 1, 'REPORTS', 'DAILY'` is a compile error, because the parser reads the `1` as the file name and then wants a comma it does not find.

A PRINT JOB END TO END

```
printer file on 1 'ZZOUT', 'REPORT'
heading on 1 "REPORT PAGE 'PL'"
printer on
print on 1 'first line'
print on 1 'second line'
printer close on 1
printer off
```

writes a record of **4 lines**, the first of which is `REPORT PAGE 1` — the heading, with `'PL'` replaced by the page number, followed by the two data lines and the blank the heading leaves under itself.

`printer close` **is what actually writes the file.** Until it runs the output is buffered in the unit. A program that ends without closing its units gets them closed for it, but a program that opens a second job on the same unit without closing the first merges the two.

Headings, footings and page breaks

```
heading {no.eject} {on print.unit} text
footing {on print.unit} text
page {on print.unit} {page.number}
```

A heading is reprinted at the top of every page and a footing at the bottom. Setting either resets the page counter and starts a new page — **unless** `no.eject` is given, which is how you set a heading without wasting the page you are on.

The text may contain codes, written inside quotes within the string:

CODE

'PL' . 'P'	the page number
'D'	the date
'T'	the time
'N'	suppress the automatic page break
'L'	a line break within the heading
'C'	centre what follows

Because the codes are quoted inside the heading, the heading itself needs the other kind of quote around it:

```
heading on 1 "SALES BY REGION 'L' 'D' 'C' PAGE 'PL'"
```

`page` forces a break, optionally resetting the number.

A heading is per print unit and it persists. It stays set until it is changed or the unit is reset, including after the program that set it has finished. A program that sets a heading on unit 0 and exits leaves the next thing the user does wearing that heading. `printer reset` **on the way out**, or set the heading on a unit you also close.

Asking about a unit

```
getpu(key, print.unit)
setpu key, print.unit, value
printer.setting(key, print.unit, value)
```

For unit 0 in a fresh session:

KEY	MEANING	VALUE
0	is the unit defined?	1
2	width in columns	80
3	length in lines	66
15	current page number	1
1002	lines left on this page	66

Other keys worth knowing:

KEY	
1	mode — where the output is going
4 · 5 · 6	top, bottom and left margins
7	spooler flags — landscape, duplex, raw mode and the rest
9	form name
12	number of copies
1003 · 1004 · 1005	heading, footing and data lines per page

`setpu` changes a setting; `printer.setting()` takes **three** arguments and reports or sets a spooler-level setting. **Not every key is valid for every call** — `printer.setting(0, 0, 0)` fails with *"Unrecognised printer setting key"*, because key 0 belongs to `getpu`, not to it.

The width and length are the unit's, not the terminal's. Unit 0 reports 80 by 66 in a session whose terminal has been set to 200 by 9999 — see `@crtwide` in SD Basic - Terminal Input and Output. A report that lays itself out from `getpu` and a screen that lays itself out from `@crtwide` are reading two different numbers, and both are right.

Capturing output instead of printing it

```
como on name
como off
```

`como` records everything the session displays into a record, which is the simplest way to keep a transcript of a run. To capture the output of one command rather than a whole session, `execute ... capturing` is better — see SD Basic - Program Control.

Suppressing output

```
hush on | off | expression
```

Discards output entirely rather than redirecting it. It applies to the session, not to a print unit, and it does not nest — the caution in SD Basic - Terminal Input and Output applies here too.

Printing on this port

A named printer is a Windows printer. `printer name` takes the name as Windows knows it, and the spooler flags in key `7` include a **raw mode** that sends bytes straight through without the driver reformatting them — which is what a line printer or a label printer needs.

There is no `lp` and no PostScript pipeline. The Linux original assumed both; this port hands the job to the Windows spooler instead. A report that worked by writing PostScript to a pipe needs a driver that accepts it, or raw mode and a printer that understands what you send.

What is not here

Nothing in the printing group has been removed from this port, but the **destination model changed**: printing goes to the Windows spooler rather than to a POSIX print command, and `printer.setting` keys that named a Linux facility do not apply.

See also

SD Basic - Terminal Input and Output · SD Basic - Data Conversion · SD Basic - Program Control.

SD Basic - Locks and Transactions

Keeping two sessions out of each other's way, and making several writes all-or-nothing.

Everything on this page exists because more than one session can be running at once. A single-user program never needs any of it; the moment a second person runs the same program, all of it matters.

SD Basic - File Handling introduces `readu` and the `locked` clause. This page is the whole picture: what each lock code means, what the other session sees, what waits and for how long, and what a transaction does to all of it.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The three kinds of lock

record lock	one record of one file. <code>readu</code> , <code>readl</code> , <code>recordlocku</code> , <code>recordlockl</code>
file lock	every record of one file, including ones that do not exist yet. <code>filelock</code>
task lock	one of 64 numbered semaphores that mean whatever you decide. <code>lock</code> , <code>unlock</code>

All three belong to the **session**, not to the program, and all three are released when the session ends.

Record locks

```
readu variable from file.variable, record.id {locked ...} then ... else ...
readl variable from file.variable, record.id {locked ...} then ... else ...
recordlocku file.variable, record.id {locked ...}
recordlockl file.variable, record.id {locked ...}
release {file.variable {, record.id}}
```

`readu` takes an **update** lock, `readl` a **shared read** lock. `recordlocku` and `recordlockl` take the same two locks without reading the record — useful when you are about to create one.

`release` with a file variable and a record id gives back one lock, with just a file variable every lock on that file, and with **no arguments at all** every lock the session holds.

A `write` **outside a transaction releases the lock; inside one it keeps it**. On the same record in the same program:

	<code>RECORDLOCKED()</code> AFTER
<code>readu</code> then <code>write</code> , no transaction	0 — the lock is gone
<code>readu</code> then <code>write</code> , inside a transaction	2 — still held

That is the single most useful fact about the two. Outside a transaction the read-modify-write cycle ends when you write. Inside one, it ends when the transaction does.

RECORDLOCKED() — what every return value means

```
recordlocked(file.variable, record.id)
```

Every one of the seven codes below was produced deliberately, three of them by a second session holding the lock:

CODE	MEANING
-3	another session holds a file lock on this file
-2	another session holds an update lock on this record
-1	another session holds a read lock on this record
0	nobody holds a lock on it
1	this session holds a read lock
2	this session holds an update lock
3	this session holds a file lock

And `status()` **after it is the other session's user number**. This is not written down anywhere else and it is the only way to find out *who* is holding the record. With the holder running as user 73 and the asker as user 74:

```

if recordlocked(f, id) < 0 then
  crt 'Record ' : id : ' is held by user ' : status()
end

```

`status()` read **73** after `recordlocked()`, after a refused `readu`, after a refused `readl`, after a refused `filelock` and after a refused `lock` — the same number every time. A positive code is your own lock and needs no enquiry.

A negative code is a snapshot, not a promise. The other session may release between your test and your read. `recordlocked()` is for reporting; the `locked` clause is for deciding.

The `locked` clause, and what happens without one

```

readu rec from f, id locked
  crt 'Someone else is editing that record.'
  return
end then
  ...
end else
  ...
end

```

Without a `locked` clause, a conflicting read waits — and it really does wait. Across two sessions, the second session's plain `readu` against a record the first is holding **blocks for 252 ms** and returns the instant the holder releases it, with `recordlocked()` then reading `2`. It does not fail, it does not time out, and there is no message. SD retries every 250 milliseconds, which is where that figure comes from.

A program without a `locked` clause is a program that can sit silently for as long as somebody else keeps a record open. **Any interactive read-for-update should have one.**

With one session holding an update lock on `R1`:

THE SECOND SESSION TRIED	RESULT
<code>readu ... 'R1' locked</code>	the <code>locked</code> branch, <code>status()</code> 73
<code>readl ... 'R1' locked</code>	the <code>locked</code> branch, <code>status()</code> 73
<code>readu ... 'R2' locked</code>	succeeded — a different record is unaffected
plain <code>readu ... 'R1'</code>	waited 252 ms, then got the lock

A read lock is not a free pass. `readl` is refused against another session's update lock, and `readu` is refused against another session's read lock. Two `readl` locks on the same record **do** coexist: both sessions hold one at once and each sees its own as `1`. When the second releases its own, `recordlocked()` goes back to **-1** — the first session's lock is still there. The code reports the lock that matters to you, not a count.

File locks

```
filelock file.variable {locked ...} {on error ...}
fileunlock file.variable
```

`filelock` **has no** `then` **or** `else` **clause, so success is invisible.** It takes only `locked` and `on error`. If you need to know whether you got it, set a flag in the `locked` branch, or ask `recordlocked()` — a `3` means the file lock is yours:

```
blocked = @false
filelock f locked
  blocked = @true
end
if not(blocked) then
  ...
  fileunlock f
end
```

Forgetting the `fileunlock` **deadlocks other sessions**, including the one that is waiting to tell you it has finished. A rendezvous record written into a file you have locked is a rendezvous that never happens.

With one session holding a file lock:

<code>recordlocked(f, 'R1')</code> from the other session	-3 , <code>status()</code> the owner
<code>recordlocked(f, 'R2')</code>	-3 as well — every record, not the one you locked
<code>readu ... locked</code> on any record	the <code>locked</code> branch
a plain <code>read</code>	succeeded — <code>[beta]</code> came back normally
one record already locked by somebody else	<code>filelock</code> took the <code>locked</code> branch

A file lock stops locking, not reading. That surprises people. Other sessions can still read every record in the file while you hold it; what they cannot do is take a lock, and therefore cannot write.

Task locks

```
lock n then ... else ...
unlock n
```

There are **64** of them, numbered 0 to 63, and SD attaches no meaning to any of them. They are for serialising something that is not a record — a nightly report, a printer, an external file.

A **lock** with no **else** clause retries for ever. The compiler generates one for you, and what it generates is `sleep 1` followed by a jump back to the `lock` — so a bare `lock 7` in a program whose lock somebody else holds is an infinite loop with a one second period and no output. Write the `else`.

With one session holding task lock 7:

<code>lock 7</code> in the other session	the <code>else</code> branch, <code>status()</code> 73 — the holder's user number
<code>lock 8</code>	the <code>then</code> branch, <code>status()</code> 0

So the `else` branch tells you who has it, exactly as the record lock enquiry does.

Taking a lock you already hold succeeds, and a task lock is not a counter — one `unlock` releases it however many times you locked it. The ownership test is "*unowned or mine*", and `unlock` clears the slot outright.

`testlock()` would answer "*who owns lock n*" without taking it, and **it is not available to an ordinary program** — see "What is not here" below.

Transactions

```
begin transaction
...
commit
...
rollback
end transaction
```

or, the same thing in the other spelling:

```
transaction start
...
transaction commit
transaction abort
```

Everything written between the start and the `commit` lands together or not at all.

Inside a transaction you must already hold the lock on every record you write or delete. This is the rule that catches everybody, and the error message names the wrong thing entirely:

```
Error 3023 (o/s 0) writing record (Possible full disk?)
```

The disk is not full. 3023 is "attempt to write/delete record with no lock". The same `write` that succeeds outside a transaction fails with 3023 inside one, and succeeds inside one if a `recordlocku` comes first. Outside a transaction no lock is needed, which is why the same line works in testing and fails in production the first time somebody wraps it.

So the shape of a transaction is:

```
begin transaction
  recordlocku f, id
  write rec to f, id
  commit
end transaction
```

`on error` catches it if you would rather test than abort:

```
write rec to f, id on error
  crt 'write refused, status ' : status()  ;* 3023 = no lock held
end
```

WHAT EACH ENDING DOES

One transaction per row, each on its own record:

	THE WRITE	LOCKS	<code>SYSTEM(1008)</code>
<code>commit</code> then <code>end transaction</code>	lands	released	+1
<code>rollback</code>	discarded	released	0

	THE WRITE	LOCKS	SYSTEM(1008)
<code>end transaction</code> with neither	discarded, silently	released	0
<code>transaction commit</code>	lands	released	+1
<code>transaction abort</code>	discarded	released	0

Falling out of the bottom of a transaction throws the work away. There is no implicit commit. A `return`, a `goto` or simply reaching `end transaction` without having executed a `commit` discards every write since the start, and nothing is printed. The record still reads whatever it held before.

Inside the transaction your own reads see your own uncommitted writes — a program reads back its own new value before the commit, and a rolled-back `delete` makes the record read `else` inside the transaction and reappear afterwards.

SYSTEM(1007) AND SYSTEM(1008)

<code>system(1007)</code>	the transaction number, or 0 when not in a transaction
<code>system(1008)</code>	the transaction level

`system(1007)` is reliable: it is a fresh number for each transaction and reads 0 the moment one ends, either way.

`system(1008)` is not, and it is a defect. The level is incremented when a transaction starts and decremented only on the paths that end in a rollback, so **every committed transaction leaves the count one too high for the rest of the session.** In a run of four unnested transactions the first reports level 1 and the fourth reports level 2. **Do not test `system(1008)` to find out whether you are in a transaction — test `system(1007)`.** This is upstream's, not this port's: `sdb64` carries the identical code.

DO NOT NEST TRANSACTIONS

A `commit` inside a Nested transaction abandons the outer one, and the outer write is lost. With an outer transaction writing `R2` and an inner one writing `R3`:

the inner record <code>R3</code>	<code>inner</code> — landed
the outer record <code>R2</code>	<code>base</code> — the write vanished
<code>system(1007)</code> after the inner <code>commit</code>	0 — the session is no longer in any transaction

Nothing is reported. The compiler generates `commit` as a jump past the `end transaction`, so the bookkeeping that would restore the outer transaction never runs, and the outer `commit` then commits an empty cache. **Keep transactions flat**, and note that this includes a subroutine that starts its own transaction while the caller is in one.

`set.trigger` attaches a program that runs on every write or delete. A trigger runs inside the caller's transaction, so it is bound by the same locking rule.

What is not here

`testlock()` and `getlocks()` are internal-only and an ordinary program cannot call them. They are in the compiler's internal intrinsic list, which only a program compiled with `$internal` in an administrator's `SDSYS` session may reach.

And the compiler does not say so. `v = testlock(5)` in an ordinary account produces, at the **last line of the program**:

```
41: Matrix TESTLOCK is not referenced in a DIM statement
WARNING: TESTLOCK is not assigned a value
```

An unknown function name is read as a matrix reference, so the complaint is about a `dim` statement you never wrote, reported nowhere near the line that caused it. If a function you are sure exists produces that message, it exists and you are not allowed to call it.

The same applies to `getlocks()`, which would list every lock in the system. **From an ordinary program the only enquiry available is `recordlocked()`**, and it answers about one record at a time.

`release.lock` is a restricted statement for the same reason and gives "*Unrecognised statement*".

Deadlock detection exists in the C, and takes the `locked` branch when it fires. It was not provoked: two sessions each waiting on the other's record is a state this set has not deliberately created.

See also

SD Basic - File Handling · SD Basic - System and Environment · SD Basic - Program Control · SD Basic - Debugging.

SD Basic - Sockets

Talking TCP from SD BASIC, as a client and as a server.

SD BASIC can open a TCP connection, and it can listen for one. That is enough to call a web service, to feed a message queue, or to let another program talk to your application without going through a file.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

One program can be both ends. `create.server.socket` calls `listen()`, so a client can connect on loopback and wait in the backlog until the same session accepts it.

The nine names

<code>create.server.socket(<i>addr</i> , <i>port</i> , <i>flags</i>)</code>	bind and listen. Returns a socket , or 0
<code>accept.socket.connection(<i>server</i> , <i>timeout</i>)</code>	take the next waiting connection
<code>open.socket(<i>addr</i> , <i>port</i> , <i>flags</i> { , <i>context</i> })</code>	dial out
<code>write.socket(<i>socket</i> , <i>data</i> , <i>flags</i> , <i>timeout</i>)</code>	returns the byte count
<code>read.socket(<i>socket</i> , <i>maxlen</i> , <i>flags</i> , <i>timeout</i>)</code>	returns the bytes read
<code>set.socket.mode(<i>socket</i> , <i>key</i> , <i>value</i>)</code>	returns 1 or 0
<code>socket.info(<i>socket</i> , <i>key</i>)</code>	ask about a socket
<code>close.socket <i>socket</i></code>	a statement , not a function
<code>server.addr(<i>name</i>)</code>	resolve a host name

The `SKT$` constants are in `SYSCOM KEYS.H`, reachable with `$include keys.h`. They are written out as numbers on this page so that what was passed is visible.

Opening the two ends

```
srv = create.server.socket('127.0.0.1', 45123, 0)
cli = open.socket('127.0.0.1', 45123, 0)
inc = accept.socket.connection(srv, 2000)
```

Flags `0` is TCP, stream. `0x00010000` selects UDP and `0x00020000` ICMP.

Both fail by returning zero, not by taking an `else` branch — they are functions, and there is no `then`/`else` on them. Test with `socket.info(s, 0)`, which is the one key that is safe to ask about a non-socket:

```
srv = create.server.socket('127.0.0.1', port, 0)
if socket.info(srv, 0) = 0 then
  crt 'cannot listen on ' : port : ', status ' : status()
  stop
end
```

How a connection fails:

a port nobody is listening on `socket.info 0, status()` **7005**, and SD **printed** *"Failed to connect: 111: Connection refused"* to the terminal by itself

a second listener on a port already in use `socket.info 0, status()` **7012**

The refusal message is printed whether you want it or not, which matters if you are polling for a service to come up.

What SOCKET.INFO tells you

KEY	MEANING	VALUE
<code>0</code>	is this a socket at all?	<code>1</code>
<code>1</code>	kind: 1 server, 2 incoming, 3 outgoing	see below
<code>2</code>	port	see below

KEY	MEANING	VALUE
3	IP address	127.0.0.1
4	blocking?	0 on every new socket
5	Nagle disabled?	0
6	keep-alive?	0
7	family: 1 IPv4, 2 IPv6	1

Key 2 means two different things and the difference is easy to miss. On the socket you dialled out with it is **the port you dialled**; on a socket you accepted it is **the far end's ephemeral port**:

	KIND	PORT
<code>create.server.socket(..., 45123, ...)</code>	1	45123
<code>open.socket(..., 45123, ...)</code>	3	45123
<code>accept.socket.connection(...)</code>	2	62602 — the client's own port, different every run

Reading and writing

```
n = write.socket(cli, 'hello world', 0, 2000)
d = read.socket(inc, 100, 0, 2000)
```

`write.socket` returned **11** for eleven bytes. `read.socket` returned the eleven bytes.

The timeout on a read does nothing unless the socket is blocking, and no socket starts out blocking. This is the trap on this page. On a socket with nothing sent to it:

THE CALL	WAITED	RETURNED
<code>read.socket(s, 100, 0, 5000)</code>	0 ms	nothing, <code>status()</code> 1011
<code>read.socket(s, 100, 1, 2000)</code>	2025 ms	nothing, <code>status()</code> 1011

THE CALL	WAITED	RETURNED
after <code>set.socket.mode(s, 4, 1)</code> , <code>read.socket(s, 100, 0, 2000)</code>	2018 ms	nothing, <code>status()</code> 1011

A five second timeout was ignored completely. **Flag 1 is `SKT$BLOCKING` and flag 2 is `SKT$NON.BLOCKING`; flag 0 means "whatever the socket is set to", and a new socket is set to non-blocking.** Either pass `1` on every read that is allowed to wait, or call `set.socket.mode(s, 4, 1)` once after opening.

This is exactly the bug that passes its own tests. On loopback, and in any test where the reply is already sitting in the buffer, a non-blocking read returns the data immediately and everything looks right — **0 ms and five bytes**. Over a real network the reply arrives a few milliseconds later and the same code returns nothing.

`status()` is **1011**, timeout, for both "nothing yet" and "waited and gave up". It does not distinguish them.

TCP IS A STREAM, NOT A MESSAGE

Ten bytes written and read back in two pieces:

<code>read.socket(s, 4, ...)</code>	<code>abcd</code>
<code>read.socket(s, 100, ...)</code>	<code>efghij</code>

Nothing marks where one `write.socket` ended and the next began. **If your protocol has messages, put a length or a terminator in them yourself** and keep reading until you have a whole one.

THE BYTES ARRIVE EXACTLY AS SENT

	SENT	RECEIVED	IDENTICAL
field, value and subvalue marks	7	7	yes
<code>char(0)</code> , <code>char(1)</code> , <code>char(255)</code> and a letter	4	4	yes

A NUL does not terminate anything and a mark is not translated. A dynamic array can go down a socket and come back as itself.

SET.SOCKET.MODE

KEY		WHAT IT DOES
4	blocking	set 0 → reads 0; set 1 → reads 1
5	disable Nagle	set 1 → reads 1
6	keep-alive	set 0 → reads 0; set 1 → reads 1
anything else		returns 0, <code>status()</code> 1006

Key 6 can be turned off as well as on: `set.socket.mode(s, 6, 0)` returns 1 and `socket.info(s, 6)` then reads 0.

When the far end goes away

```
close.socket cli
```

There are two answers, not one, and which you get depends on which end closed. In the same session:

WHAT CLOSED	READING THE OTHER END GIVES
the outgoing socket closed, read the accepted one	nothing, <code>status()</code> 7013
the accepted socket closed, read the outgoing one	nothing, <code>status()</code> 1008

7013 is `ER_SKT_CLOSED` — `recv()` returned zero, an orderly shutdown. 1008 is `ER_FAILED` — `recv()` returned an error, and `os.error()` carries the operating system's code. **A read loop must treat both as the end of the conversation.** Testing only for 7013 is a loop that spins: that guard runs its full iteration count against a closed socket and comes back with nothing every time.

Neither is 1011. An idle socket gives 1011 and may still have something to say.

`close.socket` is a statement and it **refuses anything that is not a socket**, so do not call it on the zero that a failed `open.socket` returned.

Names and addresses

```
addr = server.addr('localhost')
```

```
server.addr('127.0.0.1')
```

```
127.0.0.1
```

```
server.addr('localhost')
```

```
:::1
```

`localhost` **RESOLVES TO THE IPv6 ADDRESS ON THIS PLATFORM.** A program that resolves `localhost` and then dials the answer is dialling IPv6, while `create.server.socket('127.0.0.1', ...)` is listening on IPv4 — family **1** — and the two do not meet. Use `127.0.0.1` on both sides, or `:::1` on both sides, and do not resolve a name to get there.

And a name that does not resolve blocks. `server.addr()` calls the operating system resolver with no timeout of its own. A call for a name that cannot resolve can still be waiting after **45 seconds**, with no way to abandon it short of abandoning the session. There is no way to bound it from BASIC. **Do not put `server.addr()` on a path where a user is waiting**, and prefer a configured address to a name.

A worked client, run rather than composed

This is the shape everything above adds up to. It was compiled and run with the far end in the same session, answering in **three** pieces so that the reassembly is exercised rather than assumed:

```
skt = open.socket(host, port, 0)
if socket.info(skt, 0) = 0 then
  crt 'no connection: ' : status()
  stop
end
x = set.socket.mode(skt, 4, 1)          ;* or every read returns at once
n = write.socket(skt, 'PING' : char(10), 0, 5000)

reply = ''
chunks = 0
loop
  chunk = read.socket(skt, 2048, 0, 5000)
  s = status()
  chunks += 1
  reply := chunk
```

```
until s = 7013 or s = 1008 or index(reply, char(10), 1) > 0 or chunks > 20
repeat
close.socket skt
```

Send `PON`, `G one` and a newline as three separate writes and the loop comes back with **one** chunk containing `PONG one`, the terminator at position 9 and `status()` `0`. The `chunks > 20` arm is not decoration — with the far end closed, the same loop guarded only by `s = 7013` runs every one of its iterations, because that end reports **1008**.

The terminator is yours, not TCP's. Three writes arrived as one read here and could just as easily arrive as three; the only reason the loop knows it is finished is the newline it went looking for.

What is not here

`writpkt` is a **restricted statement** — internal programs only. In an ordinary account it is "*Unrecognised statement*". It writes an SDClient protocol packet and is not a general socket write.

No TLS. There is nothing in SD BASIC that speaks HTTPS; a socket carries whatever bytes you put on it. Anything encrypted has to terminate outside SD.

The whole of this page is TCP. `0x00010000` and `0x00020000` are the UDP and ICMP flag values.

A server that serves more than one client at a time needs something to do the waiting, and SD BASIC has no `select` over several sockets. The shape that works is one accepted connection per session.

See also

SD Basic - System and Environment · SD Basic - Sequential Files · SD Basic - String Functions · SD Basic - Locks and Transactions.

SD Basic - System and Environment

Asking SD about itself, about the machine, and about the session you are in.

This page covers the enquiries: what time is it, who am I, where is the data, what did the last thing that failed say, and what may this account do. Most of it is one function, `system()`, and most of the surprises are about which form a path comes back in.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Values that are particular to one machine — a user number, a computer name, a process id — are marked as examples; the shapes are not. What you see back will differ in the value and match in the form.

SYSTEM()

`system(key)`

THE SESSION

KEY		VALUE
7	terminal type	<code>windows</code>
9	CPU time used, ms	<code>45</code>
12	time, as <code>time()</code>	<code>70787</code>
18	user number	<code>67</code> (example)
23	break key enabled?	<code>1</code>
24	echo enabled?	<code>1</code>
25	is this a phantom?	<code>0</code>

KEY		VALUE
26	prompt character	?
1000	capturing in effect?	0
1001	case inversion on?	0
1029	internal subroutine depth	0 at the top, 1 inside a gosub
1030	login time, internal	1851017987
1031	operating system process id	605 (example)
1050	administrator?	0

THE MACHINE

KEY		VALUE
31	licence number	0
42	IP address	empty
91	is this Windows?	1
1006	Windows NT style?	0
1009	endian — 0 little	0
1010	platform name	Windows
1012	SD version	W1.0-0
1013 / 1014	user limit, without / with the phantom pool	20 / 20
1015	computer name	Gitorli (example)
1017	port number of a tcp connection	0
1028	system id	1028

Two of those three answer correctly and one does not. `system(91)` reads 1 and `system(1010)` reads Windows. `system(1006)`, "Windows NT style?", reads 0 — it is the one to leave alone. **Ask** `system(91)` **whether this is Windows**; it is the key this port sets deliberately for that purpose.

PATHS, AND THEY ARE NOT ALL IN THE SAME FORM

KEY		VALUE
32	the <code>sdsys</code> directory	<code>C:\ProgramData\SD\sdsys</code>
38	the temporary directory	<code>/cygdrive/c/WINDOWS/TEMP</code>
1011	the configuration file	<code>C:/ProgramData/SD/sd.conf</code>
1024	the directory SD was started in	<code>/cygdrive/c/Users/dmont/OneDrive/Documents</code>

Three different spellings of a Windows PATH come out of one function. A backslash path, a POSIX `/cygdrive/` path, and a forward-slash path with a drive letter. `@sdsys` agrees with key 32 and `@path` — the account directory — is in the POSIX form.

A `/cygdrive/` path handed to a Windows program does not work. Windows reads it as drive-relative and either fails silently or complains that the parent directory does not exist. This is not theoretical: it is what stopped the full-screen editors working the first time they were built for this port. There is a conversion function in the kernel and **an ordinary program cannot call it** — see "What is not here". **Take the path from configuration rather than from `system()` if a Windows program is going to see it.**

LISTS AND STRUCTURES

KEY		
1002	the call stack	field per level: <code>path</code> , then <code>offset</code> and line pairs — <code>.../BP.OUT/ZZMATH</code> at line 41, then <code>\$CPROC</code>
1003	open files	field per file, <code>unit</code> and path. <code>\$ipc</code> is always one of them
1025	environment variables	two fields: field 1 every name, field 2 every value, value-mark separated

`system(1025)` returns **2** fields, every name in the first — it is not a list of `NAME=value` pairs.

TIME

KEY		
1005	internal time	<code>date() * 86400 + time()</code> — the difference is 0
1020	milliseconds since midnight	<code>9587454</code>

`system(1020)` is the one to time something with: against a lock wait it gives 252 ms where `time()` gives 0.

DATE, TIME and TIMEDATE

```
date()    time()    timedate()
```

Taken together at one instant:

<code>date()</code>	<code>21423</code> — days since 31 December 1967
---------------------	--

<code>time()</code>	<code>70787</code> — seconds since midnight
---------------------	---

<code>timedate()</code>	<code>19:39:47 26 AUG 2026</code>
-------------------------	-----------------------------------

<code>oconv(date(), 'D4-')</code>	<code>08-26-2026</code>
-----------------------------------	-------------------------

<code>oconv(time(), 'MTS')</code>	<code>19:39:47</code>
-----------------------------------	-----------------------

`@date` and `@time` hold the same numbers, **but they are set once per command** rather than read afresh, so in a long loop they do not move while `date()` and `time()` do.

`timedate()` returns a formatted string, not a number. Do not do arithmetic on it — see SD Basic - Data Conversion for the conversion codes.

ENV()

```
env(name)
```

`env()` is case sensitive and a wrong case looks exactly like a missing variable.

<code>env('PATH')</code>	926 characters
--------------------------	----------------

<code>env('path')</code>	0 characters
--------------------------	---------------------

<code>env('ProgramData')</code>	<code>C:\ProgramData</code>
---------------------------------	-----------------------------

<code>env('NOSUCHVAR')</code>	empty
-------------------------------	-------

Windows itself treats environment variable names as case-insensitive everywhere else, which is what makes this worth knowing. `ProgramData` works only because that is exactly how Windows spells it. **Get the spelling from `system(1025)` field 1 rather than from memory.**

CONFIG()

```
config(name)
```

On a stock installation:

<code>config('FILERULE')</code>	<code>0</code>
<code>config('GRPSIZE')</code>	<code>2</code>
<code>config('MAXIDLEN')</code>	<code>63</code>
<code>config('NUMFILES')</code>	<code>80</code>
<code>config('NUMLOCKS')</code>	<code>100</code>
<code>config('SORTMEM')</code>	<code>4096</code>
<code>config('SPOOLER')</code>	<code>empty</code>

The name is case sensitive and at most eight characters. both failures now look the same, which is the point — a name that is too long is a name that does not exist, and a caller cannot tell the two apart:

<code>config('numlocks')</code> — right name, wrong case	<code>empty, status() 1004</code>
<code>config('NOSUCHKEY')</code> — nine characters	<code>empty, status() 1004</code>

Keep every `config()` name to eight characters and upper case.

Neither call aborts the caller: a name that is too long comes back empty with a status, the same as a name that does not exist.

SYSMSG()

```
sysmsg(number {, substitution ...})
```

Returns the text of one of SD's own messages, with `%s` substitutions filled in.

<code>sysmsg(2831)</code>	Unrecognised statement
---------------------------	------------------------

<code>sysmsg(6711, 'ABC')</code>	Unable to find source record ABC
----------------------------------	----------------------------------

<code>sysmsg(2201)</code>	Account name '' is not in register — an unfilled substitution comes back empty
---------------------------	--

<code>sysmsg(99999)</code>	[99999] Message not found
----------------------------	---------------------------

<code>sysmsg(1)</code>	[1] Message not found
------------------------	-----------------------

A message number is not a `status()` code. They are separate numbering schemes that overlap. `status()` **3006** is *record not found*; `sysmsg(3006)` is `Modes:`. Do not render a status code by passing it to `sysmsg()`.

`get.messages()` takes **no arguments** and returned **nothing** — zero fields — in an ordinary session. It reports messages sent between sessions, and nothing had sent any.

Continued in

SD Basic - Status, Encryption and the Machine — status codes, checksums, encryption, umask, the @variables, `os.execute` and `logmsg`.

SD Basic - Status, Encryption and the Machine

Status codes, checksums, encryption, the file mask, the @variables, os.execute and the error log.

This page continues SD Basic - System and Environment.

STATUS() and OS.ERROR()

`status()` carries the result of the last operation that sets one. It is overwritten constantly, so **read it into a variable on the line after the call you care about**, not three lines later. Every `status()` value quoted in this document set was captured that way.

`os.error()` carries the operating system's own error number from the last call that made one, and is `0` when nothing has failed.

Codes worth knowing:

<code>1004</code>	item not found
<code>1006</code>	bad action key
<code>1008</code>	action failed — <code>os.error()</code> has the detail
<code>1011</code>	timeout
<code>3001</code>	subfile not found
<code>3006</code>	record not found
<code>3007</code>	no VOC record — from a failed <code>open</code> of a name that is not in the VOC
<code>3021</code>	cannot gain exclusive access to a file
<code>3023</code>	write or delete with no lock held
<code>7005</code> / <code>7012</code> / <code>7013</code>	socket: cannot connect / cannot bind / closed

CHECKSUM()

```
checksum(string)
```

<code>checksum('ABC')</code>	<code>451</code>
<code>checksum('abc')</code>	<code>291</code>
<code>checksum('ACB')</code>	<code>448</code>
<code>checksum('')</code>	<code>0</code>
<code>checksum('A' : @fm : 'B')</code>	<code>-325</code>

It is case sensitive and order sensitive, and **it can be negative**, so a program that stores it needs a signed field. It is a change detector, not a digest: it is short, and it is not a security primitive.

SDENCRYPT() and SDDECRYPT()

```
sdencrypt(data, key, encoding)
sddecrypt(data, key, encoding)
```

Three arguments. The encoding is `201` for hex or `202` for base64.

A passphrase is not a key, and an ordinary program cannot make one. `sdencrypt('The quick brown fox', 'secretkey', 202)` returns **nothing** and sets `status()` to **10204**, a key length error. The key has to be an encoded 256-bit key, and the function that derives one from a password is `sdext()`, which is internal-only. **From an ordinary account these two functions have no usable key**, and there is no way in.

UMASK()

```
umask(n)
```

Sets the file creation mask and returns the **previous** value. A negative argument asks without setting:

CALL	RETURNED	MASK AFTERWARDS
<code>umask(-1)</code>	2	2 — unchanged
<code>umask(18)</code>	2 — the old value	18
<code>umask(-1)</code>	18	18
<code>umask(2)</code>	18	2 — restored

So the way to change it temporarily is to keep what the setting call returned and pass it back.

SENTENCE() and the @variables

`sentence()` and `@sentence` are the same thing. Both hold the command line that started the program: `RUN BP ZZMATH`.

	VALUE
<code>@who</code>	DON — upper case
<code>@logname</code> , <code>@user</code>	don — lower case
<code>@path</code>	/cygdrive/c/ProgramData/SD/user_accounts/don
<code>@sdsys</code>	C:\ProgramData\SD\sdsys
<code>@user.no</code>	67, the same as <code>system(18)</code>
<code>@tty</code>	empty in a piped session
<code>@system.return.code</code>	1
<code>@user.return.code</code>	0
<code>@crtwide</code> / <code>@crthigh</code>	200 / 9999 — whatever <code>TERM</code> last set

`@who` and `@logname` **differ in case for the same account**. Compare them with `upcase()` on both sides or the test fails on a machine where it worked.

`set.exit.status` sets what SD returns to the operating system when the session ends.

OS.EXECUTE

```
os.execute command {capturing variable}
```

It is gated per account, and a refusal aborts the program rather than setting a status. In an ordinary account:

```
don is not permitted to use OS.EXECUTE at line 10 of .../BP.OUT/ZZMATH
```

The program stops there. There is no `else`, no `on error` and no status to test, so **a program that may run in an account without the right must not reach the statement at all.**

Permission is field 2 of the account's record in the system `os.users` file — not a VOC entry and not a Windows privilege. An administrator's session passes regardless. Ask your administrator to grant it; there is nothing a program can do about it.

LOGMSG

```
logmsg text
```

Writes a line to SD's error log. It has no return value, and **what** `status()` **says afterwards is whatever the previous statement left there** — worth knowing because it looks like a result.

What is not here

A whole family of functions is internal-only, and the compiler's complaint names something else entirely.

This program, in an ordinary account:

```
v1 = kernel(28, 0)
v2 = ospath('C:', 1)
v5 = option('X')
v6 = pterm(1, '')
```

compiles to, at the **last line of the program**:

```
41: Matrix KERNEL is not referenced in a DIM statement
41: Matrix OPTION is not referenced in a DIM statement
41: Matrix OSPATH is not referenced in a DIM statement
41: Matrix PTERM is not referenced in a DIM statement
WARNING: KERNEL is not assigned a value
```

A name the compiler does not know as a function is read as a **matrix reference**, so the error is about a `dim` statement you never wrote, at a line number nowhere near the call. **If a function you are sure exists produces that, it exists and this account may not call it.** A call with three arguments gives a different but equally misleading answer — `sdext(101, 'pw', 'salt')` is *"Right bracket not found where expected"*, because a matrix takes at most two subscripts.

Internal-only: `kernel()` — and therefore the Windows path conversion — `ospath()`, `option()`, `pterm()`, `sdext()`, `testlock()` and `getlocks()`. The compiler's list is longer than that; those seven are the ones this page put in front of it. They are reachable only from a program compiled with `$internal`, which additionally requires an administrator in the `SDSYS` account.

And some statements are restricted the same way. These are *"Unrecognised statement"* in an ordinary account: `set.modes`, `reset.modes`, `remove.token`, `release.lock`, `como`, `quit`, `keyboard.input`, `writeln`, and the whole debugging family — `debug.on`, `debug.off`, `debug.set`, `breakpoint` and `watch`.

`errmsg` **is in the compiler's statement table and does not exist.** It compiles to *"Unrecognised statement"* for everybody. Its opcode was removed in July 2024 and the name was left behind in the table. **Being in the table is not evidence a statement exists.**

`sendmail`, `chgphant()` and `ccall()` **compile for an ordinary account**, and each needs something outside SD before it can do anything: `sendmail` a mail relay, `chgphant` a phantom to change, and `ccall` a C function registered into the executable. Nothing on this page depends on them.

`procread` and `procwrite` belong to the PROC language rather than to BASIC and are only meaningful inside one.

See also

SD Basic - Program Control · SD Basic - Data Conversion · SD Basic - Debugging · SD Basic - Locks and Transactions.

SD Basic - Debugging

Stopping a program where you want it, and looking at what it thinks is true.

SD has a real source-level debugger. It stops on a line, shows you the source, prints any variable with its type, changes one, sets breakpoints, watches a variable for change, and shows the call stack. It is worth ten minutes of learning and it replaces most of the `crt` statements people leave behind.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The transcripts below are the debugger's own output, reproduced exactly as it prints them — including the `help` screen. What you see on your own screen should match character for character.

Nothing works until you compile with DEBUGGING

```
basic bp MYPROG debugging
```

The `debugging` keyword is what makes every other thing on this page possible. Without it the compiler emits no per-line debug information, and the debugger has nothing to stop on. It shows in three ways:

a <code>debug</code> statement in a program compiled without it	"WARNING: DEBUG statement ignored - Not compiling in debug mode", 0 errors , and the program ran straight through
the same program started with the <code>debug</code> verb	ran straight through — no prompt, no message
the same source compiled with <code>debugging</code>	stopped, and the prompt appeared

The warning is a warning, not an error: the program compiles and runs. A `debug` statement in production code is therefore harmless but useless, and the tell that you forgot the keyword is that nothing happens.

Recompile without `debugging` when you are finished. The extra information is per line, and the debugger becomes reachable by anyone who can run the program.

Two ways in

```
debug bp MYPROG          ;* the verb: stop on the FIRST line
```

The verb stops before line 1 has run:

```
:DEBUG BP ZZDBG
1:      crt 'ZZDBG.START'

>
```

```
debug                    ;* the statement: stop where you put it
```

The statement stops on the **next** line, the one that has not run yet:

```
ZZDBG.START
4:      n = n + 1

>
```

Use the verb when you want the whole program, and the statement when you want a particular place — inside an `if` that is only true for one record, for example. The statement is the only way to stop on a condition, because there is no conditional breakpoint.

`>` is the prompt. `pdebug` is the same debugger for a phantom; the `debug` verb refuses a phantom outright with *"Cannot debug program in a phantom process"*.

The commands, as this port prints them

Typing `help` at the `>` prompt gives:

```
BRK n          Set breakpoint on line n
CLR {n}         Clear all or specific breakpoint
DUMP var path   Dump variable to given pathname
HELP           Display this screen
ABORT or Quit   Abort program
```

EXit	Exit subroutine
EP	Exit program
Goto n	Goto line n
Run {n}	Free run or to line n
SRC {n}	Display line n, current line if n omitted
SRC {m,n}	Display n lines starting at line n.
STACK	Display call stack
Step .n	Step n debug elements
Step n	Step n lines
STOP	Stop program
Watch var	Watch for changes to named variable
UnWatch	Cancel Watch action

The capital letters are the abbreviation — S for Step, R for Run, G for Goto, W for Watch, UW for UnWatch, EX for EXit, Q for Quit. ? is a synonym for / below, not for help.

Looking at a variable

```
/name
```

A slash and the name, on three kinds of variable:

```
>/N
Integer: 41

>/K
String (9 bytes): "some text"

>/ARR(1)
String (5 bytes): "first"
```

The type is part of the answer, and it is often the answer. Integer: 41 and String (2 bytes): "41" are different bugs, and so are Integer: 1 and Float: 1.500000. The rest of the shapes:

a variable that has not been assigned yet

Unassigned — **not** an empty string

a name the program never mentions

Variable not defined

a floating point variable

Float: 1.500000

The difference between the first two matters: `Unassigned` means the variable is yours and has no value yet, and *"Variable not defined"* means you have mistyped the name.

A matrix element takes its subscripts: `/ARR(1)` for one dimension, `/ARR(2,3)` for two. `/` on its own repeats the last variable you looked at, printing the same answer again.

```
set name = value
```

changes it. `SET N = 100` then `/N` gives `Integer: 100`, and the program carries on with the new value.

`dump var path` writes a variable to an operating system file, which is the way to look at something too long for the screen.

Stepping and running

<code>s</code>	one line — the debugger prints the next one, <code>5: crt 'ZZDBG.N=' : n</code>
----------------	---

<code>s n</code>	n lines — <code>s 3</code> from line 4 lands on line 7
------------------	---

<code>s . n</code>	n debug elements — finer than a line
--------------------	--

<code>r</code>	run on
----------------	--------

<code>r n</code>	run on until line n
------------------	-----------------------

<code>g n</code>	jump to line n without running the lines in between
------------------	--

<code>ex</code>	run to the end of this subroutine
-----------------	-----------------------------------

<code>ep</code>	run to the end of this program
-----------------	--------------------------------

<code>stop</code>	stop the program
-------------------	------------------

<code>q</code>	abort — prints <code>ABORT : Debugging terminated</code>
----------------	--

An `s` in the middle of an assignment sequence:

```
>/N
Integer: 41

>S
5:      m = n * 2

>/N
Integer: 42
```

— the step executed line 4, `N` moved from 41 to 42, and the debugger is now sitting on line 5.

Breakpoints

```
brk 9      set one on line 9
clr 9      clear that one
clr        clear all of them
```

`brk 9` then `r`:

```
>BRK 9

>R
9:      crt 'ZZDBG.N=' : n : ' M=' : m : ' K=' : k
```

`src` shows you the line numbers to aim at. `src 1,4`:

```
>SRC 1,4
1:      crt 'ZZDBG.START'
2:      n = 41
3:      debug
4:      n = n + 1
```

Breakpoints are on a **line number in the source**, so the source record has to still match the compiled object. The debugger checks the modification time and refuses with *"Unable to find source record"* if the source has moved on; recompile rather than trying to work around it.

There is no conditional breakpoint. Put a `debug` statement inside the `if` instead, and recompile.

Watching a variable

```
w m      stop when M changes
uw       stop watching
```

A watch is the one thing that finds "who is setting this". A watch set at line 4, then `r`:

```
>W M

>R
6:      k = 'some text'
Watched variable M has changed
Integer: 84
```

It ran on, and stopped **after** the line that changed `M`, telling you what it changed to. That is the whole answer to the commonest hard bug in a long program.

There is **one** watch at a time; `w` on another variable replaces it.

The call stack

```
stack
```

From inside a program run from the command prompt:

```
>STACK
1: /cygdrive/c/ProgramData/SD/user_accounts/don/BP.OUT/ZZDBG @ 9
Command processor
```

The object path and the line, then whatever called it. From a program a program can ask the same question without the debugger:

<code>system(1002)</code>	the call stack as a dynamic array — one field per level
---------------------------	---

<code>system(1029)</code>	internal subroutine depth — <code>0</code> at the top, <code>1</code> inside a <code>gosub</code>
---------------------------	---

It is a line-mode debugger here, and that is not a limitation of your terminal

On this port the debugger never draws a full screen. Its own test is `terminfo('sreg')`, and `sreg` — *save screen region* — is a capability of SD's own client terminals, not of a console. In a session on the shipped `windows` terminal type:

<code>terminfo('sreg')</code>	0 characters
<code>terminfo('cup')</code>	16 characters
<code>terminfo('clear')</code>	6 characters

So cursor addressing is there and the region capability is not, and the debugger takes its line-oriented path. **Nothing is lost** — every command works, as the transcripts on this page show — and there is a real gain: because it is line-oriented it can be **driven from a script**. A whole debugging session can be fed down a pipe. `@tty` reads empty in a piped session, and the debugger works there anyway.

Who has it

Read out of an account's own VOC:

VOC ENTRY	RECORD
<code>debug</code>	a <code>V</code> verb, <code>IN</code> , internal command 29 run a program in debug mode
<code>debugging</code>	a <code>K</code> keyword, 53 the compiler keyword
<code>pdebug</code>	a <code>V</code> verb, <code>CA \$PDEBUG</code> the phantom debugger

`debug` and `pdebug` are not in a standard account. Both are on the installer's `TIER.OMIT.STANDARD` list, so an account created at the standard tier has neither verb. `debugging` is a keyword rather than a verb and is unaffected — but with no `debug` verb, the `debug` **statement** is the only way in on such an account.

What is not here

`trace` **is gone**. It was removed from the compiler in July 2024. In an ordinary account it is "*Unrecognised statement*".

The programmatic debug statements are restricted. `debug.on`, `debug.off`, `debug.set`, `breakpoint` and `watch` are internal-only, and all five are "*Unrecognised statement*" in an ordinary account. They are what the debugger itself is built from. `debug` — **the plain statement** — **is not restricted**, and it is the one you want.

`debug.info()` is internal-only for the same reason.

No conditional breakpoints, no data breakpoints beyond the single watch, and no remote attach. The debugger runs in the session that runs the program.

See also

SD Basic - System and Environment · SD Basic - Program Structure · SD Basic - Program Control · SD Basic - Locks and Transactions.

SD Basic - Modern Program Structure

Scope, local subroutines and functions, class modules and objects — writing SD BASIC the way you would write anything else.

SD BASIC has scope and it has objects. A routine can have variables of its own that nothing else can see; a class module can hold state, expose methods and properties, construct and destruct itself, and inherit from another class. None of it is a bolt-on and none of it is new — it is simply not what most MultiValue documentation talks about.

This page is that half of the language. SD Basic - Program Structure covers the traditional shape — one program, one flat variable space, `gosub` to a label, `call` to a catalogued subroutine — and everything here is an alternative to it, not an extension of it.

Everything on this page was compiled and run on SD Core for Windows W1.0-0 before it was written down. Where a listing shows an error, that error was produced, not composed. The programs are in the documentation repository — <https://github.com/dmontaine/SDCoreWindowsDocs> — as `tools\probes\p18-class-base.b`, `p18-class.b` and `p18-objects.b`.

Scope, and the one thing to get straight first

A plain `gosub` label shares every variable in the program. A **local subroutine or function** does not have to:

```
local subroutine name {(arguments)}
...
return
end

local function name {(arguments)}
...
return value
end
```

But a local routine is not a closed scope, and assuming it is will catch you. It can still see every variable in the main program. What it gets that a `gosub` label does not is the ability to declare names that **do not leak back** — its arguments, and anything it declares `private`.

In one program: a local subroutine takes an argument `a` and declares `private working`, and the main program then asks what it can see:

```
ZZMATH.LOCAL.SEES.MAIN=assigned(shared.probe)=1
ZZMATH.PRIVATE.LEAKED=0
ZZMATH.ARGNAME.LEAKED=0
```

a variable set in the main program	visible inside the local routine
------------------------------------	---

a name the routine declared <code>private</code>	not visible outside it
--	-------------------------------

the routine's own argument names	not visible outside it
----------------------------------	-------------------------------

So the model is **shared by default, private on request**. If you want a routine that touches nothing it was not handed, declare every working variable `private` and take everything else as an argument. The compiler helps: naming those variables in the main program afterwards raises *WARNING: WORKING is not assigned a value*, because to the main program they are different names that have never been set.

CALLING THEM: TWO DIFFERENT FORMS, AND NEITHER IS `CALL`

A local subroutine is reached with `gosub`, not `call`.

```
gosub tally(3, 4, answer)
```

`call` looks in the catalogue. Against a local subroutine it compiles cleanly and then fails at run time:

```
000002B3: Unable to load 'TALLY' object code at line 62 of
/cygdrive/c/ProgramData/SD/user_accounts/don/BP.OUT/zzobj
```

A local function must be declared before it is used.

```
deffun doubled(n) local
```

Without that line the compiler has no way to know `doubled(21)` is a call, and reads it as a subscripted array:

```
57: Data item or constant not found where expected
57: Right bracket not found where expected
```

89: Matrix DOUBLED is not referenced in a DIM statement
WARNING: DOUBLED is not assigned a value

A local subroutine needs no such declaration — `gosub name(...)` is unambiguous on its own.

THE `RETURN` IS NOT OPTIONAL

A local routine that falls into its own `end` stops the program. The compiler emits `end` there as a stop, not a return, so there is **no error message and no further output** — and any object still in scope runs its destructor on the way out, which makes it look like a clean finish.

Leave the `return` out and the program prints everything up to the call, nothing after it, and exits zero. **Every one of the five local subroutines in SD's own `DEBUG` program carries an explicit `return`.**

`DEFFUN` DOES MORE THAN DECLARE

<code>deffun name(args) local</code>	a local function, defined below
<code>deffun name(args) calling "!external"</code>	an external one, under a name you choose
<code>deffun name(args) key "F5"</code>	bound to a function key
<code>deffun name(args) var.args</code>	variable argument count

`local` and `calling` are mutually exclusive — a function is either in this program or somewhere else. SD's `DEBUG` uses both forms, three lines apart.

Class modules

A class module is a program declared with `class` instead of `program`. It holds state between calls, exposes what it chooses, and is instantiated as many times as you like.

```
class name {inherits base {, base ...}} {max.args n}

private variable, ...
public variable, ...

public subroutine create.object    ... end
public subroutine destroy.object  ... end
public subroutine name{(args)}    ... end
public function name{(args)}      ... end
get name                          ... end
set name(value)                   ... end
```

```
end
```

Each member ends with its own `end`, and the module ends with one more. `private` and `public` declare member *variables*; `public subroutine` and `public function` declare methods; `get` and `set` declare a property.

A class is catalogued and instantiated like any other program, and **a private catalogue is enough** — nothing here needs an elevated session:

```
:basic bp zzcls
Compiling bp zzcls
0 error(s)
:catalog bp zzcls
ZZCLS added to private catalogue
```

Objects

<code>object(catalogued.name {, arguments})</code>	create an instance
<code>objinfo(variable, 0)</code>	1 if the variable holds an object
<code>objinfo(variable, 1)</code>	the class name
<code>variable = ''</code>	release it

```
o = object('ZZCLS')
```

```
ZZMATH.ISOBJ=1
ZZMATH.CLASS=ZZCLS
```

`->` REACHES EVERYTHING

The member operator is `->`, and it is the same operator for all four kinds of member. This is the part most worth having in front of you:

YOU WRITE

WHAT IT DOES

```
o->deposit(40)
```

calls a **public subroutine**

YOU WRITE	WHAT IT DOES
<code>o->total(10)</code>	calls a public function and yields its answer
<code>o->label</code>	reads a public variable
<code>o->label = 'Direct'</code>	writes one
<code>o->owner</code>	runs the <code>get</code> routine for that property
<code>o->owner = 'MIXED Case Name'</code>	runs the <code>set</code> routine for it

All six on one instance, with `deposit` called twice with 40 and 2. `total` multiplies by its argument; the `set owner` routine lower-cases what it is given into the public variable `label`, and `get owner` wraps it:

```
ZZMATH.TOTAL.X1=42
ZZMATH.TOTAL.X10=420
ZZMATH.LABEL.INITIAL=unnamed
ZZMATH.LABEL.AFTER=Direct
ZZMATH.OWNER=owner<mixed case name>
ZZMATH.LABEL.AFTER.SET=mixed case name
```

A property is one name with two routines behind it. `o->owner` on the right of an assignment runs `get owner`; on the left it runs `set owner(value)`. To the caller it reads exactly like a variable, which is the point — the class can validate, transform or compute without the caller knowing.

PRIVATE MEMBERS DO NOT EXIST FROM OUTSIDE

```
:crt p->balance
000003C1: Unrecognised property/method name (BALANCE). at line 92
```

Note the wording. It is not "*private*" — from outside the class the name is not a member at all. `private` is genuine encapsulation, not a convention.

The way to expose one is to write a public routine that returns it, which is what `reveal()` does in the class above.

Construction and destruction

`create.object` and `destroy.object` are hooks, not calls you make. **They run by themselves**, and this is where an object earns its keep over a set of subroutines and a common block:

```
ZZBASE.CREATE.OBJECT
ZZCLS.CREATE.OBJECT balance=0
...
ZZMATH.BEFORE.UNLOAD
ZZCLS.DESTROY.OBJECT balance=42
ZZBASE.DESTROY.OBJECT
ZZMATH.AFTER.UNLOAD
```

construction runs base first

`ZZBASE` then `ZZCLS`

destruction runs in reverse

`ZZCLS` then `ZZBASE`

releasing the variable is what fires it

`o = ''`, and the two destructors ran between the lines either side of it

That is the one thing a common block cannot give you. A file left open, a lock held, a socket connected — a destructor closes it when the last reference goes, whatever route the program took to get there. SD's own client library uses exactly this: `destroy.object` in `SDCLIENT` disconnects the session if one is still up.

Inheritance

```
class derived inherits base
```

The base class's public members become the derived class's, and the derived class may override them. For a class inheriting `ZZBASE`:

```
ZZMATH.WHOAMI=ZZCLS
ZZMATH.TAG=base-tag
ZZMATH.REVEAL=base-private
```

`whoami()`

the derived version won — both classes define it

`tag`

a public **variable** of the base, reached through the derived object

`reveal()`

a base public function returning the base's own `private` member

The last row is the interesting one: **a base class's private state stays private, and reaches the outside only through whatever the base chooses to expose.** Inheritance does not flatten encapsulation.

THE RUNTIME FORM

```
inherit object
disinherit object
```

These are statements, used **inside a class**, that add or remove another *instance's* members at run time rather than at compile time. Using either outside a class module is an error — *DISINHERIT not in class*. The `inherits` clause is the one to reach for first; the statements exist for the case where what you inherit is decided while running.

`objinfo` and releasing

```
objinfo(v, 0)    1 if v holds an object, 0 otherwise – safe on any variable
objinfo(v, 1)    the class name; an error if v is not an object
unload.object name
```

Key 0 is the safe one. It is the only key that does not require the variable to be an object already, so it is what you test with before anything else.

Where SD itself does this

The claim that nothing uses this is wrong, and it is worth knowing what does.

`gp1.bp/SDCLIENT`

SD's own client library is a class module — 1,040 lines, 33 members, `create.object` and `destroy.object` both defined. Connecting to another SD server from BASIC means instantiating it

`gp1.bp/DEBUG`

five `local subroutine`s and a `local function`, with `deffun ... local` above them

Choosing between the two shapes

REACH FOR

WHEN

<code>gosub</code> label	a few lines, used once, sharing the program's variables on purpose
local subroutine or function	anything with working variables of its own, or that you want to be sure touches nothing else
catalogued subroutine	logic more than one program needs
class module	state that has a lifetime — something opened, held and closed — or several instances of the same thing at once

See also

[SD Basic - Program Structure](#) · [SD Basic - Program Control](#) · [SD TCL - Programs and the Catalogue](#).

SD TCL - The Command Processor

How a command line is read, what the VOC does with it, and how sentences and paragraphs run.

TCL is the prompt you get when you start SD. It is not a shell: it does not search a path, it does not expand wildcards, and it has no built-in commands of its own. **Every command you type is a record in a file**, and this page is mostly about that file and about the small set of verbs that control the session you are typing into.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case, which is what this port uses on disk. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The prompt

The command processor is a BASIC program, `$CPROC`, and it runs a loop: show any waiting messages, print the prompt, read a line, dispatch it, repeat. The prompt character is `?` and it is what `system(26)` reports.

Messages sent by `message` are printed at the prompt, not when they arrive. They queue in the `$ipc` file and the loop drains that queue immediately before each prompt, so a message sent to a session sitting at TCL appears at once and a message sent to a session running a long report appears when the report finishes.

`report.src` turns on a line after every command reading `SRC =` and the value of `@system.return.code`. It takes `on`, `off`, or nothing at all, in which case it toggles.

Where a command goes

The first word of the line is looked up in the account's `voc` file. **What comes back decides everything else**, because field 1 of a VOC record begins with a type character:

V	a verb — the common case
K	a query-processor keyword, which may also be a verb (see below)
S	a sentence: one stored command line
PA	a paragraph: several stored command lines, with flow control

M	a menu
Q	an indirect pointer to a file in another account
F	a file
R	a pointer to a record in another file, re-read and re-parsed
X	miscellaneous data — not a command
D	a data descriptor, which is what dictionary records are

The rest of field 1 is free text and is ignored, so a verb record whose field 1 reads `Verb to compile SD BASIC program` is type `V` and the sentence is a comment. **This is worth knowing before you edit a VOC record by hand:** the first character is load-bearing and the remaining thirty are not.

NAMES ARE TRIED AS TYPED, THEN LOWER, THEN UPPER

This port shipped a change here. Where SD used to try the name exactly as typed and then in upper case, it now tries **as typed, then lower case, then upper case**. That is what lets a lower-case `voc`, `bp` and `newvoc` on disk coexist with code and habits that still type names in upper case, and it is why you can type `list voc` or `LIST VOC` and reach the same file.

Account names are the deliberate exception. They are still folded upward, and that is what makes signing in case-insensitive.

A HYPHEN IS ACCEPTED WHERE THE VERB HAS A DOT

After the three case attempts, the command processor tries the name again with any hyphens changed to dots. **So every dotted verb also answers to a hyphen.**

```
:clear-select
Cleared numbered select list 0
:clear.select
Cleared numbered select list 0
```

This is a spelling variant rather than a second verb, and two controls show it:

```
:zzz-nosuch
zzz-nosuch is not in your VOC
:ct voc create-account
Record 'create-account' not found
```

The first shows the fallback is not accepting anything at all. The second shows there is no `create-account` record in the VOC — the resolution happens in the command processor, not in the file.

The dotted spelling is the documented one and is what every page here uses. The hyphen form is supported, and it is worth knowing because a typed hyphen will not produce the error you expect.

IF THE VOC HAS NOTHING, THE CATALOGUE IS TRIED

A name that is not in the VOC is looked for in the **private catalogue** of the account and then in the **global catalogue**, `gcat`. A hit is treated exactly as though the VOC had held `V` / `CA` / *name*. This is why a program you have catalogued can be run by typing its name with no VOC record of its own.

If neither has it, the command is refused by name.

A KEYWORD CAN BE A VERB

Four records that ship are a keyword **and** a verb in one record: `break`, `count`, `display` and `off`. Their field 2 is the keyword number the query processor uses, and **fields 3 onward are a complete verb record**, which the command processor re-parses when the name is typed as a command.

This matters for counting what an account has. A tally of VOC records whose field 1 begins with `V` misses all four, and `count` is not a marginal verb. A standard account has **82** verbs, not 78.

What kind of thing a verb is

Field 2 of a verb record says how to run it, and field 3 says what to run:

<code>CA</code>	<i>name</i>	a catalogued program — 97 of the shipped verbs
<code>IN</code>	<i>n</i>	internal verb <i>n</i> , handled by the command processor itself — 45
<code>OS</code>	<i>text</i>	an operating-system command — <code>sh</code> and <code>!</code>
<code>CS</code>	<i>path</i>	a locally catalogued function

Field 4 carries dispatch options and **field 5 names a security subroutine**. If field 5 is present, that subroutine is called before the verb runs and can refuse it, in which case you are told the command is restricted. None of the shipped verbs uses field 5 — **the tiering in this port is done by giving or withholding the VOC record**, not by a security subroutine — but the mechanism is there for a site that wants a verb guarded rather than absent.

There are **41 internal verbs**, numbered, and the numbers are positional in the command processor's dispatch list. Several names share one: `off` and `quit` are both internal verb 1, `clr` and `cs` are both 2, and each of `clear.data`, `clear.input`, `clear.prompts` and `clear.select` has a run-together spelling (`cleardata` and the

rest) pointing at the same number.

Sentences and paragraphs

A **sentence** is a VOC record holding one command line. Typing its name runs that line with anything else you typed appended, so a sentence is a command with its first arguments filled in.

A **paragraph** holds several lines and is the closest thing SD Core has to a shell script. Six verbs exist for paragraphs and **four of them are refused outside one**:

<code>if</code> <i>value op value ; command</i>	conditional. Only inside a paragraph
<code>go</code> <i>label</i>	jump to a label. Only inside a paragraph
<code>stop</code>	end this paragraph, return to the caller
<code>abort</code> <i>{text}</i>	end the paragraph and everything that called it
<code>display</code> <i>text</i>	write a line to the terminal
<code>pause</code>	print a <i>press return</i> prompt and wait

`go` searches forward from the current line for a line beginning with the label; **a label prefixed with `@` is searched for from line 1 instead**, which is how you jump backwards. Labels carry a trailing colon, and one is added for you if you leave it off.

`display` understands a leading cursor-position clause, so `display @(0,0) ready` positions and then writes. A trailing colon suppresses the newline.

`data` **lines are read ahead of the verb, not executed in sequence.** Before a verb runs, the command processor scans forward over any immediately following `data`, comment and blank lines and stacks their text as typed-ahead input. So the `data` lines that feed a program are written *after* the command that runs it, and they must follow it without an intervening ordinary command.

Paragraphs nest, and `abort` unwinds all of them.

Continued in

SD TCL - The Command Stack — the command stack, the @variables, and the session verbs.

SD TCL - The Command Stack

Recalling and re-running commands, the @variables, and the session verbs.

This page continues SD TCL - The Command Processor.

The command stack

Every line you type is kept. The stack is edited with commands beginning with a dot, and `n` below is a stack position — 1 is the most recent.

<code>.l {n}</code>	list the last <i>n</i> stack lines. Defaults to 20, or a screenful
<code>.l name</code>	show a VOC record, one numbered line per field
<code>.r {n}</code>	recall line <i>n</i> to the top of the stack
<code>.r name</code>	load a sentence or paragraph from the VOC into the stack
<code>.s name s e</code>	save stack lines <i>s</i> to <i>e</i> into the VOC as <i>name</i>
<code>.a {n} text</code>	append <i>text</i> to line <i>n</i>
<code>.i {n} text</code>	insert <i>text</i> as line <i>n</i>
<code>.c {n} / old / new / {g}</code>	change <i>old</i> to <i>new</i> in line <i>n</i> ; <code>g</code> for every occurrence
<code>.d {n}</code>	delete line <i>n</i>
<code>.d name</code>	delete a sentence or paragraph from the VOC, after asking
<code>.u {n}</code>	convert line <i>n</i> to upper case
<code>.x {n}</code>	execute line <i>n</i>
<code>.x file id</code>	execute a sentence or paragraph held in any file, not just the VOC
<code>.?</code>	print this list

n defaults to 1 everywhere, and the spaces shown are required. `.d2` deletes the second stack line and `.d2` looks for a VOC record called `2`.

`.s` and `.r` together are the whole workflow: type the lines, then save them under a name, then recall them later to edit. `.s` **writes a sentence when the range is one line and a paragraph when it is more**, so the record type follows from what you saved rather than from anything you say.

`.x` *file id* is the one that reaches outside the VOC — the file name and the record id are each folded as typed, then lower, then upper.

A dot command that is not one of these is not an error. It is left alone and run as an ordinary command, so a verb whose name begins with a dot still works.

`.d` does not find a lower-case VOC record typed in upper case, and `.l` and `.r` do. `.l` and `.r` try the name as typed, then lower case, then upper case. `.d` tries only as typed and then upper case. A paragraph saved as `daily` can be listed and recalled by typing `DAILY` and cannot be deleted by typing `DAILY`. **Type the name in the case you saved it in and all three work.**

EDITING THE LINE YOU ARE ON

There are two help texts and they are different. `?` on its own prints the key map below; `.?` prints the dot-command list above. Neither is the `help` verb, which does not exist — see *What is not here*.

The key map is the same whether you reached the line by typing it or by recalling it:

KEY		ALSO
<code>Ctrl-A</code>	start of line	Home
<code>Ctrl-E</code>	end of line	End
<code>Ctrl-B</code> / <code>Ctrl-F</code>	back / forward one character	Cursor left / right
<code>Ctrl-P</code> / <code>Ctrl-N</code>	previous / next command	Cursor up / down
<code>Ctrl-Z</code>	previous command	Cursor up
<code>Ctrl-R</code>	reverse search	
<code>Ctrl-D</code>	delete character	Delete
<code>Ctrl-H</code>	backspace	Backspace
<code>Ctrl-K</code>	kill line	
<code>Ctrl-O</code>	toggle overlay	Insert

KEY	ALSO
<code>Ctrl-T</code>	transpose characters
<code>Ctrl-U</code>	upcase all
<code>Ctrl-G</code>	exit the stack

A line ending in `?` is put on the stack without being run. That is the way to park a half-written command while you go and look something up.

SAVING THE STACK ACROSS SESSIONS

<code>save.stack {name}</code>	write the stack to a named record
<code>get.stack {name}</code>	read it back
<code>clear.stack</code>	empty it

Both prompt for the name if you leave it off.

@-variables

`set` assigns an @-variable, which is then substituted into later command lines wherever its name appears.

```
set name value
set name eval expression
```

Without `eval` the value is taken literally. With `eval` the text is evaluated first, and the evaluation handles the four arithmetic operators on numeric operands. `list.vars` shows what is currently set.

The variables the system sets are readable the same way — `@system.return.code` is the one worth knowing, because most verbs set it and `report.src` exists to show it.

Session verbs

<code>who</code>	your user number and the account you are in
<code>who.am.i</code>	the same information at more length
<code>logto</code> <i>account</i>	change account without logging out
<code>off</code> · <code>quit</code>	end the session
<code>option</code> { <i>name</i> } { on off display }	set or show a session option
<code>option all off</code>	turn every session option off at once
<code>alias</code> <i>command target</i>	make <i>command</i> run as <i>target</i>
<code>alias</code> <i>command</i>	remove that alias
<code>alias</code>	list the aliases
<code>clear.abort</code>	clear a pending abort condition
<code>set.exit.status</code> <i>n</i>	set the numeric status SD exits with
<code>report.src</code> { on off }	show <code>@system.return.code</code> after each command

`who` answers with your user number and the account you are in:

```
39 DON
```

After a `logto` it grows a third part, and that is the useful one:

```
29 SDSYS from DON
```

`from DON` **is the account you logged in as**, not the one you are in and not your Windows account. So the short form means *I am still where I started* and the long form means *I have moved* — which makes `who` the quick way to find out whether a `logto` actually took effect.

`logto sdsys` **requires an elevated Windows session in this port**. Entering `SDSYS` is what confers administrator rights, so it is gated on the operating system rather than on an SD password, and the elevation obtained by one `logto` is deliberately not carried into the next one. `who` reports the account you are in, which is the quick way to confirm a `logto` actually happened.

What is not here

There is no online help and `help` is not a verb. The command processor still has internal verb 14 reserved for it and the routine is an empty stub — its body is commented out and it returns immediately. **No VOC record points at it**, in any account type, so the name is not even recognised. The F1 key at the command prompt reaches the same empty routine and therefore does nothing. **This documentation is the help system.**

`umask` is implemented and unreachable. Internal verb 35 is a working routine that reports or sets the file-creation mask, and **no VOC record points at it either**. It cannot be typed. `umask()` from SD BASIC still works — see SD Basic - System and Environment.

PROC is removed. So are `sed` and `update.record`. A `PQ`-type VOC record is **refused by name** rather than being reported as a bad dispatch code, because the record itself is valid PROC and it is the interpreter that is gone. Nothing that ships is type `PQ`, so this can only be met in a VOC record somebody wrote.

`menu` records are dispatched but no menu ships. Type `M` is handled and there is no shipped example to look at.

Who has these verbs

Everything on this page is in a **standard** account except the last two, which are administrator-only:

standard

`abort` `alias` `clear.abort` `clear.stack` `display` `get.stack` `go` `if` `list.vars`
`logto` `off` `option` `pause` `quit` `report.src` `save.stack` `set` `set.exit.status`
`stop` `who` `who.am.i`

Everything on this page is in a standard account. An account that does not have a verb does not have the VOC record for it — the name is simply not recognised rather than refused.

The two `OS` verbs are the exception and are not documented here. `sh` and `!` reach the Windows shell, are administrator-tier, and are gated a second time by a list of who may use them. They are in the **administrator documentation**, under *Operating System Access*, which is a separate set.

See also

SD TCL - Files and Records · SD Basic - Program Control · SD Basic - System and Environment.

SD TCL - Files and Records

Making, configuring and removing files; copying, renaming, deleting and dumping the records inside them.

This page covers the verbs that act on a whole file or on the records in it, from the command line. What a program does with the same files is SD Basic - File Handling; this is the part you type.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case, which is what this port uses on disk. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The two kinds of file, and what they look like in Explorer

Both kinds are a directory on Windows. This surprises people who expect a database file to be one file.

dynamic	a directory holding components named <code>%0</code> , <code>%1</code> , ... The records are hashed across them and none of it is readable from outside SD
----------------	--

directory	a directory holding one ordinary file per record , named by the record id
------------------	--

`voc`, `$ipc` and `dict.dic` are dynamic. `messages`, `accounts` and `bp` are directory files — which is why you can open a BASIC program in Notepad and why `gpl.bp` is readable in the source tree.

A dictionary is a separate file that travels with the data file. Verbs that can act on either take `dict` in front of the file name; with no `dict` they mean the data portion.

The shipped file names are lower case on disk in this port. `voc`, `bp`, `newvoc`, `accounts`, `messages` and the rest were renamed. NTFS matches without being asked, so this changes nothing about what you can type — but it is what you will see in Explorer and in the path text SD reports back.

Making and removing files

```
create.file {dict|data} voc.name directory
create.file {dict|data} voc.name dynamic {parameters}
```

```
create.file voc.name using dict filename
```

`create.file` makes the file **and** the VOC record that names it. Without `dict` or `data` it makes both portions. `using dict` *filename* takes the dictionary from an existing file instead of creating an empty one, and `no.query` suppresses the confirmation.

Dynamic files take parameters, all optional:

<code>minimum.modulus</code> <i>n</i>	the floor the file will not shrink below
<code>group.size</code> <i>n</i>	group size in 1 KB units
<code>large.record</code> <i>n</i>	the size above which a record is stored separately
<code>split.load</code> <i>n</i> · <code>merge.load</code> <i>n</i>	the load factors that trigger growth and shrink
<code>version</code> <i>n</i>	file format version
<code>directory</code> <i>path</i>	put the file at <i>path</i> instead of in the account
<code>no.case</code>	record ids are case-insensitive
<code>no.resize</code>	disable automatic resizing

```
delete.file {dict|data} voc.name {force} {no.query}  
clear.file {dict|data} file
```

`delete.file` removes the file and its VOC record. **force** is needed if the file is not empty. `clear.file` empties a file and keeps it, along with its dictionary and its indexes.

Changing a file after it exists

```
configure.file {dict} voc.name {parameters}  
configure.file {dict} voc.name default
```

The parameters are the `create.file` ones, plus `immediate` to do the work now rather than lazily, and `dynamic` or `directory` to change the file's kind. `default` puts the settings back.

Two of them restructure the whole file. Changing `group.size` or `version` requires a full restructure; the rest take effect without one. Plan for the time and the disk on a large file.

`no.case` / `case` and `resize` / `no.resize` are the pairs that turn a setting on and off. For a directory file, `binary` turns on mark mapping.

Looking at a file

```
analyse.file {dict} file {statistics} {lptr}
```

Without `statistics` it reports the shape: the account, the file name, the path, the type, the group size, the large-record size, the minimum and current modulus, the load factors and the size in bytes. With `statistics` it walks the whole file and adds record counts, per-group minimum, maximum and average, and a size distribution.

`statistics` reads every group, so it is not free on a big file.

`analyze.file` is the same verb under the other spelling.

```
fstat filename on          start collecting, clearing the counters
fstat filename {lptr}      show what has been collected
fstat filename off         stop collecting
fstat global {lptr}        the global counters
fstat reset                clear the global counters
fstat                      the periodic global summary
```

`fstat` only works on dynamic files. A directory file named on the command line gets a warning; one arriving through a select list is skipped silently. Where a file name is taken, a select list or several names may be given instead.

```
hsm {on|off|display} {user n}
```

The Hot Spot Monitor. `display` is the default.

```
list.files
```

Reports how many files are open now, the peak for the session and the configured limit, then lists them:

```
Number of files open = 2.  Peak = 6.  Limit = 80.
```

The limit is the `NUMFILES` configuration parameter. **The peak is what matters when you are sizing it** — the current figure moves constantly and tells you almost nothing.

Reaching a file in another account

```
set.file account file.name pointer.name
```

This writes a **Q-pointer** into your VOC — an indirect pointer naming another account and a file in it. After it, *pointer.name* is usable wherever a file name is, and the data stays where it is.

The account must be in the accounts register, and the name is folded to upper case before the register is read. If it is not there you get *Account name '...' is not in register* and nothing is written.

Read what `set.file` prints, not just whether it printed. Its refusals echo the names you passed in, so a script that checks its output for the pointer name it asked for will find that name in the refusal as well as in the success. Check for the success wording, and treat *not in register* and *not found* as disqualifiers.

Copying, deleting and renaming records

```
copy from {dict} src.file {to {dict} tgt.file} {s1{,t1} {s2{,t2}}...}  
copy from {dict} src.file {to {dict} tgt.file} all
```

With no `to` clause the copy is **within the source file**, which is how you duplicate a record under a new id. Each `s,t` pair is a source id and the id to give the copy; a bare `s` keeps the id. Options:

<code>overwriting</code>	replace records that already exist in the target
<code>updating</code>	copy only records that already exist in the target
<code>deleting</code>	delete the source record after a successful copy
<code>reporting</code>	list what was copied
<code>binary</code>	force binary mode when one side is hashed and the other a directory

`@system.return.code` comes back as the **number of records copied**, or negative on error.

```
copy {dict} filename id... {(options)}
```

The Pick-style copy. Its options are single letters in a trailing bracket: `d` delete source, `i` suppress the id, `n` no pagination, `o` overwrite, `p` to the printer, `s` suppress field numbers, `t` to the terminal.

```
delete {dict} file.name {record.name} {no.query}
delete {dict} file.name all
```

`@system.return.code` is the **number of records deleted**, or negative on error.

```
cname old.file.name to new.file.name
cname {dict} file.name old.record.name to new.record.name
```

A comma may be used instead of `to`. `rename` is the same verb under the other name. `@system.return.code` is `0` on success and `-1` if the arguments were wrong.

`cname` **on a file renames the VOC record too**, which is the point — the name you type and the name on disk stay together.

Seeing what is actually in a record

```
ct {dict} file.name {record.name... | *} {options}
dump {dict} file.name {record.name... | *} {options}
```

Both are the same program. `ct` is a field-by-field listing; `dump` reports the same data in binary. Options: `hex` for hexadecimal, `binary` for a full-width report that is not field-based, `no.query` to skip the prompt when a select list is in use, and `lptr {n}` to send it to a print unit.

`ct` **is the one verb here a standard account has**; `dump` is withheld. They run the same code, and the line between the tiers runs between the two VOC records rather than through the program.

Triggers

```
set.trigger file.name trigger.name {modes}
set.trigger file.name ""
```



```
set.trigger file.name
```

Set, remove, or display a file's trigger function. An empty string in quotes removes it; no argument at all displays it.

Who has these verbs

Almost everything on this page is withheld from a standard account. That is deliberate: these are the verbs that change data in bulk.

standard

`ct` `list.files` `set.file`

programmer

`analyse.file` `analyze.file` `clear.file` `cname` `configure.file` `copy` `copyp`
`create.file` `delete` `delete.file` `dump` `fstat` `hsm` `rename` `set.trigger`

A standard account can therefore look at a record and point at a file, and cannot create, empty, delete, copy or rename anything. An account that does not have a verb does not have the VOC record for it, so the name is not recognised at all rather than refused.

What is not here

Indexes are their own subject and are not on this page. For what an index is and what it does to a file, see SD Basic - Alternate Key Indexes.

Selecting records is the query processor's job, not these verbs'. `copy`, `delete` and `ct` all accept a select list built by `select` or `sselect`, and that is how they are usually driven over more than a handful of ids.

`reformat` and `sreformat` rewrite records through a dictionary and are run by the query processor, so they are documented with it.

There is no verb that repairs a damaged file. `analyse.file statistics` will tell you a file is badly overflowed; `configure.file` is what acts on that.

See also

SD TCL - The Command Processor · SD Basic - File Handling · SD Basic - Select Lists.

SD TCL - The Query Processor

Choosing records, sorting them, and printing a report, without writing a program.

The query processor is how you ask a file a question from the command line. It selects records, sorts them, formats them into columns and totals them, and it does all of that from one line of typing.

Twelve verbs on this page are the same program. `list`, `sort`, `count` and the rest are one catalogued program reached through fourteen VOC records, each carrying a number that says which verb was typed. Two of the fourteen — `select` and `sselect` — build a list instead of printing one and are covered in SD TCL - Select Lists.

SD folds case, so a query may be typed in either case. Commands and keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0. The file being queried is `voc`, because it is the one file every account has and it is interesting enough to ask real questions of.

The shape of a query

```
verb {dict} file {selection} {sort} {output fields} {modifiers}
```

Everything after the file name is optional, and the parts may be given in almost any order. The shortest useful query is a verb and a file:

```
count voc
```

```
418 record(s) counted
```

That number is particular to the account it was run in. Every account has a `voc` and no two need hold the same records — a standard account's is smaller than a programmer's, and `SDSYS`'s is different again. The counts on this page came from an administrator account.

The verbs

<code>list</code>	print the records
<code>sort</code>	print them in order
<code>count</code>	print how many there are, and nothing else
<code>sum</code>	total a field across the records
<code>list.item</code> · <code>sort.item</code>	print whole records, field by field, rather than in columns
<code>list.label</code> · <code>sort.label</code>	print in label format
<code>search</code>	select records by looking for text inside them
<code>show</code>	build a select list interactively
<code>reformat</code> · <code>sreformat</code>	write the selected fields out to another file

`sort` is `list` with an order imposed; every other pair works the same way, the `sort` half being the ordered one.

Choosing records

`with` introduces a selection clause:

```
count voc with dispatch # ""
```

```
144 record(s) counted
```

That one counts the verbs in the account, because `dispatch` is a dictionary entry that is empty for anything that is not a verb.

THE OPERATORS, AND THEIR SYNONYMS

SD accepts several spellings of each operator and they are exactly equivalent — the same token underneath. Each row below was run, and the spellings in a row returned identical counts on the same data.

equal	= · eq
not equal	# · ne · not
less than	< · lt · before
less or equal	<= · le
greater than	> · gt · after
greater or equal	>= · ge
matches a pattern	like · matches · matching
does not match	unlike · not.matching
sounds like	said · spoken

`before` and `after` read well on dates and are the same operators as `<` and `>`. All three spellings of *not equal* returned **331** on the same query, and `<`, `lt` and `before` all returned **381**.

JOINING CLAUSES

`and` (or `&`) and `or` join clauses. **Each clause repeats** `with`:

```
count voc with dispatch # "" and with dispatch # "CA"
```

```
47 record(s) counted
```

PATTERNS

`like` takes the same pattern language as SD BASIC's `matches` — `0x` for any number of characters, `1a` for one letter, `3n` for three digits, quoted text for a literal. It is set out in SD Basic - String Functions.

```
count voc with f1 like "V0X"
```

```
140 record(s) counted
```

A pattern that is not valid selects nothing and does not complain. Writing that same query as `like "V"]` returns **0 record(s) counted** — not an error, not a warning. **Always prove a pattern against a case you know matches** before believing a small answer, and be especially careful of a query that returns zero when you expected few.

OTHER WAYS TO CHOOSE

<code>every</code>	apply the test to every value of a multi-valued field
<code>when</code>	select values within a record rather than whole records
<code>no</code> <i>field</i>	records where the field is null
<code>unique</code>	one record per distinct value
<code>no.case</code>	compare without regard to case

Sorting

`by` sorts ascending, `by.dsnd` descending. Both take a field name, and both may be repeated for a second and third key.

```
sort voc with dispatch = "OS" by @id dispatch processor
```

```
@ID..... Dsp Processor.....
!          OS
sh         OS
2 record(s) listed
```

`@id` is the record id, and it is a field name like any other.

What gets printed

Name the fields you want after the selection, and they become the columns in the order you name them. With no fields named, the file's default listing is used — for `voc` that is a description built by the dictionary.

The column heading, its width and its format all come from the **dictionary** entry for the field, not from the query. `Dsp` above is the display name of the dictionary entry called `dispatch`.

<code>id.sup</code>	leave the record id out
<code>col.hdg</code> "text" · <code>display.name</code>	override a column heading
<code>fmt</code> "code"	override the format
<code>conv</code> "code"	override the conversion
<code>as</code> name	use another dictionary entry's formatting
<code>col.spaces</code> n	change the gap between columns
<code>vert</code>	one field per line instead of columns
<code>dbl.spc</code>	a blank line between records
<code>col.sup</code> · <code>col.hdr.sup</code> · <code>hdr.sup</code>	suppress column headings, both, or the page heading
<code>count.sup</code>	suppress the trailing record count
<code>det.sup</code>	suppress the detail lines, leaving totals

Headings and footings

By default the page heading is **the query you typed**, followed by a page number — which is genuinely useful on a printed report, because it says how the report was produced. `heading` replaces it:

```
sort voc with processor = "$QPROC" by @id dispatch processor heading "QP verbs"
```

```
QP verbs
@ID..... Dsp Processor.....
count      CA  $QPROC
list       CA  $QPROC
list.item  CA  $QPROC
list.label CA  $QPROC
reformat   CA  $QPROC
search     CA  $QPROC
select     CA  $QPROC
```

```

show          CA    $QPROC
sort          CA    $QPROC
sort.item     CA    $QPROC
sort.label    CA    $QPROC
sreformat     CA    $QPROC
sselect       CA    $QPROC
sum           CA    $QPROC
14 record(s) listed

```

That is the query processor listing itself — all fourteen VOC records that reach it, including `count`, which is a keyword record that doubles as a verb.

`footing` does the same at the bottom of the page. Both accept the usual options inside the text for page number, date and time.

Totals, averages and breaks

<code>total</code> <i>field</i>	a column total
<code>average</code> · <code>avg</code>	the mean
<code>max</code> · <code>min</code>	the largest and smallest value
<code>enum</code> · <code>enumerate</code>	count the values rather than total them
<code>percent</code> · <code>pct</code> · <code>%P</code>	express as a percentage
<code>calc</code> · <code>calculate</code>	evaluate an I-type against the totals
<code>break.on</code> <i>field</i>	start a new subtotal group when the field changes
<code>break.sup</code> <i>field</i>	break on a field without printing it
<code>grand.total</code> "text" · <code>caption</code>	label the final total line

`break.on` is what turns a flat listing into a report with subtotals; the field it breaks on almost always wants to be a sort key as well, or the groups come out scattered.

Taking only some of the records

<code>sample</code> n · <code>first</code> n	the first n records
--	-----------------------

<code>sampld</code> n	every n th record
-------------------------	---------------------

A sampled listing says so in its trailing line — `Sample of 3 record(s) listed` rather than `3 record(s) listed` — so a partial report cannot be mistaken for a whole one.

`sample` **TAKES THE FIRST RECORDS FOUND AND THEN SORTS THEM. IT IS NOT THE FIRST n IN SORTED ORDER.** Adding `sample 3` to the sorted listing above returns `count`, `show`, `sort` — not `count`, `list`, `list.item`, which is what the first three of that report actually are. **The sample is taken during selection and the sort happens afterwards, on the sample.** Use it to see the shape of a report cheaply; do not use it to find the top three of anything.

Where the output goes

<code>lptr</code> $\{n\}$	to print unit n , 0 by default
---------------------------	----------------------------------

<code>crt</code>	to the screen, overriding a default of print
------------------	--

<code>no.page</code> · <code>nopage</code>	do not pause at the end of each screen
--	--

<code>file</code>	to a file rather than a device
-------------------	--------------------------------

Printing, print units and `$hold` are in SD TCL - Printing and Spooling.

Querying a dictionary

Put `dict` between the verb and the file name and the query runs against the dictionary instead of the data. It is the quickest way to find out what field names a file will accept:

```
list dict voc sample 14
```


@ID.....	TYPE	LOC.....	CONV..	NAME.....	FORMAT	S/M
@ID	D	0		@ID	20L	S
NAME	D	0		NAME	16L	S
F1	D	1		F1	5L	S
DATA.NAME	D	2		DATA Pathname	18L	M
...						

Sample of 14 record(s) listed

D entries name a field by position, **I** entries compute a value, and **PH** entries are phrases — stored fragments of query that expand where they are named.

Who has these verbs

standard

count list list.item list.label search show sort sort.label sum

programmer

reformat sreformat sort.item

Reading and reporting are available to every account; the two verbs that **write** their result into another file are not.

See also

SD TCL - Select Lists · SD TCL - Files and Records · SD Basic - String Functions.

SD TCL - Select Lists

Choosing a set of records once and then using it, saving it, and combining it with another.

A select list is a list of record ids that SD is holding for you. You build one with `select`, and **the next command you type works on those records instead of the whole file**. That one sentence is most of this page.

SD folds case, so a command may be typed in either case. Commands and keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, in an administrator account. The file being queried is `voc`, because it is the one file every account has.

The one thing to understand first

```
select voc with dispatch = "OS"
```

```
2 record(s) selected to list 0
```

Now the file has 418 records and the list has 2. **The next command sees 2:**

```
count voc
```

```
2 record(s) counted
```

That is the point of select lists and it is also the trap. A list you built and forgot about silently narrows the next thing you do, and nothing about the answer says so — `2 record(s) counted` is not obviously wrong. **A surprisingly small answer is very often a leftover list.**

The list is consumed by the command that uses it. Run `count voc` again and you get 418, because the list is gone.

THE PROMPT TELLS YOU

While a list is active the command prompt gains a second colon:

```
:select voc with dispatch = "OS"  
2 record(s) selected to list 0  
::
```

:: means a list is waiting. **:** means there is not one. This is the quickest way to answer *why did that only find two records* before you go looking for a bug.

`clear.select` throws lists away without using them:

<code>clear.select {n}</code>	discard list <i>n</i>
<code>clear.select all</code>	discard all of them

```
clear.select all
```

```
Cleared all numbered select lists
```

`clearselect` is the same verb, spelled without the dot.

Building a list

<code>select file {selection}</code>	the records that match
<code>sselect file {selection}</code>	the same, sorted
<code>show file {selection}</code>	choose records interactively
<code>qselect {dict} file {saving n}</code>	a list of field values rather than record ids
<code>nselect file</code>	keep only the ids of the active list that are not in <i>file</i>
<code>form.list {dict} file record</code>	take the list from a record's contents

`select` and `sselect` take the whole selection language of the query processor — `with`, the operators, `by` — so anything you can list you can select. See SD TCL - The Query Processor.

`qselect` **reads a field out of each record and makes a list of those values.** It is how you follow a reference from one file into another: select the orders, `qselect` the customer number, and the list you are left with is customers.

`qselect` **does not tell you which list it filled.** Its completion message ends after the words *select list* with no number: `14 record(s) selected to select list`. **The list is there** — it is list 0 unless you gave `to n`. `select` and `nselect` both print the number correctly, so do not read this as the list being missing.

Numbered lists

There are **eleven**, numbered **0 to 10**. List 0 is the default and the one every verb uses when you do not say otherwise. `to n` puts the result somewhere else, which is how you keep two lists at once:

```
select voc with dispatch = "OS" to 3
select voc with dispatch = "IN" to 4
```

```
2 record(s) selected to list 3
45 record(s) selected to list 4
```

A list in 1 to 10 does not affect the next command. Only list 0 does. That is what makes the numbered lists a safe scratch area, and it is why the prompt above stayed `:` rather than becoming `::`.

Saving a list by name

Numbered lists last as long as your session. A **saved** list is a record in the account's `$savedlists` file and outlives it.

<code>save.list {name} {from n}</code>	save list <i>n</i> as <i>name</i>
<code>get.list {name} {to n}</code>	load it back into list <i>n</i>
<code>delete.list name</code>	delete the saved list

`copy.list` *name* {`,` `new`} {**from** *file*} {**to** *file*}

copy one, or `*` for all

Both default to list 0, and `save.list` defaults the name to the list number.

```
select voc with dispatch = "OS"
save.list zzos
```

```
2 record(s) selected to list 0
2 records saved to select list 'zzos'
```

```
get.list zzos
delete.list zzos
```

```
2 record(s) selected to select list 0
Deleted saved select list 'zzos'
```

`copy.list` **moves a saved list between accounts**, because `from` and `to` name the `$savedlists` file to read and write. Within one account it renames or duplicates.

Combining two lists

There are two sets of verbs and they take different things. this is the easiest mistake on the page.

TAKES		
<code>list.union</code> · <code>list.inter</code> · <code>list.diff</code>	saved list names	reads and writes <code>\$savedlists</code>
<code>merge.list</code>	list numbers	works on the numbered lists in your session

Giving a number to `list.union` does not do the numbered thing — it looks for a saved list with that name and fails:

```
list.union 3 4 5
```

```
First source list does not exist
```

BY NAME

```
list.union zzos zzin zzboth  
list.inter zzos zzin zzover
```

```
47 records saved to select list 'zzboth'  
0 records saved to select list 'zzover'
```

Both write a **saved** list, not a numbered one, so `get.list` is how you pick the result up. The two answers above check each other: the sources hold 2 and 45 records with nothing in common, so the union is 47 and the intersection is 0.

BY NUMBER

```
merge.list 3 union 4 to 5  
merge.list 3 intersect 4 to 6
```

```
47 record(s) selected to select list 5  
0 record(s) selected to select list 6
```

The operator words are `union`, `intersect` (or `intersection`) and `diff` (or `difference`). `to n` names the target, defaulting to list 0, and `count.sup` suppresses the count line.

`list.diff` and `merge.list ... diff ...` give the records in the first list that are not in the second, so **the order of the two arguments matters** where it does not for union and intersection.

Who has these verbs

All of them are in a standard account. Building, saving and combining lists is reading, and nothing on this page changes a record.

See also

[SD TCL - The Query Processor](#) · [SD TCL - Files and Records](#) · [SD Basic - Select Lists](#).

SD TCL - Alternate Key Indexes

Indexing a field so records can be found by its contents, and the step that is easy to skip.

An alternate key index lets SD find records by what is in a field instead of by record id. Five verbs make, populate, inspect and remove them.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, against a six-record file whose field 1 held five `V` and one `K`.

Use `make.index`

That is the whole of this page's advice. `create.index` and `build.index` are two halves of what `make.index` does in one command, and **the half that is easy to miss is the second one:**

```
make.index zzak f2
```

```
Added index for F2
Building index 'F2'...
6 records processed
Populating index...
```

Use the two-step form only when you have a reason to separate them.

The five verbs

<code>create.index</code> <i>file field...</i>	define an index. Does not populate it
--	--

<code>build.index</code>	<code>file field... all</code>	populate it from the records already there
<code>make.index</code>	<code>file field...</code>	both, in one command
<code>list.index</code>	<code>file field...</code>	what indexes the file has, and whether they work
<code>delete.index</code>	<code>file field... all</code>	remove them

`create.index` and `make.index` also take:

<code>no.nulls</code>	do not index records whose field is empty
<code>pathname</code>	<code>path</code> put the index somewhere other than beside the file

Why the two-step form catches people

`create.index` records the definition and **walks nothing**:

```
create.index zzak f1
```

```
Added index for F1
```

That reports success, and the index it made matches no records at all. The proof is in `list.index`:

```
list.index zzak f1
```

```
Alternate key indices for file zzak
Number of indices = 1
Index name..... En Type Nulls S/M Fmt Field/Expression
F1                N  D   Yes   S   L   1
```

The **En** column is the one to read. **N** means the index is not enabled — it exists, it is listed, and it will not find anything. After `build.index` the same line reads **Y**:

```
F1          Y  D    Yes  S   L   1
```

So `list.index` is the check, and "the index is there" is not the same question as "the index works".

`build.index` needs the file to itself

```
build.index zzak f1
```

```
Cannot gain exclusive access to file
```

That is what you get if anything has the file open — including the command you just typed. `create.index` followed by `build.index` in the same session fails this way every time, because `create.index` leaves the file open. From a fresh session it succeeds:

```
Building index 'F1'...
6 records processed
Populating index...
```

`make.index` does not have this problem, which is the practical reason to prefer it.

A session that was killed rather than logged out also holds the file. Its entry stays in the user table, `listu` still lists it, and `build.index` is refused with the message above until it is cleared. If a build is refused and you are certain nothing is using the file, run `listu` before looking for anything more exotic.

Asking what a file has

```
list.index zzak f1 f2
```

```
Alternate key indices for file zzak
Number of indices = 2
Index name..... En Type Nulls S/M Fmt Field/Expression
```

F1	Y	D	Yes	S	L	1
F2	Y	D	Yes	S	L	2

COLUMN

En	enabled — Y only after a successful build
Type	D for a field, I for a computed expression
Nulls	whether empty values are indexed — no.nulls at creation is what changes it
S/M	single- or multi-valued
Fmt	the key's format code, as a dictionary entry carries it
Field/Expression	the field number, or the I-type expression

A file with none says so plainly:

```
File has no indices
```

Using an index

Nothing changes in how you write a query. An index makes a selection on that field faster; it does not add syntax and it does not change the answer:

```
count zzak with f1 = "V"
```

```
5 record(s) counted
```

That is the same command, and the same result, as before the index existed. **Index the fields you actually select on** — each index makes every write to the file a little more expensive, and one that is never used in a **with** clause costs and returns nothing.

Once built, an index maintains itself: ordinary writes and deletes keep it current with no further action. Reading an index from a program — **selectindex**, **indices()** — is in SD Basic - Alternate Key Indexes, which also has the detail on why **an already-open file variable never learns about a new index.**

Removing an index

```
delete.index zzak all
```

```
Deleted index F1  
Deleted index F2
```

`all` removes every index on the file. Naming them individually also works — with one catch:

`delete.index` **matches the index name exactly, and index names are held in upper case.**

`delete.index zzak f1` answers "*Unrecognised index name (f1)*" — on a file where `list.index zzak f1` had just found the index and printed it. `list.index` **folds case and** `delete.index` **does not**, so a name that lists perfectly well may still refuse to delete. Use `all`, or type the name exactly as `list.index` prints it.

Deleting the file removes its indexes with it.

Who has these verbs

All five are programmer verbs. A standard account has none of them, and does not need them — an index changes how a query runs, not how it is written, so a standard account gets the benefit of every index without being able to create or destroy one.

See also

SD Basic - Alternate Key Indexes · SD TCL - Files and Records · SD TCL - The Query Processor.

SD TCL - Programs and the Catalogue

Compiling SD BASIC, running it, and giving it a name you can type.

Getting from source to something you can type is three steps: **compile it, put it in a catalogue, then call it by name**. Only the first is compulsory.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, against a two-line program in an account's `bp` file.

Compiling

```
basic {file} record
```

The file defaults to `bp`. What comes back names the file and record, then the error count:

```
basic bp zzprog
```

```
Compiling bp zzprog
WARNING: Final END statement is missing
0 error(s)
Compiled 1 program(s) with no errors
```

A warning is not an error and the program still runs. The line above is a warning, the count says `0 error(s)`, and that program ran. **Read the count, not the presence of output.**

`@system.return.code` is the number of programs compiled, and negative on a fatal error.

The object code goes into `bp.out`, a separate file beside `bp`, one record per program. **Its name is lower case even when the source record is not** — a source record `ZZPROG` compiles to an object record `zzprog`. That is this port's naming, and it is worth knowing before you go looking for the object by the name you typed.

Running without cataloguing

```
run {file} record
```

```
run bp zzprog
```

```
ZZPROG OK
```

`run` needs the file and record name every time. It is what you use while you are still writing the thing.

The catalogue is what gives a program a name

```
catalog {file {call.name}} {program | *} {local | global | pcode} {no.xref}
```

Catalogue it and the name becomes a command:

```
catalog bp zzprog  
zzprog hello
```

```
ZZPROG added to private catalogue  
ZZPROG OK
```

The program sees the whole command line. `@sentence` in the program above read `zzprog hello`, so arguments arrive as typed and the program parses them itself.

THREE CATALOGUES, AND THEY ARE NOT THE SAME

private	the account's own. The default when no keyword is given
----------------	---

local	a VOC entry in the account, so the name works only there
--------------	--

global	<code>gcat</code> in the system account — every account sees it
---------------	--

Global cataloguing requires administrator privilege, and on this port that means an elevated session.

Without it you get "Command requires administrator privileges". **The same gate applies to an implicit global catalogue** — one chosen by a `*`, `!`, `_` or `$` prefix on the call name rather than by the `global` keyword — so the prefix is not a way round it.

Private and local cataloguing need none of this and work in a programmer's own account.

SEEING AND REMOVING

```
map {all} {lptr {n}} {file {name}}
```

```
System catalogue map at 23:12:07 on 26 Aug 2026                                Page 1
Catalogue name..... Compiled.  Time....  ...Obj  ..Xref  ..Size
ZZPROG                26 Aug 26   23:12:07    189      6    195
                                Total:      195
```

`Obj` and `Xref` are the object-code and cross-reference sizes in bytes, and `Size` is the total. An empty one says so:

```
Catalogue is empty
```

```
delete.catalog name... {global | local}
```

```
ZZPROG deleted from the private catalogue
```

`map` also takes a `file` option, which writes the map to a file instead of the screen. **It asks before overwriting** — "File xx will be cleared and overwritten. Continue (Y/N)?" — so it is not usable from a script driving SD down a pipe.

Formatting source

```
format {file} {record} {case}
```

`format` rewrites a BASIC source record with standard indentation. **It says nothing at all when it succeeds** — no output is the success case, which is worth knowing before you go looking for a confirmation that never comes. `case` additionally applies case conversion.

With no record name it uses the active select list.

Compiling dictionaries

```
cd {dict | data} file {i-type}... {no.query} {no.page}
cd local
cd all
```

`compile.dict` is the same verb under its longer name. It compiles the A, C, I and S-type entries in a dictionary; with no names it does all of them, or uses the active select list if there is one.

<code>cd local</code>	every dictionary in the account
-----------------------	---------------------------------

<code>cd all</code>	every dictionary visible from the account
---------------------	---

Compiling a dictionary is a performance step, not a correctness one: it is what a query using those entries runs against.

Named common blocks

```
list.common {all}
delete.common name
```

```
No named common blocks present
```

Named common survives between programs in a session, and `list.common` is how you see what is holding memory. `all` additionally shows the system blocks and is internal-only.

The two that are not like the others

`generate` builds `$include` token definitions from a dictionary. **It is interactive** — it asks for the token type, the prefix and several other things — so it cannot be driven from a script.

`debug` runs a program under the debugger, taking the same arguments as `run`. It needs a terminal and **refuses inside a phantom** with *"Cannot debug program in a phantom process"*. The debugger itself is covered in SD Basic - Debugging.

Who has these verbs

standard

`format` `list.common`

programmer

`basic` `catalog` `catalogue` `cd` `compile.dict` `debug` `delete.catalog`
`delete.catalogue` `delete.common` `generate` `map` `run`

A standard account can run catalogued programs and cannot make them. That is the whole point of the split: an application is deployed by cataloguing it, and the people who use the application need none of the verbs on this page to run it.

`format` and `list.common` are the two exceptions, and both are harmless — one tidies source the account can already read, the other reports on the session's own memory.

See also

SD Basic - Program Structure · SD Basic - Debugging · SD TCL - Files and Records.

SD TCL - The ed Line Editor

The editor that runs inside SD, needs no terminal, and can be driven from a script.

SD has three editors and they are all installed with it. `ed` **is the one that runs *inside* SD**: it is written in SD BASIC, draws nothing, and needs neither a terminal nor operating-system permission. That is what makes it the editor for a phantom, an API session, a session driven down a pipe, and anything you want to automate.

The other two are edit and micro, which hand the record to a full-screen Windows program. Use one of those when you are sitting at a terminal and want to see the whole record at once.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part. `ed`'s **own commands are shown in upper case**, which is how `ed`'s `HELP` prints them; it folds case too.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, against a scratch record in an account's `bp` file.

Starting it

```
ed {dict} file {record...} {no.query}
```

Naming the record opens it straight away:

```
ed bp zzed
```

```
bp zzed
New record
----:
```

`----` is the prompt. A record that exists reports its length instead:

```
bp zzed
```

```
3 line(s)
```

Omit the record name and `ed` asks for one, and asks again after each edit until you give it an empty answer. **Give several names and it works through them in order** — the record after the current one is reached by filing or quitting the one you are on:

```
ed bp zzed zzedb
```

```
bp zzed
3 line(s)
----: Q
bp zzedb
New record
----:
```

A file name is tried as typed, then lower case, then upper. A file that matches none of the three stops the verb before the editor starts:

```
ed nosuchfile zzed
```

```
File not found
```

`dict` edits the file's dictionary rather than its data. It is the same addressing as everywhere else, and a dictionary with no such entry is a new record like any other:

```
ed dict voc who.am.i
```

```
DICT voc who.am.i
New record
```

The commands

MAKING A RECORD FROM NOTHING

`I` inserts one line after the current one. At the top — where a new record starts — it inserts at line 1:

```

ed bp zzed
I * ZZED - a scratch record for the SD TCL editing page
I crt 'ZZED OK'
I end
T
P20
FI

```

```

bp zzed
New record
----: I * ZZED - a scratch record for the SD TCL editing page
Bottom at line 1
----: I crt 'ZZED OK'
Bottom at line 2
----: I end
Bottom at line 3
----: T
Top
----: P20
0001: * ZZED - a scratch record for the SD TCL editing page
0002: crt 'ZZED OK'
0003: end
Bottom at line 3
----: FI
'zzed' filed in bp

```

I **with no text** starts a multi-line insert instead, prompting `0001=` for each line until you enter an empty one.
IB is the same as **I** but inserts before the current line rather than after it.

MOVING ABOUT

T to the top — *before* line 1, so **I** inserts at 1

B to the last line

n to line *n*

+ n, **- n** *n* lines forward or back

G n, **PO n** to line *n*, the long ways round

G< , G>	to the first or last line of the block
-----------------------------	--

LOOKING

P n^*	display n lines from the current line
----------------	---

PL { - } n	n lines relative to the current line
----------------------------	--

PP n^*	n lines centred on the current line
-----------------	---------------------------------------

PB	the block
-----------	-----------

L <i>string</i>	the next line containing <i>string</i>
------------------------	---

F { <i>col</i> } <i>string</i>	the next line with <i>string</i> at column <i>col</i> , default 1
---------------------------------------	--

M <i>pattern</i>	the next line matching a pattern
-------------------------	----------------------------------

COL	a column ruler
------------	----------------

?	where you are and what is set
----------	-------------------------------

P leaves the current line at the last line it printed, and that is what the next command acts on. It is the one thing on this page that catches people, because "display" does not sound like a move. **P20** on a three-line record, then **DE**, and it is **line 3** that goes:

```
----: P20
0001: * ZZED - a temporary record for the SD TCL editing page
0002: crt 'ZZED OK'
0003: end
Bottom at line 3
----: DE
Bottom at line 2
```

? is worth knowing before you ask anyone else:

```
----: ?
File name           = bp
Record name         = zzed
Current line        = 3
No block defined
```

```
OOPS will undo 'I end'
Searches are case-sensitive
Expansion of non-printing characters : Disabled
Non-printing character entry       : Disabled
Verification of block actions      : Enabled
```

Searches are case-sensitive until you say otherwise — `CASE OFF` makes `L` and `F` insensitive, `CASE ON` puts it back.

CHANGING

<code>I {text}, IB {text}</code>	insert after, insert before
<code>R text</code>	replace the current line
<code>A {text}</code>	append to the end of the current line
<code>C/ old / new / {n}{ G }{ B **}</code>	change <i>old</i> to <i>new</i> in <i>n</i> lines. <code>G</code> every occurrence, <code>B</code> over the block
<code>R/ old / new** / ...</code>	the same, without moving
<code>D n, DE n</code>	delete <i>n</i> lines from the current one
<code>DUP {n}</code>	duplicate the current line
<code>CAT {string}</code>	join the next line onto this one
<code>FORMAT</code>	re-indent BASIC source

The delimiter of a `C` command is whatever character follows the `C`, so a change involving `/` can use `,` or `:` instead.

`OOPS` UNDOES THE LAST THING THAT CHANGED THE RECORD

One step, and it names what it is undoing:

```
----: OOPS
Undoing 'DE'
```

`?` tells you in advance what it would undo, which is the line above reading `OOPS will undo 'I end'`.

BLOCKS

< sets the start of a block at the current line, **>** the end, **<>** makes the current line a one-line block. Then **COPY** copies it after the current position, **MOVE** copies and deletes the original, **DROP** deletes it, and **PB** displays it. **<** at the top clears the block.

VALUES ON A LINE

A field is a line, so **ed** handles fields on its own. **A value mark and a subvalue mark are single unprintable characters**, and a line holding them displays as one run with a stray glyph in it. **EV** opens the current line as if its values were lines:

```
----: EV
Editing values. Use SV to save changes or QV to discard changes
----: P20
0001: NAME
0002: Smith
0003: Jones
Bottom at line 3
----: QV
EV mode changes discarded
Returned from EV mode
```

SV leaves **EV** keeping the changes, **QV** discards them. Three levels are available: values, subvalues, and the elements of a compound I-type.

A mark is typed as **^ and its character code:** **^253** for a value mark, **^252** for a subvalue mark, **^^** for a literal **^**. The line above was made with **I NAME^253Smith^253Jones**.

LEAVING

FI	write the record and end the edit
FIB	file it, then basic it
FIBR	file it, compile it, and run it
SAVE	write it and stay in the editor
Q , QUIT	end the edit, discarding changes
X	abandon the whole run when a select list is in use
FD , DELETE	delete the record from the file and end the edit

```
----: FI
'zzed' filed in bp
```

Q asks only **If you changed something**, and it asks on the same line:

```
----: Q
Record changed - OK to quit? Y
```

Anything but **Y** and you are still in the editor. On an unchanged record **Q** says nothing at all and returns to TCL.

SAVE and **FI** with a different file or record name ask before they overwrite, and so do **FD** and **DELETE**. Written back to their own record they ask nothing at all, which is what lets **ed** be driven from a script.

XEQ RUNS A TCL COMMAND WITHOUT LEAVING THE EDITOR

@FILE, **@ID** and **@LINE** are substituted, so a command can be about whatever you are editing:

```
----: XEQ ct @FILE @ID
bp zzed
1: * ZZED - a scratch record for the SD TCL editing page
2: crt 'ZZED OK'
3: end
```

And what it showed there was the filed record, not the one on the screen. The buffer at that moment read "*a temporary record*" — the change had not been filed. **XEQ** runs outside the editor and sees the file, which is the answer you want from **basic** and the wrong one from **ct**.

HELP

HELP alone prints every command. **HELP** with a topic prints one:

```
----: HELP FI
File [[filename] record.id]
      Write record and end edit
```

THE COMMAND STACK, AND PRESTORED SEQUENCES

Every command you type is kept. **.L** lists the stack, **.R***n* recalls line *n* to the top, **.X***n* executes it again, and **.A**, **.C**, **.I** and **.D** append to, change, insert and delete stack lines.

`.X` **with a file and record instead of a number** runs a *prestored edit sequence* — a record holding one `ed` command per line:

```
ed bp zzseq
I ED
I L scratch
I C/scratch/altered/
I FI
FI
```

```
ed bp zzed
.X bp zzseq
```

```
bp zzed
3 line(s)
----: .X bp zzseq
0001: * ZZED - a scratch record
0001: * ZZED - a altered record
'zzed' filed in bp
```

Field 1 is a marker, not a command. it must begin with `E` — `ED` is the usual thing to write — and `ed` refuses a record that does not with "*Record is not a prestored edit sequence*". Nothing was run for it above, and the line numbers in error messages count it: an unrecognised command in **field 2** is reported as line 2.

`PAUSE` inside a sequence stops it and `.XR` resumes, `.XK` abandons it; `LOOP` repeats from a given line.

A prestored sequence is not an unattended mechanism. One bad command **abandons the rest of the sequence and then asks a question**, and the answer is read from wherever the session's input comes from.

```
----: .X bp zzs2 Error - Unrecognised command at line 2 of prestored commands Did you mean to
insert this text (Y/N)? N
```

The same is true of a multi-line `I`, which is refused outright inside a sequence. **If you are driving `ed` down a pipe, give every command its arguments** — `I` with text, `FI` with no name, never `FD` — and it runs through without stopping.

Who has it

`ed` **is a programmer verb.** A standard account does not have it, and does not have `edit` or `micro` either — an account that runs an application does not edit the records behind it.

See also

SD TCL - The edit Screen Editor · SD TCL - The micro Screen Editor · SD TCL - Programs and the Catalogue.

SD TCL - The edit Screen Editor

Microsoft Edit, a menu bar and a dozen keys — the one to reach for when you just want to change a record.

```
edit {dict} file record
```

`edit` opens a record in **Microsoft Edit**, a small full-screen editor that ships with Windows. It has a menu bar, the shortcuts everyone already knows, and nothing else to learn. **That is the point of it:** if you want to fix a line in a record and get on with your day, this is the one.

It has no syntax highlighting and no command language. For SD BASIC source — or for anything where you want highlighting, split windows, or a command bar — use `micro` instead. For a session with no terminal, or one you are driving from a script, use `ed`.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The keys below are Microsoft Edit's own, for the version SD checks for — **1.2.1**, which is what ships in current Windows. SD installs that editor and calls it; it does not implement it, so where a binding differs the editor is right.

Both editors are installed with SD

You do not install anything. SD's installer checks for both editors and installs whichever is missing, machine-wide, so every account SD creates can reach them. Microsoft Edit is usually already there — it is part of current Windows, at `C:\Windows\System32\edit.exe`.

If that could not happen — an offline machine, or one whose policy blocks the package manager — the verb says so and names the command that installs it, rather than opening nothing and reporting the record unchanged.

The keys

`Ctrl-S` saves and `Ctrl-Q` quits. Those two are most of what anyone needs.

Ctrl-S	save
Ctrl-Q	exit
Ctrl-W	close the file
Ctrl-O · Ctrl-N	open a file · new file
Ctrl-Z · Ctrl-Y	undo · redo
Ctrl-X · Ctrl-C · Ctrl-V	cut · copy · paste
Ctrl-A	select all
Ctrl-F · Ctrl-R	find · replace
Ctrl-G	go to line:column
Ctrl-P	go to file
Alt-Z	toggle word wrap

The menu bar is the help. There is no help screen — the Help menu holds only *About* — and there does not need to be one: **F10** or **Alt** and the menu's letter opens a menu, and every command is listed there **with its shortcut printed beside it**.

Alt-F	File
Alt-E	Edit
Alt-V	View
Alt-H	Help

Save As is on the File menu and is the one common command with no shortcut of its own: **Alt-F** then **A**.

What SD does around the editor

The two screen editors are **one SD program with two names**, so everything in this section is equally true of micro.

the working copy	the record is copied into <code>\$hold</code> as <code>record .editing</code> , and the editor is run on that. It is removed on every exit, including the ones that fail
-------------------------	--

saving	"Save? <Y>es, <N>o", then for a <code>bp</code> record "Compile?" and "Catalogue?"
---------------	--

a <code>dict</code> record	is always saved and re-compiled with <code>cd</code>
----------------------------	--

finishing	"<E>xit or <R>e-edit", so a compile error can be fixed without starting again
------------------	---

A compiled dictionary record is truncated to its first 15 fields while you edit it, which is what you want: the fields after them are the compiled form, and `cd` rebuilds them when you save.

Marks, and how to type one

A **field** mark is a line break, so a text editor handles fields on its own. The other three marks are single control characters an editor would either draw as a stray glyph or drop, so each has a token you type instead.

Every token is `~` and one more character, and `~` is the only escape character.

<code>~~</code>	a value mark
-----------------	--------------

<code>~`</code>	a subvalue mark
-----------------	-----------------

<code>~!</code>	a text mark
-----------------	-------------

<code>~-</code>	a literal <code>~</code> , where one would otherwise be misread
-----------------	---

<code>~,</code>	a literal <code>,</code> , where one would otherwise read as a separator
-----------------	--

`SMITH~~JONES~~BROWN` is a three-value field; `RED~`BLUE~~GREEN` is two values, the first with two subvalues.

MARKS IN A ROW ARE SEPARATED BY A COMMA

Written token against token a run of marks cannot be read, so SD puts a comma between them. A text mark, a text mark and a value mark, one after another, is:

```
~!,~!,~~
```

Type the comma yourself when you enter marks in a row. It is a separator and not data — which is why a literal comma standing in exactly that position, between two marks, is written `~,`.

THE CONVERSION IS LOSSLESS

No record is refused and none is mangled, whatever it contains. A tilde is written `~-` **only where the character after it would make the pair look like a token** — another `~`, a backtick, a `!`, a `-`, a `,`, or a mark. Everywhere else a tilde is left exactly as you wrote it, so `a~b` is still `a~b` and ordinary source reads normally.

Two gates, and both are separate from the verb

the VOC tier	decides whether you have the verb at all
---------------------	--

<code>os.users</code> field 2	the <code>OS.EXECUTE</code> field, decides whether it may run
--------------------------------------	---

An editor runs outside SD, so it needs operating-system permission that `ed` does not. It comes from a record in the system file `os.users` whose field 2 reads `yes`, **and only an administrator can put one there**.

An administrator gets one without asking. An account created with the **ADMINISTRATOR** tier — which is the tier of the account SD's installer makes for whoever installs it — is written into `os.users` as it is created, with both fields `yes`. So an administrator reaches the operating system **without elevating**, and these two verbs work in an ordinary session.

For an administrator it is a rule, and it cannot be turned off. For every other tier it is a grant, and there are keywords for it:

<code>create.account user</code> <i>name ...</i> <code>os-on</code>	give the new account <code>OS.EXECUTE</code> — and these two verbs
---	--

<code>create.account user</code> <i>name ...</i> <code>sh-on</code>	give it the <code>sh</code> verb
---	----------------------------------

<code>modify.account</code> <i>name</i> <code>os-on</code> <code>os-off</code>	change it afterwards
--	----------------------

<code>modify.account</code> <i>name</i> <code>sh-on</code> <code>sh-off</code>	the same for the <code>sh</code> verb
--	---------------------------------------

They are four switches over two fields rather than four names for one state, so `sh-off` leaves `OS.EXECUTE` alone. `modify.account` **needs an elevated session**, as it always has — you elevate to grant somebody the right not to have to.

The four refuse an administrator, in both directions. An administrator has all three routes — `ssh`, the API and the operating system — as a rule, and none of them is `modify.account`'s to change:

```
:modify.account don os-off
don is an administrator and always reaches the operating system
```

It is the same refusal `ssh`, `api`, `both` and `none` already give for an administrator, and it is deliberate rather than an oversight: the whole purpose of the tier is that it grants unlimited access.

The record is ordinary data, so an administrator can also edit it by hand with `ed os.users` *name* from `sdsys`. An account of any other tier starts with no record and is refused until somebody grants it.

An elevated session passes anyway, whatever the file says — otherwise an empty `os.users` would lock the machine's own administrator out.

And a session with no terminal is refused before anything is written — an API session, or a script driving SD down a pipe:

```
:edit bp zzed
edit needs a terminal to draw on, and this session has none.
ed, the line editor, works anywhere.
```

Both usage errors name the verb you typed rather than the program behind it:

```
:edit
No file name specified.  Usage: edit {dict} <file> <record>
:edit bp
No record name specified.  Usage: edit {dict} <file> <record>
```

Who has these verbs

`edit`, `micro` and `ed` are all **programmer verbs**. A standard account has none of them.

See also

SD TCL - The micro Screen Editor · SD TCL - The ed Line Editor · SD TCL - Files and Records.

SD TCL - The micro Screen Editor

The capable one — SD BASIC highlighting, a command bar, split windows and a help screen.

```
micro {dict} file record
```

`micro` opens a record in **micro**, a full-screen editor that does rather more than edit does. Two things make it the one to use for programming:

it highlights SD BASIC

statements, reserved words, intrinsics, strings and comments, from rules SD generates out of the compiler's own tables and ships

it has a command bar and a help screen

so everything it can do is reachable and readable without leaving the editor

It also has multiple files open at once, split windows, a plugin system and rebindable keys. **If you only want to change one line in a record**, `edit` is less to think about.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The keys below are micro's own, read from the default bindings and the help text inside the executable SD installs — **micro 2.0.15**. The SD half of the page describes SD Core for Windows W1.0-0.

Both editors are installed with SD

You do not install anything. SD's installer checks for both editors and installs whichever is missing, machine-wide, so every account SD creates can reach them.

If that could not happen — an offline machine, or one whose policy blocks the package manager — the verb says so and names the command that installs it, rather than opening nothing and reporting the record unchanged.

The keys

Ctrl-S saves, **Ctrl-Q** quits, and **Ctrl-G** **opens the help screen**, which is the answer to everything not on this page.

Ctrl-S	save
Ctrl-Q	close the file — and quit, if it is the last one open
Ctrl-G	open and close help
Ctrl-E	the command bar — see below
Alt-G	show and hide the key hints
Ctrl-Z · Ctrl-Y	undo · redo
Ctrl-X · Ctrl-C · Ctrl-V	cut · copy · paste
Ctrl-K · Ctrl-D	cut the current line · duplicate it
Ctrl-A	select all
Ctrl-F	find
Ctrl-N · Ctrl-P	find next · find previous
Ctrl-O	open a file
Ctrl-T	open a new tab
Alt-A · Alt-E	start · end of line
Alt-B · Alt-F	previous · next word

Function keys work too, for anyone who prefers them: **F1** help, **F2** save, **F3** or **F7** find, **F4** or **F10** quit.

Ctrl-Q **closes the file rather than the editor**. With one record open — which is how SD starts it — that is the same thing. With a second file opened by hand it is not, and micro stays up until the last one is closed.

The command bar

Ctrl-E opens a `>` prompt at the foot of the screen. It takes micro's own commands, and the two worth knowing on the first day are:

<code>> help</code>	the same screen as <code>Ctrl-G</code>
<code>> help keybindings</code>	every key, including the ones not listed above
<code>> set option value</code>	change a setting for this session

Saving is broken for an ordinary account in W1.0-0

`micro` **draws, edits and highlights correctly and then cannot save**, unless the session is elevated:

```
Permission denied. Save with sudo not supported on Windows
```

The record you were editing is not touched and nothing is lost — the failure is on micro's side of the working copy, and quitting without saving leaves the record exactly as it was.

It is not a permission problem with your file or your account. SD points micro's configuration directory at a folder under `C:\Program Files`, which micro has to write to and an ordinary account may not. It is recorded as a release blocker in the project's fix lists.

Until it is fixed, use `ed` for anything you intend to save. It runs inside SD, needs no external editor and no operating-system access, and is documented on SD TCL - The ed Line Editor. `micro` is still useful for reading a record with syntax highlighting.

Highlighting SD BASIC

SD SHIPS THE RULES AND POINTS micro AT THEM. They are generated from the BASIC compiler's own tables rather than typed by hand, so the list of statements, reserved words and intrinsics is the one the compiler actually accepts.

It works on a record from a `bp` file and not on other records, and the reason is worth knowing: micro decides which language a file is by matching its *name*, and a record called `MY.REPORT` tells it nothing. So SD names the working copy `*record*.editing.sdbasic`, and that suffix is what micro recognises. A record edited out of any other file is plain text.

What SD does around the editor

The two screen editors are **one SD program with two names**. The working copy, the save and compile questions, the two permission gates and the refusals are the same for both, and they are set out on SD TCL - The edit Screen Editor.

In short: the record is copied into `$hold`, `micro` is run on the copy, and on the way out SD asks whether to save it and — for a `bp` record — whether to compile and catalogue it. Both verbs need `OS.EXECUTE` permission and a real terminal, and refuse a session driven down a pipe:

```
:micro bp zzed
micro needs a terminal to draw on, and this session has none.
ed, the line editor, works anywhere.
```

Marks, and how to type one

A **field** mark is a line break, so `micro` handles fields on its own. The other three have a token you type instead. **Every token is `~` and one more character, and `~` is the only escape character.**

<code>~~</code>	a value mark
<code>~`</code>	a subvalue mark
<code>~!</code>	a text mark
<code>~-</code>	a literal <code>~</code> , where one would otherwise be misread
<code>~,</code>	a literal <code>,</code> , where one would otherwise read as a separator

Marks in a row are separated by a comma — a text mark, a text mark and a value mark is `~!,~!,~~`. The conversion is lossless: no record is refused and none is mangled. The full rules are on SD TCL - The edit Screen Editor.

Who has these verbs

`micro`, `edit` and `ed` are all **programmer verbs**. A standard account has none of them.

See also

SD TCL - The edit Screen Editor · SD TCL - The ed Line Editor · SD TCL - Programs and the Catalogue.

SD TCL - Printing and Spooling

Print units at the command prompt, keeping a job open, and capturing a session instead of printing it.

Printing is set up at the prompt and used from a program. **This page is the prompt half:** the verbs that point a print unit somewhere, hold it open, and capture what a session displays. The statements a program uses — `print on`, `heading`, `printer file` — are in SD Basic - Printing, and nothing here repeats them.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** something you type as it stands.

Print units, and where they point

Unit 0 is the terminal. Every other unit is somewhere you have sent it. `setptr` is how you look and how you set:

```
:setptr display
Unit Width Depth Tmgn Bmgn Mode Options
    0    80    66    0    0    1
```

COLUMN

Unit	the print unit number a program names in <code>print on n</code>
Width, Depth	the page, in characters and lines
Tmgn, Bmgn	top and bottom margins, in lines
Mode	what the unit does with the output
Options	everything else that has been set on the unit

The depth is 66 and the terminal is not 66 lines. Unit 0's page is a printed page, not your screen — it is the size a report is paginated to when it goes to a printer. The screen's own size is `term`, which is a different setting on a different page: SD TCL - The Terminal and the Session.

Setting a unit

```
setptr unit, width, depth, top.margin, bottom.margin, mode {, options}
setptr default, width, depth, top.margin, bottom.margin, mode {, options}
setptr display
setptr unit, display
```

The positional arguments are all optional — `setptr 1,,,,,3` sets only the mode of unit 1 — and `nodefault` keeps the values you did not name rather than resetting them.

`setptr` **asks before it changes a unit, and `brief` is how you stop it.** That confirmation is a prompt, so a `setptr` inside a phantom, a script or an API session **hangs waiting for an answer nobody is there to give.** Give it `brief` in anything that is not a person at a keyboard.

The options are many and most are printer-specific. The ones worth knowing at the prompt:

<code>as</code> <i>name</i> , <code>as pathname</code> <i>path</i>	the name or path a mode 3 unit writes to
<code>at</code> <i>printer</i>	the Windows printer to send to
<code>brief</code>	suppress the confirmation prompt
<code>keep.open</code>	hold the unit open between jobs — see below
<code>copies</code> <i>n</i> , <code>landscape</code> , <code>portrait</code> , <code>duplex</code>	passed to the Windows spooler
<code>newline</code> <code>cr</code> <code>lf</code> <code>crlf</code>	the line ending written
<code>nfmt</code>	send the data with no formatting at all
<code>style</code> <i>name</i>	apply a report style

`paper.size`, `cpi`, `pcl`, `symbol.set`, `weight` and `overlay` are the printer-formatting set and behave as they always did.

The same settings in keywords: `printer`

```
printer {unit} query           report one unit
printer {unit} at printer.name send it to a Windows printer
printer {unit} file file.name record send it to a record
printer {unit} width n | lines n the page
printer {unit} top.margin n | bottom.margin n | left.margin n
printer {unit} keep.open | close
printer {unit} reset           back to the defaults
```

`printer` **does the same job as** `setptr` **and does not ask first.** That is the reason to know it exists: `setptr` confirms before it changes a unit unless you remember `brief`, and `printer` has no confirmation at all. **It is the form to use from a phantom, a script or an API session.**

More than one keyword may be given, in any order. The unit number comes before the keywords and **defaults to 0**, the terminal.

`query` **is the per-unit report**, and it names things `setptr display`'s columns do not:

```
:printer query
PRINT UNIT 0
  Page width      : 80
  Lines per page  : 66
  Top margin      : 0
  Bottom margin   : 0
  Left margin     : 0
  Mode            : 1
  Printer         : (Default)
```

```
:printer 1 query
PRINT UNIT 1
  Page width      : 80
  Lines per page  : 66
  Top margin      : 0
  Bottom margin   : 0
  Left margin     : 0
  Mode            : 3
  Target          : $hold P1
```

The last line names the destination, and its wording tells you whether one has been set:

Printer : (Default)	mode 1, no printer named — the Windows default
Printer : <i>name</i>	mode 1, that Windows printer
Target : \$hold P <i>n</i>	mode 3, nothing named — the unit's own hold record
Pathname : <i>path</i>	mode 3, that destination

`setptr display` shows none of this; it is a table of numbers.

Setting and putting back in one session — the two `query` reports are cut to the lines that moved:

```
:printer 1 width 132 lines 60
:printer 1 query
  Page width      : 132
  Lines per page  : 60
:printer 1 reset
:printer 1 query
  Page width      : 80
  Lines per page  : 66
```

Nothing is printed on the way through — the settings are the report, and `query` is how you see them. `reset` puts width, depth and all three margins back to their defaults **and clears the destination as well**: the printer name on unit 0, the file and record on any other unit. (*Unit 1 above had no destination set either side of the reset, so its last line reads `Target` both times — that is the wording for "nothing named", not evidence that `reset` left it alone.*)

<i>Invalid print unit number</i>	<code>printer</code> with nothing after it, or a unit outside the range
<i>Unexpected token (%1)</i>	a keyword it does not know
<i>Printer name not recognised</i>	<code>at</code> with a name Windows does not have

Holding a unit open

```
sp.open
sp.close
```


By default a unit finishes its job and releases when the program closes it, so two reports become two print jobs. `sp.open` sets the keep-open flag so they become one; `sp.close` clears it again. The flag shows up in `setptr`'s Options column, which is the way to check it took:

```
:setptr display
  0   80   66   0   0   1
:sp.open
:setptr display
  0   80   66   0   0   1 KEEP.OPEN
:sp.close
:setptr display
  0   80   66   0   0   1
```

Neither verb prints anything. The Options column is the only report you get, and it is worth looking at — a keep-open flag left set is why a report occasionally arrives stapled to the one before it.

Printing records

```
spool file.name record.id... {lines n m} {lnum} {lptr n}
```

With no ids it prints select list 0, so the usual shape is a `select` followed by a `spool`. With no file name at all it refuses rather than guessing:

```
:spool
File name required
```

<code>lines</code> <i>n m</i>	print only lines <i>n</i> to <i>m</i> of each record
<code>lnum</code>	number the lines
<code>lptr</code> <i>n</i>	send to print unit <i>n</i> rather than the default

Looking at what is queued

`sp.view` shows the records in `$hold` and offers to print them.

It is a full-screen form and cannot be driven down a pipe. It asks for a printer number, then whether to print, then a confirmation — three prompts, so a script that runs it stops and waits. Like the screen editors, it wants a person at a terminal. Everything on this page except `sp.view` can be scripted.

`$hold` is an ordinary directory file in the account, so a script that needs to work with held reports can read it with `select`, `list` and `spool` instead.

Capturing a session instead of printing it

```
como on record.name
como off
```

`como` **writes everything the session displays into a record** in the `$como` file, which it creates in the account the first time you use it. End to end:

```
:como on zzcomo
COMO file activated to zzcomo
:who
11 DON
:date
Thursday, 27 August 2026 01:13pm
:como off
COMO file deactivated
```

and the record afterwards:

```
:ct $como zzcomo
$como zzcomo
1: COMO file activated to zzcomo
2:
3: 11 DON
4:
5: Thursday, 27 August 2026 01:13pm
6:
```

It captures the prompts and the echoes, not just the output. The record is a transcript of the session, which is what makes it useful for showing somebody what you did — and it means a COMO of a long run is larger than the output alone. `como off` **is captured too**, as the last thing before the file closes.

To capture one command rather than a whole session, `execute ... capturing` in a program is the better tool — see SD Basic - Program Control.

Report styles

```
report.style
report.style name
report.style off
```

A report style is a stored set of print settings applied by name. With no argument the verb says what the default is:

```
:report.style
No default report style has been set
```

which is what a fresh account reports. `off` clears it again.

Printing on this port

A named printer is a Windows printer, and printing goes to the Windows spooler rather than to a POSIX print command. There is no `lp` and no PostScript pipeline. The consequences for a report that assumed either are set out in SD Basic - Printing, and they apply the same way to a unit set up with `setptr`.

See also

SD Basic - Printing · SD TCL - The Terminal and the Session · SD TCL - The Query Processor.

SD TCL - The Terminal and the Session

Screen size, terminal behaviour, silence, the inactivity timer, and writing to the error log.

Two different things are called the terminal. **term** is the page — how wide and how deep SD thinks your screen is, which decides where output wraps and when it pauses. **pterm** is the wire — how bytes are treated on the way in and out. They are set separately and are almost never confused once you have seen both.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** something you type as it stands.

The page: **term**

<code>term</code>	display the settings
<code>term {width} {, lines} {type}</code>	set them
<code>term colour {bgc}{,fgc}</code>	set display colour
<code>term default</code>	back to the defaults
<code>term display</code>	show the terminfo bindings

```
:term
Page width: 200
Page depth: 9999
Device      : windows
```

Depth is what decides paging. A listing longer than the page pauses; set the depth very large and it does not. That is why scripts driving SD open with something like `term 200,9999` — a script has nobody to press a key at a `Press any key` prompt, and a paged listing down a pipe stops for ever.

Width sets only the width, and the two are independent:

```
:term 132
:term
Page width: 132
Page depth: 9999
```

The depth stayed at 9999 because only the width was given.

SD'S DEFAULT PAGE IS 120 × 36

Not 80 × 24. SD's default terminal size is **120 columns by 36 lines**, and it is not a cosmetic choice: the shipped `@` dictionary records and the default `list` report layouts are formatted for that width. A report that looks wrapped or truncated on a narrow window is usually the window, not the report.

Where the number comes from, in order, when a session starts:

-
- 1 the `LINES` and `COLUMNS` environment variables, if they are numeric
 - 2 otherwise the terminfo entry's `lines` and `cols`
 - 3 **otherwise 36 and 120**
-

and whatever comes out is then raised to the minimum of 10 lines by 20 columns if it is smaller. So an ssh session or a console window normally gets its real size, and **120 × 36 is what SD falls back to when nothing else answers** — a phantom, a piped script, a service.

`term default` **puts that pair back, and it prints nothing when it does.** The verb sets the same 120 × 36 the login path falls back to, then returns without a message — so the way to see that it worked is a bare `term` after it, not the command itself:

```
:term default :term Page width: 120 Page depth: 36
```

`term 120,36` does the same thing by hand and is worth knowing for the cases where you want some other size back.

`Device` is the terminfo entry in use, `windows` on this port. If a terminal type has no terminfo entry SD falls back to `windows` rather than failing, so this line is worth reading when screen handling misbehaves.

The wire: `pterm`

```
:pterm display
Break trapping: On (char ^C)
Case inversion: Off
Input newline: CR
Output newline: CRLF
Binary mode (client to server): Off
```

```
Binary mode (server to client): Off
Telnet negotiation: Off
```

`pterm` **with no keyword prints nothing.** It is not a display verb that has gone quiet — it takes a keyword and `display` is one of them. The full set:

<code>pterm display</code>	the settings above
<code>pterm lptr</code>	send them to the default printer
<code>pterm break on</code> <code>off</code> <code>c</code>	enable, disable, or set the break character
<code>pterm case invert</code> <code>noinvert</code>	case inversion on input
<code>pterm newline cr</code> <code>lf</code> <code>crlf</code>	the output line ending
<code>pterm return cr</code> <code>lf</code>	the input line ending
<code>pterm binary on</code> <code>off</code>	pass bytes through untranslated
<code>pterm telnet on</code> <code>off</code>	recognise <code>TN_IAC</code>
<code>pterm reset</code> <i>string</i>	the terminal reset string
<code>pterm prompt</code> <code>":"</code> <code>:::"</code>	change the command prompt and the continuation prompt

Input `CR`, output `CRLF` is the shipped setting and it matches what SD writes to everything else externally readable on this port.

`pterm break off` **disables the break key for the session.** It is the right thing before a job that must not be interrupted and the wrong thing to leave set, because it is also how you stop a runaway program.

Silence: `hush`

```
hush on
hush off
hush expression
```

`hush on` **discards output entirely** — it does not redirect it, and nothing is kept. With no argument it toggles.

It suppresses the command echo as well as the output, which is more thorough than people expect. Below, `who` is typed **twice** — once inside the silence and once after it — and the transcript contains one:

```
:hush on
:who
14 DON
:term 132
```

The first `who`, and `hush off` itself, produced no line at all. **If you are debugging a script that has gone quiet, an unmatched `hush on` looks exactly like a hang.**

The bell

```
bell on
bell off
```

Sets or clears the character SD sends to sound the terminal bell. **Neither form prints anything**, and `bell` with no keyword is an error rather than a toggle: *Mode keyword missing or invalid*.

Clearing type-ahead and stored prompts

```
clearinput      clear.input
clearprompts    clear.prompts
```

`clearinput` discards anything typed ahead but not yet read. `clearprompts` discards the inline prompt and response text a previous command left behind. Neither prints anything.

Each has two spellings and they are the same verb. `clear.input` and `clear.prompts` are separate VOC records pointing at the same internal verb number as the run-together forms, which is the pattern described on SD TCL - The Command Processor. Use whichever reads better; nothing distinguishes them.

Neither can be demonstrated down a pipe, and `clearinput` should not be sent down one at all. In a piped session the input stream *is* the script, so discarding unread input discards the commands that have not run yet — including the `off` that would have ended the session. **A run that did this hung, was killed, and left a session in the user table that only an elevated `sd -cleanup` could clear.** Both verbs are for a person at a terminal.

The inactivity timer

```
autologout
autologout n
```

```
:autologout
Autologout is disabled
```

which is the shipped state. Setting a timer ends a session that has been idle too long — useful on a machine where people walk away from a prompt, and worth thinking about before enabling on a machine where a long-running interactive job is normal.

The command echo: `echo`

```
echo on
echo off
echo
```

`echo off` **stops SD echoing what you type**; with no keyword it toggles. Neither form prints anything, and `echo off` suppresses its own confirmation along with everything else.

```
:echo off
:28 DON
:who
28 DON
```

That is one `who` under `echo off` and one after `echo on`. **The first has no `:who` line above it** — its output ran straight on after the prompt — and `echo on` produced no line of its own either, because the echo it restores does not apply to the command that restores it.

It is not `hush`, and confusing the two wastes time. `echo` decides whether your **input** is shown back; `hush` decides whether SD's **output** is shown at all. A session that has gone completely silent has `hush on` set; one where the commands vanish but the answers still appear has `echo off`.

Dates and times

<code>date</code>	today, spelled out
<code>date internal</code>	today as SD's internal day number
<code>date n</code>	day number n, spelled out
<code>date some-date</code>	that date's day number
<code>time</code>	the time and the date
<code>time internal</code>	seconds since midnight

```
:time
14:10:34 27 AUG 2026
:date
Thursday, 27 August 2026 02:10pm
```

`date` **prints the time and** `time` **prints the date**. Both print both, in different formats and a different order, and neither name says so. `date` is the long spelled-out form and `time` is the short one.

`internal` on either gives the raw number a program would see:

```
:date internal
21424
:time internal
51088
```

the day number and seconds since midnight. `date` **with an argument is a converter, and which way it converts depends on what you give it** — a bare number is read as a day number and comes back as a date, anything else is read as a date and comes back as a day number:

```
:date 20000
Monday, 03 October 2022
:date 4 jul 2026
21370
:date zznotadate
Invalid date format
```

`time` takes no argument other than `internal`.

WHICH WAY ROUND A DATE IS READ: `DATE.FORMAT`

```
date.format on | off | display
```

```
:date.format display  
European date format is off
```

`on` selects European (day-first) date format for the session, `off` returns to the default, and `display` reports which is in force. If a default conversion code other than `D` is set, `display` names that too.

`date.format` **with no keyword prints nothing** — like `pterm`, it wants one.

Setting the machine's date is a different thing entirely — it is an administrator verb, it changes the clock for the whole installation rather than for your session, and it is in the **administrator documentation** under *Accounts and Security*.

Waiting

```
sleep n  
sleep hh:mm{:ss}
```

A number is seconds; a time is *until* that time, rolling over midnight if it has already passed. `sleep` **with no argument returns immediately** rather than sleeping for ever — the argument parses as zero.

Writing to the error log

```
logmsg text
```

Everything after the verb is the message. It prints nothing, and what it writes is a stamped, attributed entry in the system error log:

```
27 Aug 26 13:13:31 User 11 (pid 1786, don):  
    nightly run finished
```

The user number, the process id and the account name are added for you, so a message does not need to say who wrote it. The log is `C:\ProgramData\SD\sdsys\errlog`, shared by every session, which makes `logmsg` the right way for a phantom or a scheduled job to report something a person will read later — there is no terminal to print to and no transcript kept.

See also

SD TCL - The Command Processor · SD TCL - Printing and Spooling · SD Basic - Terminal Input and Output.

SD TCL - Processes and Phantoms

Seeing what a session is doing, running work in the background, and taking a snapshot of a running program.

Every SD session is a process with a **user number**, and almost everything on this page takes one. The number is not the Windows process id and it is not the account name: it is SD's own handle on a session, it is reused after a session ends, and it is what `pstat` and `pdump` both expect.

A **phantom** is a session with no terminal. You start one, it runs a command in the background under its own user number, and what it would have displayed is written to a file instead.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The listings on this page were produced by running them, on SD Core for Windows W1.0-0. The two exceptions are `phantom` and `pdebug`, and the reason is on the page: neither can be driven down a pipe, so both are described from their source rather than shown.

What is not on this page

Seeing other people's sessions, and ending them, are administrator verbs. `listu` lists every session on the machine and `logout` *n* ends one, and neither is in an ordinary account's VOC. They are documented in the **administrator documentation**, under *Sessions and Locks*, which is a separate set your administrator may or may not have given you.

`logout` **with no argument is the exception and every account has it** — it ends your own session and is `quit` under another name. That is worth knowing before typing it intending to list something.

What is here is **your own processes**: what this session is doing, work you started in the background, and how to look at either.

What a session is doing: `pstat`

```
pstat {user n} {level n} {no.page} {lptr {n}}
```

With no arguments it reports every session; `user n` restricts it to one.

```
:pstat
User Detail
  27 Account: DON
      Command: pstat
      $PSTAT 141 (262)
  12 (Not responding)
```

Each line under a user number is the account it is in, the command it is running, and where in that command it currently is — program name, source line, and the address in hexadecimal.

THE LEVELS ARE ADDITIVE

default	the current program only
---------	--------------------------

<code>level 1</code>	the whole call stack
----------------------	-----------------------------

<code>level 2</code>	the internal subroutine stack — where each <code>gosub</code> will return to
----------------------	---

<code>level 3</code>	both
----------------------	------

From inside a running program, which is where the levels are worth having — at the prompt there is nothing on the stack but `pstat` itself:

```
User Detail
  27 Account: DON
      Command: pstat user 27 level 1
      $PSTAT 141 (262)
      Command processor
      /cygdrive/c/ProgramData/SD/user_accounts/don/BP.OUT/ZZMATH 34 (152)
      Command processor
```

Read it downwards: `$PSTAT` is running, called by the command processor, which was reached by `execute` from the program `ZZMATH` at line 34, which the command processor started. `level 3` **adds an indented line under each frame** for the `gosub` inside it.

`(NOT RESPONDING)` MEANS THE PROCESS IS GONE, NOT BUSY

`pstat` does not read another session's memory. It **asks** — it posts an event to the target and waits for the answer, for up to four seconds. A session that never answers gets that line.

That is the cheapest way to find a dead session. A session killed from outside SD keeps its entry in the user table and its slot is still counted; `pstat` is what tells you nothing is behind the entry. User 12 in both listings above is one of those. **Clearing it is an administrator's job** — reporting the user number is the useful thing you can do.

Running something in the background: `phantom`

`phantom command`

The command is everything after the verb. SD creates a new process, gives it a user number, and answers at once:

Started phantom user 31

Your session does not wait, and the phantom has no terminal, so anything it would have displayed goes to a **como** record instead — in the account's `$COMO` file, named `PH n _ ddmmyy _ hhmmss` for the phantom's user number and start time. `$COMO` is created if the account has not got one. The first line in the record is the phantom's own banner:

Phantom 31 started at 14:22:07 27 Aug 2026

so a phantom that produced nothing else is still distinguishable from one that never started.

REFUSAL	WHEN
<i>Phantom command missing</i>	<code>phantom</code> with nothing after it
<i>Phantom processes cannot be started within a transaction</i>	inside <code>begin transaction</code>
<i>Failed to create phantom process</i>	the operating system would not create the process

A phantom cannot be started from a scripted session, and this is the one to know. When SD is fed commands down a pipe, the phantom child inherits that pipe. The job then never completes — not even after the parent session has exited — and the only way out is to kill the process, which leaves an entry in the user table that needs an elevated `sd -cleanup` to clear. **`phantom` is for a person at a prompt, or for a program, and not for a piped script.** The listings above are quoted from the verb's own message texts for that reason.

A phantom is not a scheduled job. It runs once, now. Recurring work is a Windows scheduled task that starts SD, and that is a different subject.

WHAT A PHANTOM DOES NOT INHERIT

A phantom is a new SD process, not a branch of yours. It is started with `fork` and `exec` and goes through login of its own, so **nothing your session holds in memory reaches it** — no `@`-variables, no unnamed common, no open files, no active select list. **The command string is the only thing passed**, through a record in SD's `$IPC` file, and the session's option settings go with it only when the `INHERIT` option is set.

What phantoms have I started: `status`

```
status
```

```
:status
```

```
There are no phantom processes started by this process
```

which is the answer in any session that has not started one. When there are some, it prints a user number, the start time and the command for each:

```
User   Started           Command
```

It reports only your own children. It is not a system-wide view and it is not `listu` — a phantom somebody else started does not appear, and neither does one your session started before you `logto`'d somewhere else. The register it reads is keyed by the parent's user number.

Dumping a process: `pdump`

```
pdump n
```

Writes a snapshot of session *n* to a file and carries on; nothing is stopped and nothing is lost. The file is `sddump.n` in the directory named by the `DUMPDIR` configuration parameter, or in SD's own system directory when `DUMPDIR` is empty, which is how it ships:

Dumping process state as C:\ProgramData\SD\sdsys/sddump.27

It holds the @-variables, the current sentence and command, the call stack, open files and their locks, and named and unnamed common — which is to say **it holds application data**, and it is written where any SD user can read it. Treat a dump as you would the data of the program that produced it, and delete it when the question it was written to answer has been answered.

REFUSAL

WHEN

User number required

`pdump` with no number

Not logged in

no session has that user number

PDUMP not allowed for processes run under other usernames

the `PDUMP` configuration parameter has bit 1 set and the session is not elevated

It is an event, not a call. `pdump` marks the target and returns; the target writes the file when it next looks at its event flags. A process that is wedged somewhere that never checks will not produce one.

Debugging a phantom: `pdebug`

`pdebug`

attach to a phantom that is waiting for a debugger

`pdebug` command

start command as a phantom and attach to it

`pdebug` puts the SD debugger on **this** terminal and the program in a **phantom**, which is the only way to debug something that has no terminal of its own. With a command it starts the phantom for you; with none it waits for one to ask for a debugger, and `Q` gives up waiting.

It refuses inside a phantom or an API session — *Phantom debugger can only be executed from an interactive session* — because it needs a terminal to draw the debugger on.

Do not send `pdebug` down a pipe. It polls the keyboard while it waits, and in a piped session the input stream is the script, so it consumes the commands that have not run yet. Together with the phantom it starts, that is two ways for one verb to hang a scripted session. **It is described here from source for exactly that reason.**

The debugger itself — the commands it takes once it is attached — is in SD Basic - Debugging.

Who has these verbs

standard

`status`

programmer

`phantom`

`pstat`

`pdebug`

`pdump`

The only one every account has is `status`, and it reports nothing but that account's own phantoms. The rest are programmer verbs: making a background process, and looking at one.

There are two gates and they are not the same one. The tier decides whether you have the verb at all; **elevation decides whether it does anything to somebody else.** `pstat` will report any session; `pdump` *n* is yours to use on your own processes and refuses for another Windows account's without an elevated session.

See also

SD TCL - Locks · SD TCL - The Terminal and the Session · SD Basic - Debugging.

SD TCL - Locks

The two kinds of lock, giving back the ones your session holds, and what you cannot do without an administrator.

There are two unrelated things called a lock, and knowing which is which is most of what this page has to say. They do not interact, and the names do not make that obvious.

database locks	on a record or on a whole file. Taken by <code>readu</code> , <code>readl</code> and <code>filelock</code> in a program, and by <code>ed</code> while it holds a record
task locks	64 numbered flags, 0 to 63, with no connection to any data . A program takes one by agreement to keep other programs out of something

The statements a program uses to take either are in SD Basic - Locks and Transactions, and this page does not repeat them.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Only one lock verb is in an ordinary account

Inspecting locks and forcing them open are administrator verbs. `list.readu`, `list.locks`, `lock`, `clear.locks` and `unlock` are not in a standard or programmer account's VOC, so the names are not recognised at all. They are in the **administrator documentation**, under *Sessions and Locks*, which is a separate set your administrator may or may not have given you.

What every account has is `release`, which gives back locks this session holds. That is the whole of the TCL lock interface for an ordinary programmer, and the reason is defensible: a lock you took is yours to give back, and a lock somebody else took is not yours to look at or remove.

Giving your own locks back: `release`

```
release filename id {id ...}
```

```
release filelock filename
```

It is the TCL form of the BASIC statement of the same name, needs no special rights, and acts **only on locks this session holds**. The file is named the way you would name it to any other verb.

```
:release
File not found
:release voc
Record id required
```

A record id is compulsory in the first form — there is no *release everything* at the prompt. `release filelock` *name* gives back a file lock, which has no id to give.

A `write` outside a transaction already releases the record lock, so `release` at the prompt is for the case where a program took a lock and ended without writing — usually because you stopped it — and for a file lock, which nothing releases implicitly.

What to do when something is stuck

You cannot see the lock table without the administrator verbs, so the useful sequence is:

-
- 1 `release` anything you know your own session took
 - 2 `status` and `pstat`, on SD TCL - Processes and Phantoms, to see whether the program you think is holding it is still alive
 - 3 if it is not yours, **report the file and record to an administrator** — they can list who holds it and force it open
-

A dead session's database locks are not released. A session killed from outside SD keeps both its user-table entry and its record and file locks, so everything wanting that record waits for a process that is not there.

Nothing an ordinary account can type will clear that, and guessing at it wastes time — the recovery is an elevated `sd -cleanup`, which is an administrator's.

Task locks, and why you will rarely meet one

A task lock protects something that is **not a file** — a nightly job, an external resource, a sequence that must not run twice at once. Two programs agree that lock 7 means *this job*, and the number carries no other meaning.

Taking and releasing one is `lock` and `clear.locks`, both administrator verbs, so in practice task locks are taken from **inside a program** with the BASIC `lock` and `unlock` statements, which any account may compile and run. That is the route to reach for; the TCL verbs exist for an administrator inspecting or clearing the table by hand.

A task lock held by a killed session is not given back by `sd -cleanup`. It stays held, by a user number nothing is behind, until SD itself is restarted — a defect, and it is recorded in the project's fix lists. If a job guarded by a task lock will not start again after a crash, that is the first thing to suspect, and clearing it needs an administrator.

Who has these verbs

standard

`release`

administrator

`list.readu` `list.locks` `lock` `clear.locks` `unlock`

The split is between your locks and everybody's. Giving back what you hold is something any session may do. Looking at the machine's lock table, taking a numbered flag, or forcing another session's lock open are all administrator verbs, and `unlock` needs an elevated session on top of that.

See also

SD Basic - Locks and Transactions · SD TCL - Processes and Phantoms · SD TCL - Files and Records.

SD VOC - Structure and Usage

The Vocabulary file: all ten record types, and every field of each one.

The VOC — Vocabulary — is the file that makes a command line work. Type a name, and the command processor looks it up here. The name of a verb, a file, a paragraph, a stored sentence, a keyword — all of them are records in the VOC. This page is about the record types, their fields, and what the verbs that manage the VOC do to them.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case, which is what this port uses on disk. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every record on this page was read from a stock account VOC on SD Core for Windows W1.0-0. The records were written by `CREATE.ACCOUNT`, which copies them from `voc_template` in the system directory. The counts are what a standard account holds; an administrator account holds more.

The ten record types

Field 1, character 1, is the type. The rest of field 1 is free text and is ignored, so a verb record whose field 1 reads `Verb to compile SD BASIC program` is type `V` and the rest is a comment. **The first character is load-bearing and the remaining thirty are not.**

TYPE	NAME	WHAT IT DOES
<code>V</code>	verb	the common case — dispatches to a program or an internal routine
<code>K</code>	keyword	a query-processor keyword; four of these are also verbs
<code>S</code>	sentence	one stored command line
<code>PA</code>	paragraph	several stored command lines, with flow control
<code>M</code>	menu	dispatched, but no menu ships
<code>Q</code>	Q-pointer	an indirect pointer to a file in another account
<code>F</code>	file	a file — local, or pointing into <code>@SDSYS</code>
<code>R</code>	remote	a pointer to a record in another file, re-read and re-parsed

TYPE	NAME	WHAT IT DOES
X	text	miscellaneous data — not a command
D	descriptor	a data descriptor, which is what dictionary records are

PQ is a valid type and is refused. PROC was removed from this port (23 Aug 2026). A record of type PQ is reported as "*PROC is not supported*" rather than being dispatched, because the record itself is valid PROC and it is the interpreter that is gone.

WHAT A STOCK VOC HOLDS

Read from `voc_template` in the system directory, a stock account VOC carries 426 records:

TYPE	COUNT
V	137
K	248
F	16
R	10
PA	4
S	2
PH	2
Q	2
X	3
Verb - Full screen editor	2

The last row is `edit`, the full-screen editor that was removed on 23 Aug 2026. Its type field reads `Verb - Full screen editor` rather than `V`, which means it is not a verb and cannot be dispatched — but it is still in the template, and `EDIT` at the command prompt reports "*Full screen editor is no longer supported*" rather than "*verb not found*". **Being in the VOC is not evidence a command works.**

The F record — a file

```
001  F
002  bp
003
```

This is the `bp` entry — the BASIC source file. Field 1 is `F`, field 2 is the operating-system path to the data portion, and field 3 is the path to the dictionary portion. An empty field 3 means the file has no separate dictionary.

FIELD

1	<code>F</code> , optionally followed by a comment
2	data path — a name in the account directory, or <code>@SDSYS/</code> <i>name</i> for a system file
3	dictionary path, or empty if none

THE EIGHT POINTERS INTO @SDSYS

A stock account VOC carries eight F-records whose data path begins `@SDSYS/`. They point at files in the system directory that every account needs:

VOC ID	FIELD 2
<code>voc</code>	<code>@SDSYS/voc.dic</code> (field 3 — the dictionary)
<code>newvoc</code>	<code>@SDSYS/newvoc</code>
<code>messages</code>	<code>@SDSYS/messages</code>
<code>syscom</code>	<code>@SDSYS/syscom</code>
<code>\$MAP</code>	<code>@SDSYS/...</code>
<code>dict.dic</code>	<code>@SDSYS/...</code>
<code>sd.voclib</code>	<code>@SDSYS/...</code>
<code>\$ipc</code>	(system IPC file)

These are **read-only to a network session**. The account-root gate in the file engine allows them on read paths but sets `FV_RDONLY` on the file variable, so every write path in the engine refuses them. An administrator in `SDSYS` is exempt; an ordinary account cannot write these files.

THE \$ACC RECORD

```
001  F
002  .
003
```

`$ACC` is the account itself. Field 2 is `.`, which means the current account directory. It is how a program opens "the account I am in" by name rather than by path.

The V record — a verb

```
001  V
002  CA
003  $CREATEF
004
```

This is `create.file`. Field 2 is the dispatch type, field 3 is the target, and the remaining fields carry options.

FIELD 2	FIELD 3	WHAT IT DOES
<code>CA</code>	<i>catalogue name</i>	a catalogued program — 99 of the shipped verbs
<code>IN</code>	<i>number</i>	internal verb <i>n</i> , handled by the command processor itself — 42 verbs
<code>OS</code>	<i>text</i>	an operating-system command — <code>sh</code> and <code>!</code> , and nothing else
<code>CS</code>	<i>path</i>	a locally catalogued function

Those four rows account for 143 of the 147 verbs an administrator account has. The remaining four are the keyword records described above — `break`, `count`, `display` and `off` — where field 2 holds a keyword number rather than a dispatch type, and it is field 3 that marks the record as a verb.

Field 4 carries dispatch options and **field 5 names a security subroutine**. If field 5 is present, that subroutine is called before the verb runs and can refuse it. **None of the shipped verbs uses field 5** — the tiering in this port is done by giving or withholding the VOC record, not by a security subroutine — but the mechanism is there for a site that wants a verb guarded rather than absent.

INTERNAL VERBS

`IN` verbs are numbered, and the numbers are positional in the command processor's dispatch list. Several names share one: `off` and `quit` are both internal verb 1, `clr` and `cs` are both 2, and each of `clear.data`, `clear.input`, `clear.prompts` and `clear.select` has a run-together spelling (`cleardata` and the rest) pointing at the same number.

THE FOUR KEYWORD-VERBS

Four records ship as a keyword **and** a verb in one: `break`, `count`, `display` and `off`. Their field 2 is the keyword number the query processor uses, and **fields 3 onward are a complete verb record**, which the command processor re-parses when the name is typed as a command.

This matters for counting what an account has. A tally of VOC records whose field 1 begins with `v` misses all four, and `count` is not a marginal verb. A standard account has **82** verbs, not 78.

The Q record — an indirect pointer

```
001  Q
002  SDSYS
003  accounts
```

This is `sd.accounts`. Field 2 is the account name, field 3 is the file name in that account. `set.file` writes Q-pointers; see below.

`md` is a Q-pointer to `voc` — the VOC's own dictionary is reached through it, and it is how `dict voc` finds its records.

The R record — a remote record reference

```
001  R
002  SD.VOCLIB
003  listf
```

This is `listf`. Field 2 is the file name, field 3 is the record id. An R-record is **re-read and re-parsed every time it is used**, so editing the referenced record changes what the command does without touching the VOC entry. The `LIST` family — `listf`, `listfl`, `listfr`, `listk`, `listpa`, `listph`, `listq`, `listr`, `lists`, `listv` — are all R-records pointing into `SD.VOCLIB`.

The S record — a sentence

```
001 S
002 DISPLAY <<@PATH>>
```

This is `where`. A sentence is one stored command line. Typing its name runs that line with anything else you typed appended, so a sentence is a command with its first arguments filled in.

The PA record — a paragraph

```
001 PA
002 TERM WINDOWS
003 TERM 120,36
004 PTERM CASE NOINVERT
```

This is `login`, the paragraph that runs at sign-on. A paragraph holds several lines and has flow control: `if`, `go`, `stop`, `abort`, `display`, `pause`. Paragraphs nest, and `abort` unwinds all of them.

`data` lines are read ahead of the verb, not executed in sequence. Before a verb runs, the command processor scans forward over any immediately following `data`, comment and blank lines and stacks their text as typed-ahead input.

THE PH RECORD — A PHRASE

```
001 PH
002 GRAND.TOTAL "'L'"
```

This is `no.grand.total`. A phrase is a stored fragment that is substituted into a command line wherever its name appears. `PH` is treated as `S` for most purposes — the difference is that a phrase is designed to be used inside another command rather than typed on its own.

`without` is also `PH`, holding `WITH NO` — so `list stock without qty` expands to `list stock with no qty`.

The K record — a keyword

```
001 K
002 26
```

This is `#`, the comment keyword. Field 2 is the keyword number the query processor uses. Keywords are not commands in their own right; they are the words the query processor — `LIST`, `SELECT`, `SORT` and the rest — recognises inside a command line. `all` is `K` with number 5; `after` is `K` with a number; `and` is `K`.

The 248 K-records in a stock VOC are the query processor's vocabulary, and they are what makes a sentence like `list stock with qty > 100 by supplier` parse.

The X record — text

```
001 X
002 Most of SD is licensed under the GPL v3.0 ...
```

This is `$licence`. An X-record is **not a command** — it is miscellaneous data the VOC holds so that a program or a user can read it by name. `$contrib` and `$RELEASE` are also X-records. Nobody dispatches them; they are read with `read` from BASIC or with `ct` from the command line.

Continued in

SD VOC - Using and Managing the VOC — how the command processor uses the VOC, the verbs that manage it, and account creation.

SD VOC - Using and Managing the VOC

How the command processor reads the VOC, the verbs that manage it, and what account creation puts there.

This page continues SD VOC - Structure and Usage.

How the command processor uses the VOC

The command processor is a BASIC program, `$CPROC`, and it runs a loop: show any waiting messages, print the prompt, read a line, dispatch it, repeat. The first word of the line is looked up in the account's `voc` file.

NAME RESOLUTION

A name is tried **as typed, then lower case, then upper case**. This is the change that lets a lower-case `voc`, `bp` and `newvoc` on disk coexist with code and habits that still type names in upper case.

Account names are the deliberate exception. They are still folded upward, which is what makes signing in case-insensitive.

IF THE VOC HAS NOTHING, THE CATALOGUE IS TRIED

A name that is not in the VOC is looked for in the **private catalogue** of the account and then in the **global catalogue**, `gcat`. A hit is treated exactly as though the VOC had held `V` / `CA` / *name*. This is why a program you have catalogued can be run by typing its name with no VOC record of its own.

If neither has it, the command is refused by name.

Verbs that manage the VOC

SET.FILE — WRITE A Q-POINTER

```
set.file account file.name pointer.name
```

Writes a Q-pointer into the VOC — field 1 `Q`, field 2 the account name, field 3 the file name. After it, *pointer.name* is usable wherever a file name is, and the data stays where it is.

The account must be in the accounts register. The name is folded to upper case before the register is read. If it is not there you get *Account name '...' is not in register* and nothing is written.

UPDATE.ACCOUNTS — REFRESH FROM NEWVOC

```
update.accounts
```

Re-runs `LOGIN`'s `update.voc` paragraph, which copies any changed records from the system `NEWVOC` into the account's VOC. **Since 17 Aug 2026 this is incremental** — it re-copies only the records whose stamp has changed, not the whole file. `TIER.OMIT.STANDARD` and `TIER.ADD.ADMINISTRATOR` in `NEWVOC` control what each account type receives.

The tier test is the one this record exists for. `update.accounts` on a standard account must not give back the verbs that were withheld when the account was created. A standard account starts with fewer verbs than an administrator account, and `update.accounts` preserves that difference.

COPY — COPY RECORDS INTO THE VOC

```
copy from voc messages,myname  
copy from src.file to dict tgt.file record.id
```

`copy` can write into the VOC the way it writes into any file. The `from voc` form copies a VOC record under a new name, which is how an alias is made without `alias`.

DELETE — REMOVE A VOC RECORD

```
delete voc name
```

Removes the record. `.d` **name at the command prompt asks first**; `delete voc name` does not.

.S AND .R — SAVE AND RECALL

```
.s name s e  
.r name
```

`.s` writes stack lines `s` to `e` into the VOC as `name` — a sentence when the range is one line, a paragraph when it is more. `.r` loads a sentence or paragraph from the VOC into the stack. **The record type follows from what you saved** rather than from anything you say.

THE LIST FAMILY

Ten verbs list VOC entries by type. They are all R-records into `SD.VOCLIB`:

VERB	LISTS
<code>listf</code>	all files (F-records)
<code>listfl</code>	local files only
<code>listfr</code>	remote files only
<code>listk</code>	all keywords (K-records)
<code>listpa</code>	all paragraphs (PA-records)
<code>listph</code>	all phrases (PH-records)
<code>listq</code>	all Q-pointers
<code>listr</code>	all remote references (R-records)
<code>lists</code>	all sentences (S-records)
<code>listv</code>	all verbs (V-records)

COUNT — COUNT THE RECORDS

```
count voc
count voc with v = "V"
```

`count` over the VOC gives the total record count. With a selection clause it counts by type or by any other field.

CT — DISPLAY A RECORD

```
ct voc name
```

Displays the record field by field. This is the way to see what a VOC entry actually contains without an editor.

The VOC and account creation

`CREATE.ACCOUNT` copies the VOC from `voc_template` in the system directory. The template holds 426 records; `NEWVOC` holds 395. The difference is that `voc_template` carries the A-verbs — `abort`, `alias`, `all`, `after`, `and` and the rest — that are also keywords, and `NEWVOC` does not. An account gets the union of the two, less whatever its tier omits.

TIER	WHAT IT GETS
administrator	everything in <code>voc_template</code> plus everything in <code>NEWVOC</code> , less <code>TIER.OMIT.STANDARD</code>
standard	everything in <code>voc_template</code> plus everything in <code>NEWVOC</code> , less <code>TIER.OMIT.STANDARD</code> (42 names)
programmer	standard plus the programmer verbs from <code>TIER.ADD.ADMINISTRATOR</code>

The tier is in the VOC, not in the verb. A standard account does not have `create.file` because the VOC record for it is not there, not because a security subroutine refuses it. The name is simply not recognised. This is the design: tiering is done by giving or withholding the record.

Case on disk

Since 18 Aug 2026 the VOC ids are stored in lower case — `list`, `create.account` and so on. This changes nothing about what you type — SD tries a name as typed, then lower, then upper. What it changes is what SD prints back: `CT VOC LIST` answers `VOC list`.

The file-pointer entries — `bp`, `bp.out`, `gpl.bp`, `gpl.bp.out` — moved to lower case on 19 Aug 2026. `VOC`, `NEWVOC`, `ACCOUNTS`, `MESSAGES`, `SYSCOM` and `QFILE` are still upper case on disk and are next.

What is not here

There is no verb that edits the VOC directly. `ED` edits any file, and `ED VOC name` is how you edit a VOC record by hand. But there is no verb whose purpose is to create or modify VOC entries — `set.file` writes one kind, `.s` writes two, and everything else is done with `ED` or with `copy from voc`.

PROC is removed. A `PQ`-type record is refused by name. Nothing that ships is type `PQ`, so this can only be met in a VOC record somebody wrote.

menu records are dispatched but no menu ships. Type `M` is handled and there is no shipped example to look at.

EDIT is in the **VOC** but **cannot run**. Its type field reads `Verb - Full screen editor`, which is not `V`, so the command processor does not dispatch it. The record is there so that **EDIT** at the prompt prints *"Full screen editor is no longer supported"* rather than *"verb not found"*.

See also

SD TCL - The Command Processor · SD TCL - Files and Records · SD Basic - File Handling · SD Basic - Dynamic Arrays.

SD Dictionaries - Structure

The dictionary file: every record type, and every field of each one.

A dictionary is a file that describes a data file. It holds the field names, the field numbers they point at, the format in which they print, and the expressions that compute values that are not stored. Every `LIST`, `SELECT`, `SORT` and `COUNT` command reads the dictionary to find out what to read and how to display it. This page is about the record types in a dictionary, their fields, and how they relate to the data file.

SD folds case, so a field name may be typed in either case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every record on this page was read from the system dictionaries on SD Core for Windows W1.0-0.

The `voc.dic` dictionary — the dictionary of the VOC — is a stock file whose records illustrate every type.

What a dictionary is

Every data file has a dictionary. The VOC record for a file of type `F` carries two paths: field 2 is the data path, field 3 is the dictionary path. If field 3 is empty, the file has no dictionary and cannot be used by the query processor — `LIST` on it fails with *"no dictionary"*.

The dictionary is itself a hashed file. Its operating-system name is the file name with `.dic` appended — `bp.dic`, `voc.dic`, `stock.dic`. The `dict` verb opens it: `dict file.name` is the same as opening `DICT file.name` from BASIC.

The system file `dict.dic` in `@SDSYS` is the dictionary of the dictionary — its own dictionary, which describes the fields a dictionary record has. It is what `dict dict.dic` reads.

The dictionary record types

Field 1, character 1, is the type. The rest of field 1 is free text and is ignored, so a record whose field 1 reads `D Data for stock qty` is type `D` and the rest is a comment.

TYPE	NAME	WHAT IT DOES
------	------	--------------

<code>D</code>	direct	references a field number in the data record
----------------	--------	--

TYPE	NAME	WHAT IT DOES
I	indirect	an expression that returns a value — a virtual field
C	calculated	like I but compiled with the BASIC compiler and returning via @ANS
A	A-type	Pick-style data definition, with a different field layout
S	S-type	identical to A in SD — kept for compatibility
L	link	used only in query processor commands, to join to another file
PH	phrase	a stored fragment substituted into a query command line
X	text	miscellaneous data — not a field definition

A and S are the Pick layout, not the SD layout. They carry the same information as D but in different fields — field 3 holds the display name, field 7 the conversion, field 8 the correlative, field 9 the justification, field 10 the width. SD supports them because applications imported from Pick use them. New dictionaries should use D and I.

The D record — a direct field reference

```
001  D
002  2
003
004  F2
005  5L
006  S
```

This is F2 from voc.dic. Field 1 is D, field 2 is the field number in the data record, and the remaining fields control how it appears in a query.

FIELD	NAME	WHAT IT HOLDS
1	type	D, optionally followed by a comment
2	location	the field number in the data record
3	conversion	a conversion code, or empty
4	display name	the column heading; \ for none; 'R' prefix right-justifies

FIELD	NAME	WHAT IT HOLDS
5	format	width and justification — see below
6	S/M	S for single-valued, M for multivalued
7	association	a name grouping multivalued fields, or empty
8	user	free for the application — SD does not read it

PSEUDO-FIELDS

Field 2 is not always a literal field number. Three values have special meaning:

LOCATION	WHAT IT READS
0	the record id (@ID)
9998	@NI — the number of items in a multivalued field set
9999	the record length in bytes (@NB)

A dictionary record with location 0 is how the record id appears in a query. The **@ID** record in **voc.dic** has location 0 and display name **@ID**.

THE FORMAT FIELD

Field 5 is the format specification — the column width and justification the query processor uses when it prints the field. It is a compact syntax:

SYNTAX	MEANING
<i>n</i> L	left-justified, <i>n</i> characters wide
<i>n</i> R	right-justified, <i>n</i> characters wide
<i>n</i> T	text — left-justified, wrapping at word boundaries
<i>n</i> C	centred, <i>n</i> characters wide
<i>n</i> L <i>d</i>	left-justified with <i>d</i> decimal places (numeric)
<i>n</i> R <i>d</i>	right-justified with <i>d</i> decimal places (numeric)

5L means five characters, left-justified. **20L** means twenty characters, left-justified. **10R2** means ten characters, right-justified, two decimal places. The format field is also what **FMT()** uses in BASIC.

SINGLE AND MULTIVALUED

Field 6 is `S` or `M`. A single-valued field holds one value per record; a multivalued field holds many, separated by value marks. The query processor prints each value of a multivalued field on its own line unless the field is in an **association**.

ASSOCIATIONS

Field 7 names an association — a group of multivalued fields whose values are parallel. If `qty` and `price` are both multivalued and both carry association `LINE.ITEM`, the query processor prints them side by side: `qty` value 1 next to `price` value 1, and so on.

An association is a name, not a structure. The fields that share one are linked by having the same name in field 7, not by anything stored separately. There is no association record.

The I record — an indirect field

```
001 I
002 IF DATA.NAME # '' THEN SUBR("!FType",DATA.NAME) ELSE ''
003
004 FType
005 8L
006 S
```

This is `FType` from `voc.dic`. Field 1 is `I`, field 2 is an expression in SD BASIC syntax, and the remaining fields are the same as for a D-record. The expression is compiled to object code, which is stored in field 16 onward, and executed when the query processor needs the value.

WHAT AN I-TYPE EXPRESSION CAN USE

An I-type expression is a single BASIC expression — not a program. It can use:

VARIABLE	WHAT IT HOLDS
<code>@ID</code>	the record id
<code>@RECORD</code>	the entire data record as a dynamic array
<code>n</code> or <code>F n</code>	field <i>n</i> of the data record — <code>F2</code> is field 2
<code>@NI</code>	the number of values in the longest multivalued field
<code>@NB</code>	the record length in bytes

VARIABLE	WHAT IT HOLDS
@FILE.NAME	the name of the file being queried
@USER.NO	the user number of the session
@PATH	the account path

It can call `SUBR()` to invoke a catalogued subroutine, use `TRANS()` to read from another file, and use any BASIC function — `LEN()`, `FIELD()`, `RAISE()`, `OCONV()`, `ICONV()`, `COUNT()`, `SUM()`, `DATE()`, `TIME()` and the rest. It cannot run loops or call statements that take a variable number of arguments.

COMPILATION

An I-type is compiled the first time it is used after it is written or edited. The compiled object is stored in field 16 onward of the same dictionary record, and a stamp in field 15 tells the query processor whether the object is current. **Editing field 2 invalidates the object.** The next query recompiles it. If the expression has a syntax error, the query reports it and the field appears empty.

THE FIELD 15 STAMP

Field 15 is the system information field. SD stores a hash of the expression source there when it compiles the I-type. When a query opens the dictionary, it compares the hash to the source — if they differ, the I-type is recompiled. This is why field 15 has a large number in a stock I-type record and is empty in a D-type record.

The C record — a calculated field

A C-type is like an I-type but is compiled with the full BASIC compiler rather than the I-type expression compiler. The expression in field 2 is a complete BASIC program that returns its result in `@ANS`. C-types are rare in stock dictionaries — I-types cover most computed fields — but they exist for compatibility with applications that use them.

The A record — a Pick-style field

An A-type (or S-type) uses a different field layout:

FIELD	NAME	WHAT IT HOLDS
1	type	A or S, optionally followed by a comment
2	location	the field number in the data record

FIELD	NAME	WHAT IT HOLDS
3	display name	the column heading
4	association	a name grouping multivalued fields
5 - 6		not used
7	conversion	a conversion code
8	correlative	a correlative code — like a mini I-type
9	justify	L or R
10	width	column width

The correlative in field 8 is a Pick-style expression — a short code that transforms the field value before display. SD compiles correlatives the same way it compiles I-types, storing the object in field 16 onward.

The PH record — a phrase

```
001 PH
002 GRAND.TOTAL "'L'"
```

A phrase is a stored fragment substituted into a query command line wherever its name appears. PH records in a dictionary work the same way as PH records in the VOC — the difference is that a dictionary phrase is scoped to the file whose dictionary holds it.

The L record — a link

An L-type is used only in query processor commands. It names another file to join to, so that fields from both files can appear in one report. An L-record carries the file name and the field to match on, and the query processor reads from the linked file for each record it lists.

The X record — text

An X-record in a dictionary is miscellaneous data — not a field definition. It can hold anything the application wants to store by name in the dictionary.

Continued in

SD Dictionaries - Using and Creating — how the query processor uses a dictionary, the system dictionaries, and creating one.

SD Dictionaries - Using and Creating

How the query processor reads a dictionary, the system dictionaries, and creating one.

This page continues SD Dictionaries - Structure.

How the query processor uses the dictionary

When a query command runs — `LIST STOCK DESCRIPTION QTY` — the query processor opens the dictionary of `STOCK` and looks up each name the command mentions.

NAME RESOLUTION IN THE DICTIONARY

A name is looked up **as typed, then lower case, then upper case**, the same resolution the VOC uses. If the dictionary has no record by that name, the query reports *"field name not found"* and stops.

If the name is not in the file's own dictionary, SD looks in the account VOC for a phrase of the same name — a `PH` record that expands to one or more dictionary field names. This is how saved selection clauses work: `list stock with qty > 100` can use a phrase `high.qty` holding `WITH QTY > 100` that is stored in the VOC, not the dictionary.

WHAT THE QUERY PROCESSOR READS

For each field name in the command, the query processor reads the dictionary record and extracts:

1. **The field number** (field 2 for D, A and S; the expression for I and C) — where to get the data.
2. **The conversion code** (field 3) — how to transform it for display.
3. **The display name** (field 4) — what to head the column.
4. **The format** (field 5) — how wide and how justified.
5. **The S/M flag** (field 6) — whether to expect multiple values.
6. **The association** (field 7) — which fields to print side by side.

For an I-type, it compiles the expression (if the stamp has changed), executes it for each record, and uses the result as the field value.

THE DEFAULT FORMAT

If field 5 is empty, the query processor picks a default — ten characters, left-justified for strings, right-justified for numbers. The column heading defaults to the field name in upper case.

The system dictionaries

Three dictionaries ship in `@SDSYS` and are pointed at by every account's VOC:

VOC ID	DICTIONARY OF
<code>voc</code>	the VOC itself (<code>@SDSYS/voc.dic</code>)
<code>dict.dic</code>	the dictionary file (<code>@SDSYS/dict.dic</code>)
<code>newvoc</code>	the NEWVOC template

The `voc.dic` dictionary is what `dict voc` reads. It has records for `@ID`, `F1` through `F5`, `TYPE`, `FTYPE`, `DESC`, `DATA.NAME`, `DICT.NAME`, `NAME`, `PROCESSOR`, `DISPATCH` and `IS.REMOTE` — the fields a VOC record has. Several of these are I-types that compute their display from other fields, which is why `dict voc` shows a type column that reads `V`, `K`, `F` and so on rather than the raw field 1.

Creating and editing dictionaries

ED — THE EDITOR

`ED DICT file.name field.name` is the way to create or modify a dictionary record by hand. The editor writes the record when you file it, and if it is an I-type, the stamp in field 15 is cleared so the next query recompiles it.

CREATE.FILE — THE DICTIONARY IS CREATED WITH THE FILE

When `CREATE.FILE` creates a data file, it creates the dictionary with it. The dictionary starts empty — no field records — and you add them with `ED` or by copying from another file's dictionary.

COPY — COPY DICTIONARY RECORDS

```
copy from dict src.file to dict tgt.file field.name
```

`copy` can copy dictionary records between files, which is the way to clone a field definition from one file's dictionary to another.

GENERATE — CREATE INCLUDE FILES FROM DICTIONARIES

`GENERATE` reads a dictionary and writes a BASIC include file (`.H`) that defines field numbers as constants. This is how a program avoids hard-coding field numbers: `$INCLUDE file.H` brings in `F1`, `F2` and the rest, and if a field number changes, the include file is regenerated and every program that includes it picks up the change on

recompile.

What is not here

There is no verb that lists dictionary records by type. `DICT file.name` displays the records, and `LIST DICT file.name` runs the query processor over them, but there is no `listd` or `listdict` verb. `ED DICT file name` edits one record; `DICT file` displays several.

L-types are not in a stock dictionary. Type `L` is handled by the query processor and there is no shipped example. It exists for applications that need cross-file joins in a query.

The `C` type is rare. Stock dictionaries use `D` for direct fields and `I` for computed ones. `C` exists for compatibility and is functionally a subset of `I` — use `I` for new computed fields.

See also

SD Dictionaries - Conversions and Formatting · SD VOC - Structure and Usage · SD TCL - The Query Processor · SD Basic - File Handling · SD Basic - Dynamic Arrays.

SD Dictionaries - Conversions and Formatting

Every conversion code SD recognises, and what each one does to a stored value.

A dictionary record does two things: it says where to find the data (field 2), and it says how to display it (fields 3 and 5). Field 3 is the **conversion code** — a compact instruction that transforms a stored value into a display value. This page is every conversion code SD recognises; field 5, the format specification, is on the page that continues it.

SD folds case, so a conversion code may be typed in either case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every conversion code on this page was read from the SD source on SD Core for Windows W1.0-0. The C engine in `op_oconv.c` and `op_iconv.c` is the authority; the descriptions here follow what it does, not what an upstream manual says it ought to do.

Conversion codes — field 3

A conversion code transforms a value. In a dictionary record, field 3 holds the code the query processor applies to each value before it prints it. In BASIC, the same codes are the second argument to `OCONV()` (for output) and `ICONV()` (for input).

A conversion code is a string whose first character or two identifies the conversion; the rest is parameters. Multiple conversions can be chained with value marks, and each is applied in turn.

DATE CONVERSION — D

D {y} {c} {fmt}

A date conversion takes an SD internal date (days since 31 Dec 1967) and formats it for display. The *y* digit controls how many year digits to show; *c* is a calendar code; *fmt* is a format string in brackets.

YEAR DIGITS	WHAT IT SHOWS
(omitted)	four-digit year (default)
2	two-digit year

YEAR DIGITS	WHAT IT SHOWS
4	four-digit year

The format string, if present, is enclosed in square brackets and is made of these elements, separated by commas:

ELEMENT	WHAT IT PRODUCES
D	day of month, one or two digits
DD	day of month, two digits
M	month, one or two digits
MM	month, two digits
MA	month name, abbreviated (Jan)
ML	month name, full (January)

ML and the rest of this table are elements of a date mask, used inside a **D** conversion such as **D[DD ML YYYY]**. They are not conversions in their own right: `oconv(20899, 'ML')` returns the value unchanged, because **ML** alone is not a date conversion and SD passes through what it cannot convert. | **Y** | year, two digits | | **YY** | year, four digits | | **J** | Julian day of year | | **Q** | quarter number | | **W** | day of week, abbreviated | | **WL** | day of week, full | | **N** | day of year |

D2[DD/MM/YY] produces `29/08/26`. **D[DD MMM YYYY]** produces `29 Aug 2026`. Without a format string, the default follows the system date format — **D** alone produces `29 Aug 2026` on a machine set to European date order.

TIME CONVERSION — MT

MT {H} {S} {c}

A time conversion takes an SD internal time (seconds since midnight) and formats it.

OPTION	WHAT IT DOES
H	use 12-hour format with AM/PM suffix
S	include seconds
c	separator character (defaults to colon)

`MT` produces `14:30`. `MTH` produces `02:30 PM`. `MTHS` produces `02:30:45 PM`.

MASKED DECIMAL — MD, ML, MR

`MD d {f} {x{c}}`

`ML d {f} {x{c}}`

`MR d {f} {x{c}}`

A masked decimal conversion formats a number with a fixed number of decimal places. `MD` is the general form; `ML` left-justifies the result; `MR` right-justifies it.

PARAMETER	MEANING
<i>d</i>	number of decimal places (0-9)
<i>f</i>	scale factor (0-9), defaults to <i>d</i>
<i>x</i>	field width
<i>c</i>	padding character (defaults to space)

`MD2` formats a number with two decimal places: `1234.56`. The scale factor matters when the stored value is an integer that represents a scaled number — `MD20` with scale factor `0` treats `123456` as `1234.56`.

CASE CONVERSION — MC

`MCL` `MCU` `MCC` `MCT`

CODE	WHAT IT DOES
<code>MCL</code>	convert to lower case
<code>MCU</code>	convert to upper case
<code>MCC</code>	capitalise each word
<code>MCT</code>	title case — the first letter of every word: <code>hello there world</code> becomes <code>Hello There World</code>

RADIX CONVERSION — MX, MO, MCD, MCX

`MX` `MO` `MCD` `MCX`

CODE	WHAT IT DOES
<code>MX</code>	decimal to hexadecimal
<code>MO</code>	decimal to octal
<code>MCD</code>	hexadecimal to decimal
<code>MCX</code>	decimal to hexadecimal (same as MX)

`MX0C` converts a hexadecimal string to characters. `MB0C` and `M00C` do the same for binary and octal.

BOOLEAN CONVERSION — B

B

Converts a value to `Y` (true) or `N` (false). A zero or null is `N`; anything else is `Y`. For input conversion, `Y` becomes `1` and `N` becomes `0`.

LENGTH CONVERSION — L

L`{n} {,m}`

Returns the length of the string. If *n* is given, the result is true if the length is *n*; if *n* and *m* are given, true if between *n* and *m*.

PATTERN MATCHING — P

P`(xx)`

Tests whether the value matches a pattern. The pattern is made of codes: `N` for numeric, `A` for alphabetic, `X` for any character, with a count. `P(3N2A)` matches three digits followed by two letters.

RANGE TEST — R

R`n,m{;n,m...}`

Tests whether the value falls in one or more ranges. `R1,10;20,30` is true if the value is between 1 and 10 or between 20 and 30.

SUBSTRING EXTRACTION — T

`T{n,}m`

Extracts a substring. `T3,5` extracts characters 3 through 7. `T5` is shorthand for the first 5 characters.

CONCATENATION — C

`C;v1;v2;v3`

Concatenates the value with the supplied strings. `C;ABC;` produces the value followed by `ABC`.

SUBSTITUTION — S

`S;v1;v2;v3`

Substitutes the value with the n -th string. `S;Y;N` produces `Y` if the value is `1`, `N` if it is `2`.

GROUP CONVERSION — G

`G{skip}df`

Splits a string into groups. *skip* is the number of characters to skip between groups; *d* is the delimiter; *f* is the number of characters per group.

BINARY CONVERSIONS — IF, IL, IS

`IFx` `ILx` `ISx`

CODE

WHAT IT DOES

<code>IFx</code>	8 bytes to floating point
<code>ILx</code>	4-byte binary integer to number
<code>ISx</code>	2-byte binary integer to number

x is `L` (low byte first), `H` (high byte first), or omitted for the machine's native byte order.

BASE64 — B64

```
B64
```

For output, encodes the value as Base64. For input, decodes a Base64 string back to binary.

TRANS — T

```
Tfile;cv;i;o
```

A translation conversion: looks up the value in *file*, using conversion *cv*, and returns field *i*. If not found, returns *o*. This is the same as the BASIC `TRANS()` function and is how a dictionary field can display data from another file.

FIELD EXTRACTION — ANGLE BRACKETS

```
<f,v,s>
```

Extracts field *f*, value *v*, subvalue *s* from the value treated as a dynamic array. This is the same as the BASIC angle-bracket extraction.

Continued in

SD Dictionaries - Formats and Expressions — format specifications, I-type expressions, correlatives and chaining.

SD Dictionaries - Formats and Expressions

Format specifications, l-type expressions, correlatives, and chaining conversions.

This page continues SD Dictionaries - Conversions and Formatting.

Format specifications — field 5

Field 5 controls how the query processor lays out the column. It is also the argument to the BASIC `FMT()` function, and the syntax is the same.

```
{length} {fill.char} justification {decimal_places} {options}
```

PART	MEANING
<i>length</i>	field width in characters
<i>fill.char</i>	a quoted character to pad with (defaults to space)
justification	<code>L</code> , <code>R</code> , <code>T</code> , <code>C</code> or <code>U</code>
<i>decimal_places</i>	number of decimal places (0-9)
<i>scale_factor</i>	a second digit — the scale factor

JUSTIFICATION

CODE	MEANING
<code>L</code>	left-justified
<code>R</code>	right-justified
<code>T</code>	text — left-justified, wrapping at word boundaries
<code>C</code>	centred
<code>U</code>	left-justified, unchanged — no trimming

NUMERIC OPTIONS

After the justification, a format can carry these flags:

FLAG	MEANING
\$	prefix with the currency symbol
,	insert thousands separators
Z	suppress zero — print nothing for zero
B	blank if negative — print spaces for a negative number
C	suffix CR for negative, blank for positive
D	suffix DB for negative, blank for positive
E	enclose negative in angle brackets
M	suffix minus for negative
N	prefix minus for negative

EXAMPLES

FORMAT	INPUT	OUTPUT
10L	hello	hello
10R	123	123
10R2	1234.5	1234.50
10R2,	1234567.89	1,234,567.89
10R2\$	1234.5	\$1234.50
8L#	hello	###hello
20T	long text	wrapped at 20 characters
10RZ	0	(empty)

DATE SHORTCUT — D IN THE FORMAT FIELD

If the format field begins with `D`, it is not a format at all — it is a date conversion. `D2[DD/MM/YY]` in field 5 does the same as putting the same string in field 3. This is a convenience: a D-type record can carry its date conversion in either field.

HOW CONVERSION AND FORMAT INTERACT

Conversion (field 3) and format (field 5) are applied in sequence: the conversion transforms the stored value first, then the format controls how the converted value is laid out in the column.

STEP	WHAT HAPPENS
1. Stored value	the raw data from the record field
2. Conversion (field 3)	transforms the value — date formatting, numeric masking, case conversion, translation lookup
3. Format (field 5)	controls width, justification, padding, and numeric display options

A date field stores an internal date (days since 31 Dec 1967). The conversion `D2[DD/MM/YY]` turns `20899` into `29/08/26`. The format `10L` left-justifies it in a 10-character column. Both are needed: the conversion makes the value readable, the format makes the column neat.

A format cannot convert. If the format field begins with `D`, it is a date conversion, not a format — see above. But a format that begins with a digit, `L`, `R`, `T`, `C` or `U` is always a layout specification and never transforms the data. If you need the data transformed, put the conversion in field 3.

When both fields are present, the conversion runs first and the format is applied to the converted result. When only the format is present, it is applied to the raw stored value. When only the conversion is present, the converted value is printed with a default format — the query processor works out a width from the data.

COMBINED EXAMPLES

Every row below was run on W1.0-0 and shows what came back, including the trailing spaces the format adds.

FIELD 3 (CONVERSION)	FIELD 5 (FORMAT)	STORED VALUE	OUTPUT
<code>D2/</code>	<code>10L</code>	<code>20899</code>	<code>03/20/25</code>
<code>D</code>	<code>12L</code>	<code>20899</code>	<code>20 MAR 2025</code>
<code>MD2</code>	<code>10R2,</code>	<code>1234567</code>	<code>12,345.67</code>
<code>MD2,</code>	(none)	<code>1234567</code>	<code>12,345.67</code>

FIELD 3 (CONVERSION)	FIELD 5 (FORMAT)	STORED VALUE	OUTPUT
MCU	10L	hello	HELLO
MCT	(none)	hello there world	Hello There World
(none)	10R	123	123
MX	8R	255	FF
MB	(none)	5	101

Three of these are worth reading twice.

Internal date 20899 is 20 March 2025. Day zero is 31 December 1967, so an internal date does not correspond to anything you can guess.

D2/ produces 03/20/25 — month first. The bare **D** conversion produces **20 MAR 2025**, with the month abbreviated in capitals. If you want day first, say so in the conversion rather than relying on a default.

MD2 scales rather than truncating. MD2 on 1234567 is **12345.67**, not **1234.56** — the digit count says where the decimal point goes, and no digits are lost.

I-type expressions — field 2 for type I

When the dictionary record is type **I**, field 2 is not a field number — it is an expression. The expression is compiled and executed for each record the query processor lists, and the result is the field value.

WHAT THE EXPRESSION CAN CONTAIN

An I-type expression is a single SD BASIC expression. It can use:

- **Field references:** **F2** is field 2 of the current record. **@RECORD** is the whole record as a dynamic array.
- **Pseudo-variables:** **@ID** (the record id), **@NI** (number of values in the longest multivalued field), **@NB** (record length), **@FILE.NAME**, **@USER.NO**, **@PATH**.
- **BASIC functions:** **LEN()**, **FIELD()**, **INDEX()**, **COUNT()**, **SUM()**, **OCONV()**, **ICONV()**, **DATE()**, **TIME()**, **RAISE()**, **DOWNCASE()**, **UPCASE()**, **TRIM()**, **STR()**, **SPACE()**, **SEQ()**, **CHAR()**, **ABS()**, **INT()**, **MOD()**, **SQRT()**, **TRANS()** and the rest.
- **Subroutine calls:** **SUBR("name", arg, ...)** invokes a catalogued subroutine. The **FTYPE** I-type in **voc.dic** calls **SUBR("!FTYPE", DATA.NAME)**.
- **Conditional expressions:** **IF condition THEN value ELSE value**. The **FTYPE** I-type is **IF DATA.NAME # '' THEN SUBR("!FTYPE",DATA.NAME) ELSE ''**.

WHAT THE EXPRESSION CANNOT CONTAIN

An I-type expression cannot use:

- **Statements:** no `OPEN`, `READ`, `WRITE`, `PRINT`, `INPUT`, `FOR`, `LOOP`, `GOTO`, `GOSUB`.
- **Multiple statements:** no semicolons between statements. The whole field 2 is one expression.
- **Dimensioned arrays:** no `DIM`, no `MAT`.
- **Common blocks:** no `COMMON`.

If you need any of these, write a subroutine, catalogue it, and call it with `SUBR()` from the I-type.

COMPILATION AND THE STAMP

The expression is compiled to p-code the first time it is used after it is written or edited. The object is stored in field 16 onward, and a hash of the source is stored in field 15. When a query opens the dictionary, it compares the hash to the current source — if they differ, the I-type is recompiled.

Editing the expression invalidates the object. You do not need to do anything to trigger recompilation — the next query that reads the field does it. If the expression has a syntax error, the query reports it and the field appears empty.

I-TYPE AND CONVERSION

An I-type record can have both an expression (field 2) and a conversion code (field 3). The expression runs first, producing a value, and the conversion is then applied to that value. An I-type that returns an internal date with `D` in field 3 will display it formatted — the expression computes the date, the conversion formats it.

Correlatives — field 8 for type A

A-type records carry a correlative in field 8. A correlative is a Pick-style expression — a short code that transforms the field value before display. SD compiles correlatives the same way it compiles I-types, storing the object in field 16 onward.

Correlative codes are a Pick inheritance and are rarely used in new SD dictionaries. An I-type expression is easier to read and can express calculations a correlative string cannot. The correlative mechanism exists so that dictionaries imported from Pick systems continue to work.

Chaining conversions

A conversion code can contain multiple conversions chained with value marks (char 253). Each is applied in turn to the result of the previous one. This is how a dictionary field can extract a substring, translate it, and format it in one pass — the value mark separates the steps.

Input and output conversion

The same conversion code works for both `OCONV()` (output) and `ICONV()` (input), but not every code is meaningful in both directions. `MCL` converts to lower case on output; on input it does the same. `D2[DD/MM/YY]` formats a date on output; on input it parses one. `MX` converts to hex on output; on input it converts from hex.

Some codes are output-only or input-only. `B64` encodes on output and decodes on input. `P(xx)` is a pattern test — it returns the original value or null on output, and accepts or rejects on input.

See also

[SD Dictionaries - Structure](#) · [SD VOC - Structure and Usage](#) · [SD TCL - The Query Processor](#) · [SD Basic - String Functions](#) · [SD Basic - Data Conversion](#).

SD File System Concepts

The two file types, group structure, overflow, on-disk layout, and how the hash algorithm works.

SD stores data in files, and each file is one of two types: a **directory file** or a **dynamic file**. The two have different on-disk layouts, different performance characteristics, and different rules for what can read them.

Directory files

A directory file is an **ordinary Windows folder** with one file per record. The record id is the file name.

On disk	C:\ProgramData\SD\user_accounts\<account>\<filename>\
Record id	the file name
Readable by	any Windows program (Notepad, Excel, etc.)
Record ids	matched case insensitively
<code>create.file</code> option	<code>no.case</code> for explicit case-insensitive ids

The `bp` file is a directory file. VOC, `batch.jobs`, `os.users`, and the dictionaries are all directory files.

SD writes directory file records with CR+LF line endings so that Windows programs can open them. Reading handles CR+LF, LF, and CR on its own.

Dynamic files

A dynamic file is stored in SD's own binary format. It has a hash-based group structure and resizes automatically as data grows or shrinks.

On disk	binary files in the account directory
---------	---------------------------------------

Record id	up to <code>MAXIDLEN</code> , which is 63 by default
Readable by	SD only
Group size	set by <code>GRPSIZE</code> in <code>sd.conf</code> , 2 KB by default
Resizing	automatic, at a load of 80% to grow and 50% to shrink

GROUP STRUCTURE

A dynamic file is divided into **groups**. Each group is a bucket holding the records that hash to it. `analyse.file` reports the structure of a real file:

```
:analyse.file zzauditf
Account      : /cygdrive/c/ProgramData/SD/user_accounts/don
File name    : zzauditf
Path name    : /cygdrive/c/ProgramData/SD/user_accounts/don/ZZAUDITF
Type         : Dynamic, version 2
Group size   : 2 (2048 bytes)
Large record size : 1638
Minimum modulus : 1
Current modulus : 1
Load factors  : 80 (split), 50 (merge), 0 (current)
File size (bytes) : 6144 (4096 + 2048)
```

The **modulus** is the number of groups. A newly created file starts at 1.

IT GROWS ONE GROUP AT A TIME

This is the part most worth understanding, because the obvious guess is wrong.

The file does not double. When the load passes the split factor, SD adds **one** group and redistributes the records of **one** existing group into it. When the load falls below the merge factor, one group is merged away. The modulus therefore moves by one in either direction and is not a power of two.

Two consequences follow:

Growth is incremental	there is no pause while a large file is rebuilt, because no resize ever touches more than two groups
The cost is spread	a file that grows steadily splits steadily, rather than stalling at each doubling

`minimum modulus` is the floor: merging never takes the file below it. Set it with `configure.file` when you know a file will refill after being cleared.

OVERFLOW

When a group holds more data than fits in its primary buffer, further buffers are chained to it. That chain is **overflow**, and it has to be scanned on every read that lands in that group, so heavy overflow shows up as slow reads.

Overflow is normal in small amounts. It becomes a problem when a group holds records that will not fit however the file is sized — the *large record size* in the listing above is the threshold at which a record is stored separately instead.

`analyse.file` reports the distribution; `fstat` reports read and write statistics.

The dictionary file

Every data file has a companion dictionary, named `<FILENAME>.DIC` in the same account directory. `create.file` makes both parts and says so:

```
:create.file zzauditf
Created DICT part as ZZAUDITF.DIC
Created DATA part as ZZAUDITF
Added default '@ID' record to dictionary
```

`delete.file` removes both, and the VOC entry with them. The dictionary defines the fields for the query processor. See SD Dictionaries - Structure and SD Dictionaries - Conversions.

The sub-file structure

An account directory contains:

<code><FILENAME></code>	the data part (a folder for a directory file, binary for a dynamic one)
<code><FILENAME>.DIC</code>	the dictionary part
<code>voc</code>	the account's VOC
<code>bp</code>	BASIC source

<code>bp.out</code>	the compiled objects
<code>\$hold</code>	deferred print output
<code>\$svlists</code>	saved select lists, reached through the VOC name <code>\$savedlists</code>

File analysis

```
analyse.file customers
analyse.file customers statistics
fstat customers on
```

<code>analyse.file</code>	the file's structure — type, group size, modulus, load factors and size, as shown above. <code>statistics</code> adds the group distribution
<code>fstat</code>	read and write counts. It is a switch : <code>fstat file on</code> starts collecting, <code>off</code> stops, and <code>fstat</code> alone reports. <code>fstat global</code> covers every file and <code>fstat reset</code> clears the counters

`fstat` collects nothing until it is turned on, so a report immediately after an `on` is empty rather than wrong.

Creating and configuring files

```
create.file customers
create.file orders directory
create.file dict customers
configure.file customers default
```

<code>create.file</code> <i>name</i>	a dynamic file, which is the default. <code>directory</code> makes a directory file instead, and <code>dict</code> creates only the dictionary part
<code>delete.file</code> <i>name</i>	removes the data part, the dictionary part and the VOC entry. <code>dict</code> or <code>data</code> removes only one part

`clear.file` *name* empties a file, keeping its structure. `dict` or `data` selects a part

`configure.file` *name* changes group size and load factors. `default` puts them back

There is no `static` file type. The two types are directory and dynamic.

`delete.file` refuses `voc` and `$acc`, and the refusal no longer depends on the case you type them in.

SD Standard Subroutines

The 42 catalogued names beginning with !, which of them an application may use, and how each is called.

SD ships forty-two catalogued routines whose names begin with `!`. They are in the global catalogue, so any account reaches them without cataloguing anything of its own.

Most exist to support SD itself and are not an interface for applications. They are listed in full because a name in a stack trace, a `map` listing or an error message has to be findable, and because a handful are genuinely useful. Which is which is said plainly below rather than left for you to discover.

These are not a versioned interface. A routine here can change between releases in ways a documented function would not. If you call one, test what it returns rather than assuming its shape holds.

Some are functions and some are subroutines

Get this right first, because calling one the wrong way does not produce a helpful error.

A **subroutine** is reached with `call`, and its results come back in its arguments:

```
call "!ERRTEXT", text, 3035
```

A **function** is declared with `deffun` and returns a value:

```
deffun ftype(path) calling "!FTYPE"  
kind = ftype(@path)
```

`SUBR()` calls a subroutine from an l-type expression in a dictionary.

Nothing here is typed at the TCL prompt. These are called from SD BASIC.

The ones an application may reasonably use

PATHS AND NAMES

<code>!ABSPATH</code>	<code>abspath((dir), rel)</code> — resolve <i>rel</i> against <i>dir</i> and return an absolute path
<code>!PATHTKN</code>	<code>pathtkn((path))</code> — split a path into its components
<code>!FTYPE</code>	<code>ftype(path)</code> — what kind of file is at <i>path</i>
<code>!VALID_OS_NAME</code>	<code>valid_os_name(name)</code> — is <i>name</i> usable as an operating system account name
<code>!VALID_OS_PATH</code>	<code>valid_os_path(path)</code> — is <i>path</i> a well-formed host path
<code>!VALID_SHELL_CMD</code>	<code>valid_shell_cmd(cmd)</code> — is <i>cmd</i> acceptable to hand to a shell

The three validators are the ones SD's own administrative verbs use before handing a value to Windows. If you are building a command line or an account name from input, call them rather than reimplementing the rules.

SESSIONS AND USERS

<code>!USERNAME</code>	<code>username(userno)</code> — the name of the session with that user number
<code>!USERNO</code>	<code>userno((name))</code> — the user number of a named session

ERRORS, FORMATTING AND SORTING

<code>!ERRTEXT</code>	<code>call "!ERRTEXT", expansion, errno</code> — the message text for a status code
<code>!FORMAT</code>	<code>call "!FORMAT", in.rec, out.rec, default.file, indent.step, case.option, errors</code> — the source formatter behind the <code>format</code> verb
<code>!SORT</code>	<code>call "!SORT", in.list, out.list, mode</code> — sort a list
<code>!VOCREC</code>	<code>call "!VOCREC", rec, (id)</code> — build or check a VOC record

`!ERRTEXT` is the useful one. A program that traps a status code and wants to log something a person can read should call it rather than keep its own table of numbers.

SELECTION LISTS ON SCREEN

!PICK	call "!PICK", item, top.line, item.list, title, pick.pos	— a scrolling chooser
-------	--	-----------------------

!PICKLST	call "!PICK.LIST", value, ...	— the list variant
----------	-------------------------------	--------------------

Both need a real terminal. Neither does anything useful in a phantom, a scheduled job or an API session.

PRINTING AND PROGRAM INFORMATION

!PCL	pcl(key, arg1 ... arg6)	— printer control sequences, variable arguments
------	-------------------------	---

!PROG_INFO	call "!PROG_INFO", obj_dir_path, obj_name, prog_name, prog_obj_sz, prog_comp_time, prog_comp_date, error_msg	
------------	--	--

THE COMMAND PARSER

!PARSER	call "!PARSER", key, type, string, keyword, voc.rec, quote.char	— variable arguments
---------	---	----------------------

!PARSER is the tokeniser SD's own verbs use to read their arguments, and the keys it takes are defined in SYSCOM/PARSER.H. It is the most useful of the internal routines and the one most likely to change.

The internal ones

These exist for SD's own use. They are catalogued because SD's programs are ordinary compiled programs and reach them the same way anything else does.

Calling them from an application is not supported. Several refuse a session that is not elevated or not administrative, and some change the state of the machine.

!ATVAR	!SETVAR	read and set the @ variables
--------	---------	------------------------------

!GETPU	!SETPU	read and set per-user values
--------	--------	------------------------------

!CREATE_USER !DELETE_USER !SET_PASSWD	the Windows account half of create.account, delete.account and modify.password
!CRED_SET !CRED_VERIFY	write and check a credential in the credential store
!SD_GET_SALT !SD_KEY_FROM_PW	the key derivation behind that credential
!EUID_SET !EUID_RESTORE	the POSIX effective identity calls
!ELEVATE	starts, uses and stops the elevated helper
!PS_SCRIPT !PS_SCRIPT_OUT	run a PowerShell script through that helper, without and with its output
!IS_USER !IS_GROUP !IS_GRP_MEMBER !IS_SD_USER !OS_GROUP	Windows account and group questions
!PROFILE_DIR	profile_dir(username) — where a Windows profile lives
!SD_ADMIN_TIER !TIER_ALLOWS	the account tier, and whether it permits something
!SDCLIENT	the server side of the client API

What is not here

Documentation for other MultiValue systems describes `!`-prefixed routines SD Core for Windows does not have. There a reader may come looking for:

!OCONV !ICONV	not catalogued. <code>oconv()</code> and <code>iconv()</code> are ordinary BASIC functions and do the same work
!SCREEN	not catalogued, and there is no equivalent

To settle the question for any name, use `map` in the account. It lists what the catalogue holds. A name that is not there is not available, and calling it fails at run time rather than at compile time.

See also

SD Dictionaries - Conversions and Formatting · SD TCL - Programs and the Catalogue · SD Basic - Program Structure.

SD Client API

The sdclilib client library, its functions across four language bindings, and the server status codes.

The SDClient API lets an external application connect to SD, open files, read and write records, execute commands, call subroutines, and manage select lists — all through a single shared library.

The API is a normal way for any account to use SD, not a facility reserved for developers and administrators. A person running a custom GUI program that talks to SD needs API access and may need nothing else.

The library

Four DLLs are installed. They are built from one source: each pair is the same code compiled twice under two output names, so the two files are not identical on disk but behave identically.

64-bit	sdclilib.dll	and	sdclient.dll
--------	--------------	-----	--------------

32-bit	qmclilib.dll	and	qmclient.dll
--------	--------------	-----	--------------

The `*clilib` names are what existing applications ask for; the `*client` names are for new work. **Renaming one to the other does not work**: an import library records the DLL name its symbols come from, so an application built against `sdclilib` loads `sdclilib.dll` whatever you call the file on disk. That is why both names are built rather than one being a copy of the other.

The architecture must match the application, not the machine. A 32-bit application on 64-bit Windows needs the 32-bit DLL. The 32-bit build is a shipping deliverable rather than a testing convenience.

WHERE THEY ARE, AND HOW TO USE THEM

C:\Program Files\SD\usr\clients\client64\	sdclilib.dll	sdclient.dll
C:\Program Files\SD\usr\clients\client32\	qmclilib.dll	qmclient.dll

Copy the DLL your application needs either **beside the application's own executable** or into `C:\Windows\System32`. Those are the two supported routes and either works.

`C:\Program Files\SD\usr\bin` is normally on the system **PATH**, because the installer offers to put it there — the "Add SD Core to the system PATH" task, which is ticked unless you untick it, and it is what makes `sd` run from any directory. **All four client DLLs live in that directory too**, so an application may find one without your having copied anything. That is convenient, and it is not the same as choosing which copy it loads: PATH order decides, and a stale copy earlier on the PATH wins. **Put the DLL where your application will find it deliberately.**

`usr\clients` holds the DLLs **and one import library for each** — `.dll.a` files, GNU-style, for linking rather than loading:

```
client64\    libsdclilib.dll.a  libsdclient.dll.a
client32\    libqmcclilib.dll.a libqmclient.dll.a
```

They sit beside the DLLs so that everything a client *build* needs is in one place. **No header travels with them:** `sdclilib.h` reaches every installation at `C:\ProgramData\SD\sdsys\syscom\sdclilib.h`.

All four DLLs also appear in `C:\Program Files\SD\usr\bin`, **beside** `sd.exe`, and each copy is byte-identical to the one under `usr\clients`. The 32-bit pair is there because `usr\bin` is on the PATH, and that is where a **32-bit administrative utility** finds its client. Those copies are the server's own and are not the ones you should be taking — with one exception, which is the next section. **The import libraries are deliberately not in** `usr\bin`: a linker input has no business in a directory that goes on the PATH.

Connection

<code>SDConnect(host, port, user, pass, account)</code>	over the network, to port 4243
---	---------------------------------------

<code>SDConnectLocal(account)</code>	on the same machine. Sends no password and never did
--------------------------------------	--

`SDConnectUDS` (Unix Domain Socket) appears in the header but is not applicable on Windows. The Windows port supports local and TCP connections only.

SDCONNECTLOCAL HAS A REQUIREMENT THE OTHER DOES NOT

`SDConnectLocal` starts a session by running `sd.exe`, and **it looks for** `sd.exe` **beside itself** — in the directory the loaded DLL came from, not on the PATH.

So a copy of the DLL sitting next to your own application will not make a local connection: there is no `sd.exe` there. Two ways round it:

Load the copy in `usr\bin` it is beside `sd.exe`, which is why that copy exists

Use `SDConnect` instead connect to `127.0.0.1` on port 4243 like any other client

`SDConnectLocal` sends no password at all. It takes the identity of the process that called it and checks that account's grants, so the account has to be one the calling Windows user may enter.

The login is SCRAM-SHA-256. A client that sends a password in clear is refused. The server sets a puzzle only someone who knows the password can answer, and the password itself is never sent in any form. The server also proves itself to the client — another program that grabbed the port before SD started cannot pretend to be SD.

Run `modify.password` again for every account that uses the API after upgrading. The stored credentials changed shape and the old ones cannot be converted — the password was never kept anywhere, by design.

A session is confined to its own account. An API session can open everything inside its own account and the shipped SDSYS files every account needs, but cannot open, rename, delete or list anything else.

Server status codes

CONSTANT	VALUE	MEANING
<code>SV_OK</code>	0	success
<code>SV_ON_ERROR</code>	1	an <code>ON ERROR</code> clause fired
<code>SV_ELSE</code>	2	an <code>ELSE</code> clause fired
<code>SV_ERROR</code>	3	an error occurred
<code>SV_LOCKED</code>	4	the record is locked by another session
<code>SV_PROMPT</code>	5	the command issued a prompt and is waiting for input

Function reference

CONNECTION MANAGEMENT

FUNCTION	RETURNS	DESCRIPTION
<code>SDConnect(host, port, user, pass, account)</code>	Boolean	connect over TCP
<code>SDConnectLocal(account)</code>	Boolean	connect on the same machine
<code>SDConnected()</code>	Boolean	is a session active
<code>SDDisconnect()</code>	none	end the current session
<code>SDDisconnectAll()</code>	none	end all sessions
<code>SDGetSession()</code>	Integer	get the current session number
<code>SDSetSession(session)</code>	Boolean	switch to a session
<code>SDLogto(account)</code>	Boolean	switch to another account

FILE OPERATIONS

FUNCTION	RETURNS	DESCRIPTION
<code>SDOpen(filename)</code>	Integer (file number)	open a file
<code>SDClose(fileNo)</code>	none	close a file
<code>SDRead(fileNo, id, err)</code>	String	read a record
<code>SDReadl(fileNo, id, wait, err)</code>	String	read with a shared lock
<code>SDReadu(fileNo, id, wait, err)</code>	String	read with an update lock
<code>SDWrite(fileNo, id, data)</code>	none	write a record
<code>SDWriteu(fileNo, id, data)</code>	none	write with an update lock
<code>SDDelete(fileNo, id)</code>	none	delete a record
<code>SDDeleteu(fileNo, id)</code>	none	delete with an update lock

FUNCTION	RETURNS	DESCRIPTION
<code>SDRecordlock(fileNo, id, updateLock, wait)</code>	none	set a record lock
<code>SDRelease(fileNo, id)</code>	none	release a record lock
<code>SDMarkMapping(fileNo, state)</code>	none	enable/disable mark mapping

RECORD MANIPULATION

FUNCTION	RETURNS	DESCRIPTION
<code>SDExtract(src, fno, vno, svno)</code>	String	extract a field, value or subvalue
<code>SDIns(src, fno, vno, svno, newdata)</code>	String	insert into a dynamic array
<code>SDDel(src, fno, vno, svno)</code>	String	delete from a dynamic array
<code>SDReplace(src, fno, vno, svno, newdata)</code>	String	replace a field, value or subvalue
<code>SDLocate(item, src, fno, vno, svno, pos, order)</code>	Boolean	find a position for ordered insert

STRING FUNCTIONS

FUNCTION	RETURNS	DESCRIPTION
<code>SDField(str, delim, first, occurrences)</code>	String	extract a field from a delimited string
<code>SDDcount(src, delim)</code>	Long	count delimiters
<code>SDChange(str, old, new, occurrences, start)</code>	String	replace substrings
<code>SDMatch(src, template)</code>	Boolean	match a string against a pattern
<code>SDMatchfield(src, template, component)</code>	String	extract a matched component
<code>SDSubstr(...)</code>	String	substring (language-specific)

COMMAND EXECUTION

FUNCTION	RETURNS	DESCRIPTION
<code>SDExecute(cmd, err)</code>	String	execute a TCL command

FUNCTION	RETURNS	DESCRIPTION
<code>SDRespond(response, err)</code>	String	respond to a prompt from <code>SDExecute</code>
<code>SDEndCommand()</code>	none	end a multi-line command
<code>SDCall(name, argcount, ...)</code>	none	call a catalogued subroutine (pass by reference)
<code>SDCallx(name, argcount, ...)</code>	Integer	call a catalogued subroutine (pass by value)

`SDCall` and `SDCallx` are variadic: 0 to 20 arguments. Each binding provides per-arity wrappers — `SDCall0` through `SDCall20`, and `SDCallx0` through `SDCallx20`.

SELECT LISTS

FUNCTION	RETURNS	DESCRIPTION
<code>SDSelect(fileNo, listNo)</code>	none	select all records in a file
<code>SDSelectIndex(fileNo, indexName, indexValue, listNo)</code>	none	select by index value
<code>SDSelectLeft(fileNo, indexName, listNo)</code>	String	select left of an index cursor
<code>SDSelectRight(fileNo, indexName, listNo)</code>	String	select right of an index cursor
<code>SDSetLeft(fileNo, indexName)</code>	none	position cursor left
<code>SDSetRight(fileNo, indexName)</code>	none	position cursor right
<code>SDReadNext(listNo, err)</code>	String	read next select list id
<code>SDReadList(listNo, err)</code>	String	read the whole list as a dynamic array
<code>SDClearSelect(listNo)</code>	none	clear a select list
<code>SDRelease(fileNo, id)</code>	none	release a read lock

SESSION AND PACKAGE MANAGEMENT

FUNCTION	RETURNS	DESCRIPTION
<code>SDEnterPackage(name)</code>	Boolean	enter a package context

FUNCTION	RETURNS	DESCRIPTION
<code>SDExitPackage(name)</code>	Boolean	exit a package context
<code>SDGetArg(argNo)</code>	String	get a command argument

ERROR HANDLING

FUNCTION	RETURNS	DESCRIPTION
<code>SDError()</code>	String	get the last error message
<code>SDDebug(mode)</code>	none	enable/disable debug mode
<code>SDStatus()</code>	Long	get the last server status code
<code>SDFree(ptr)</code>	none	free a returned pointer

Language bindings

The API headers define bindings for four languages. The function signatures are identical in meaning; the syntax differs.

GAMBAS3

```
Library "./sdclilib"
Extern SDConnect(Host As String, Port As Integer, UserName As String,
    Password As String, Account As String) As Boolean
Extern SDRead(FileNo As Integer, Id As String,
    ByRef Errno As Integer) As String
```

PUREBASIC

```
PrototypeC.l P_SDConnect(host.p-utf8, port.l, user.p-utf8,
    pass.p-utf8, account.p-utf8)
PrototypeC.i P_SDRead(fno.l, id.p-utf8, *err)
```

Per-arity prototypes are provided for `SDCa11` and `SDCa11x`: `P_SDCa110` through `P_SDCa1120`, `P_SDCa11x0` through `P_SDCa11x20`.

```

type
  TSDConnectFn = function(Host: PAnsiChar; Port: LongInt;
    User, Pass, Account: PAnsiChar): LongInt; cdecl;
  TSDReadFn = function(FileNo: LongInt; Id: PAnsiChar;
    Err: PLongInt): PAnsiChar; cdecl;

```

A `LoadSdCliLib` function loads the library and resolves all function pointers; `UnloadSdCliLib` releases it.

PYTHON (CTYPES)

```

import ctypes
_lib = ctypes.CDLL('./sdclilib.dll')

SDConnect = _lib.SDConnect
SDConnect.argtypes = [ctypes.c_char_p, ctypes.c_int,
    ctypes.c_char_p, ctypes.c_char_p, ctypes.c_char_p]
SDConnect.restype = ctypes.c_int

SDRead = _lib.SDRead
SDRead.argtypes = [ctypes.c_int, ctypes.c_char_p,
    ctypes.POINTER(ctypes.c_int)]
SDRead.restype = ctypes.c_char_p

```

Per-arity `CFUNCTYPE` definitions are provided for `SDCall` and `SDCallx` wrappers.

What the API cannot do

<code>sh</code> and <code>OS.EXECUTE</code>	refused over the API
Open files outside the account	refused (status 3035 — <i>not permitted</i>)
Reach the credential file	never, and cannot be added
Enumerate accounts	refused; all three failure cases give the same message

SD Glossary

Terms used in SD Core documentation, from account to VOC.

This glossary defines the terms used throughout the SD Core for Windows documentation set. Terms are listed in alphabetical order.

A

Account — a workspace: a directory containing one or more files, a VOC, and its own `bp` source file. An account maps to a Windows group (`sdu_<name>` for user accounts, `sdg_<name>` for group accounts). Entry to an account is membership of its Windows group.

Administrator — the highest account tier. An administrator account is a member of the Windows `Administrators` group and receives the full VOC: 416 records. Administration is gated on elevation, not on a password.

Alternate key index — a secondary access path to records in a file, built from the values in a nominated field. Created with `create.index`, built with `build.index`.

Association — a group of dictionary fields that share multivalues. When fields are associated, a value mark in one corresponds to a value mark in all of them at the same position.

Attribute — a field in a dictionary record that defines one column of a query report. Dictionary field 1 is the name, field 2 is the type, field 3 is the conversion, field 4 is the correlative, field 5 is the format, field 6 is the header, field 7 is the association.

B

Background process — a process started with `phantom` that runs without a terminal. The process that starts it does not wait for it.

Break key — the interrupt key (Ctrl+C on Windows). Stops a running program or query and enters the debugger if one is active.

BASIC — the programming language. See SDBasic.

bp — the source file. An account's `bp` file is a directory file — an ordinary Windows folder with one file per program. `gp1.bp` is the shipped source for the system programs.

C

Catalogue — the registry of compiled programs. A catalogued program can be run by name from any account that can reach it. Private catalogue is per-account; local catalogue puts the VOC entry in the account's own VOC; global catalogue is system-wide and requires an administrator.

Common block — a named memory area shared between subroutines. Declared with `common` or `common /name/`. Used for passing data without arguments.

Conversion code — a dictionary field 3 entry that transforms data on input (`iconv`) or output (`oconv`). Examples: `D` for dates, `MR` for masked decimal, `MC` for case conversion.

Correlative — a dictionary field 4 entry that defines pre-processing of a field value before the conversion is applied. Examples: `A` (arithmetic), `F` (function), `I` (index).

D

Dictionary — a file that defines the fields of a data file for the query processor. Each record in a dictionary file describes one field. The dictionary file is named `<filename>.dict` and lives beside the data file.

Directory file — a file type where each record is a file on disk, stored in an ordinary Windows folder. Record ids are file names. The `bp` file is a directory file.

Dynamic array — a string containing field marks, value marks and subvalue marks. SDBasic's primary data structure for record manipulation. Functions: `extract`, `ins`, `del`, `replace`, `dcount`, `locate`.

Dynamic file — a file type stored in SD's own binary format, with a hash-based group structure and automatic resizing. Not readable by ordinary Windows programs.

F

Field mark — the delimiter (char 254) that separates fields in a dynamic array.

File — a collection of records, each identified by a record id. A file has a data portion and a dictionary portion. The two file types are directory files and dynamic files.

Format — dictionary field 5. Controls how a value is displayed: width, alignment, masking. The format specification syntax supports date masks, numeric masks, and text formatting.

F-pointer — a VOC entry of type `F` that points to another file, possibly in another account. A remote file pointer.

G

Group — (1) a Windows group used for account membership. (2) In a dynamic file, the bucket that holds records hashed to the same slot.

Group account — a shared workspace with no Windows account and no sign-in of its own. Created with `create.account group`. Reached with `logto` or through an F-pointer.

I

I-type — a dictionary field type whose value is computed by an SDBasic expression rather than stored in the record. The expression is compiled and stored in field 2 of the dictionary record.

Index — see Alternate key index.

L

List — see Select list.

Lock — a claim on a record or file that prevents other sessions from updating it simultaneously. Update locks (`readu`, `writetu`, `deletetu`) are exclusive; shared locks (`readl`) are not. Task locks are per-session and never block the same session.

M

Mark mapping — the translation of dynamic array marks to and from the separators used by sequential file I/O. Controlled per file with `SDMarkMapping` in the API, or by the `setmark` and `setemark` statements in SDBasic.

Matrix — a two-dimensional array in SDBasic. Declared with `dimension` or `dim`. Indexed from 1.

Multivalue — a value within a field, separated by a value mark (char 253). A field can hold any number of multivalues; each multivalue can hold any number of subvalues.

O

Overflow — in a dynamic file, the condition where a group has more data than fits in its primary bucket and additional buckets are chained to it. Excessive overflow degrades performance and is reported by `analyse.file`.

P

p-code — the compiled bytecode that SD executes. Stored in the pcode library in `<sysdir>\bin` and loaded into shared memory at start-up.

Paragraph — a VOC entry of type `PA` that holds a sequence of TCL commands. Replaces the removed PROC language.

Phantom — see Background process.

Programmer — the middle account tier. Gets $82 + 42 = 124$ verbs: everything a standard account has, plus the development set (compile, catalogue, edit, file creation, index management, bulk record editing, process introspection).

Q

QMSYS — what the system account is called in OpenQM. In SD it is `SDSYS`, and `sdsys` on disk. Nothing in SD Core answers to the name QMSYS; it is here so that a reader coming from OpenQM finds the entry.

Query processor — the reporting language: `list`, `select`, `count`, `sort`, `sum`, and the rest. Reads dictionary definitions to format output. Invoked from TCL or from SDBasic with `select`, `readnext`, `getlist`.

R

Record — a single entry in a file, identified by a record id. A record is a dynamic array: a string of fields, values and subvalues.

Record id — the unique key of a record within a file. In a directory file, the record id is the file name.

S

SDBasic — the programming language of SD. A compiled BASIC with dynamic arrays, multivalue string functions, file I/O, select lists, sockets, sequential files, CSV handling, and transaction support.

Select list — an ordered list of record ids, built by `select`, `sselect`, `getlist`, or the API's `SDSelect`. Read with `readnext` or `SDReadNext`. Stored in the `$savedlists` file.

Sentence — a VOC entry of type `S` that holds a single TCL command with substitution parameters.

Session — one connection to SD, from sign-in to `off`. Each session runs as the invoking user's Windows identity.

Standard — the lowest account tier. Gets 82 verbs: enough to run an application and nothing that edits code or data in bulk. The default when no tier keyword is given on `create.account`.

Subvalue mark — the delimiter (char 252) that separates subvalues within a multivalue.

Suspended — a fourth account tier that denies all entry. Reversible with `modify.account <tier>`. Does not touch the VOC or Windows group membership.

T

TCL — the command processor. Reads what you type at the `:` prompt and dispatches it.

Terminfo — the terminal capability database. Defines what sequences a terminal sends for keys and what sequences to use for screen control. SD ships 63 definitions compiling to 100 terminal names. The terminfo compiler (`sdtic`) is not shipped with SD Core.

Transaction — a group of file updates that succeed or fail together. Declared with `start transaction` and committed with `commit` or rolled back with `rollback`.

V

Value mark — the delimiter (char 253) that separates multivalues within a field.

VOC — the Vocabulary file. Maps command names to the programs behind them. Each account has its own VOC. VOC entries are of types `V` (verb), `F` (file pointer), `PA` (paragraph), `S` (sentence), `K` (key), and `W` (external command).

W

Windows service — how SD runs on Windows. The service **String Database (SD)** is created by the installer, starts automatically at boot, and is removed by the uninstaller. Stopping it ends every session on the machine.

SD Terminal Information (Terminfo)

The terminfo database, the 63 definitions that ship, and the compiler that ships with them.

SD uses a terminfo database to know what byte sequences a terminal sends for each key, and what sequences to emit for screen control — cursor movement, clear, colour, bold, underline.

Most installations never touch it. The 63 shipped definitions cover the terminals in ordinary use, and `windows` works with any Windows console, cmd, PowerShell or Windows Terminal session.

The default terminal

```
term
```

```
Page width: 120
Page depth: 36
Device      : windows
```

The default terminal type is `windows`. This is an exact copy of the `linux` definition, which had the right byte sequences for Windows consoles, cmd, PowerShell and Windows Terminal.

Existing accounts keep their old setting until their VOC is updated. Until then, `term windows` sets it for the session.

What ships

63 terminfo definitions ship with SD, compiling to **100 terminal names** — the extra names are variants. For example, `vt100-w` and `vt220-at` are aliases that map to the same definition as their base name.

Common names that work out of the box: `windows`, `linux`, `vt100`, `vt220`, `wyse60`, `ansi`, `xterm`.

A name that is not installed is refused and your current type is kept:

```
:term vt320
Unrecognised terminal name
```

There is no plain `vt320` — the shipped name is `vt320-at`.

Setting the terminal

```
term          * report current type and page size
term windows  * set for this session
term 120,36    * set page size
term default  * restore 120 x 36 (prints nothing; check with bare term)
```

The page size is worked out at login in this order:

1. the `LINES` and `COLUMNS` environment variables if they are numeric
2. the terminfo entry's `lines` and `cols`
3. otherwise 36 and 120

A console or ssh session normally gets its real window size. 120 × 36 is the fallback when nothing answers — a phantom or a piped script.

The page is 120 × 36

SD's default is **120 columns by 36 lines**, not 80 × 24. The shipped dictionary records and the default `list` report layouts are formatted for 120 columns. A console window narrower than that makes ordinary reports look wrapped or truncated — which reads as a formatting bug and is not one.

The terminfo source

`terminfo.src` ships with SD. It is a text file containing all 63 definitions in source form. You can read it to see what a terminal definition looks like, and you can use it as a template for a custom definition.

Compiling a definition

The terminfo compiler ships with SD. `sdtic.exe` is in `C:\Program Files\SD\usr\bin`, beside `sd.exe`. Nothing needs building and nothing needs downloading.

```
sdtic {options} src...
```

<code>-d</code>	decompile a named terminal entry instead of compiling
<code>-i</code>	display an index of terminal names
<code>-p</code> <i>path</i>	use the terminfo library at <i>path</i>
<code>-t</code> <i>name</i>	compile only <i>name</i> ; may be repeated
<code>-v</code>	verbose
<code>-x</code>	do not overwrite entries that already exist

Without `-d` it compiles the source files you name. With `-d`, the argument is a terminal name to decompile rather than a file.

Two things follow from `-d` that are worth knowing. It is how you read a shipped definition without hunting through `terminfo.src`, and it is how you produce a starting point for a variant of one:

```
sdtic -d vt220 > myterm.src
```

Edit that, then compile it back in. `-x` is the safe form while you are experimenting, because it refuses to overwrite a definition that already exists.

The compiled definitions live under `C:\ProgramData\SD\sdsys\terminfo`, in single-letter directories by first letter. Writing there needs administrator rights, so run `sdtic` from an elevated prompt when you are adding a definition for the machine.

SD Programming 101

A tutorial with worked example programs - lessons 1 to 6, from hello world to reading a file with a select list.

This page is a tutorial. The rest of the `User` set is reference: each page covers one subject and every statement on it. Here we put them together into programs that do something, and each program builds on the one before.

Every program on this page compiles and runs on SD Core for Windows W1.0-0. The output shown is what SD printed. If you type the program in, compile it and run it, you will see the same thing.

The reference pages are linked at the end of each section. Read them when you want the full story on a statement or function — this page shows what you need to make the programs work and no more.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part.

Before you start

You need an SD session and a `bp` file. If you have signed in and created a file called `customers` as the Introduction suggests, you already have `bp` — every account gets one.

```
sd
```

You land at the `:` prompt. That is TCL — the command processor. From here you compile, catalogue and run programs. The programs themselves are written in SDBasic, which is a different language from TCL.

Your `bp` file is an ordinary Windows folder. You can write programs in `ed` from inside SD, in `edit` or `micro` if your account can run external programs, or in any text editor you like — Notepad, VS Code, anything. The folder is on disk at:

```
C:\ProgramData\SD\user_accounts\<account>\bp
```

1. Hello world

The smallest program:

```
program hello
  print 'hello, world'
end
```

Type it into `ed` — `ed bp hello`, `i`, type the three lines, a full stop on its own line to stop inserting, `fi` to file and exit. Or save it as `hello` in the `bp` folder with a text editor.

Compile and run it:

```
basic bp hello
run bp hello
```

```
hello, world
```

Every program starts with a declaration — `program` here — and ends with `end`. Between them are statements. `print` sends a line to the terminal. The string is in single quotes; double quotes work too, but single quotes are what the rest of the documentation uses and what SD's own source uses.

`program` **is optional**. A source record with no declaration is a program. But every program on this page has one, because naming the program is how you know what it is when it is catalogued and called from another program.

2. Variables and input

A program that asks your name and says hello back:

```
program hello2
  print 'what is your name?' :
  input name
  print 'hello, ' : name
end
```

```
basic bp hello2
run bp hello2
```

```
what is your name? dave
hello, dave
```

`input` reads a line from the terminal into the variable `name`. The colon at the end of the first `print` stops it printing a newline, so the question mark is on the same line as the answer.

Variables are not declared. A variable that has never been assigned is an **unassigned variable** — reading it is an error, not an empty string. This is deliberate: it catches typos.

The colon `:` in a `print` statement is the concatenation operator for output. It does not add spaces — it joins the pieces with nothing between them. To put a space between *hello*, and the name, the string itself carries the space: `'hello, '`.

3. Variables and arithmetic

SDBasic variables are **type variant** — a variable holds whatever type the last assignment gave it, and the type can change from one line to the next. A variable that held a string can be used in arithmetic on the next line: the value is converted to numeric form for the calculation without changing the variable itself.

Numeric values are held as **integers wherever possible**. Conversion to floating point occurs when a result is non-integer or too large to store as an integer. This is invisible — you do not declare a variable as integer or real.

```
program maths
  print 'first number?' :
  input a
  print 'second number?' :
  input b
  print 'sum      = ' : a + b
  print 'product = ' : a * b
  print 'quotient = ' : a / b
  print 'remainder= ' : mod(a, b)
end
```

```
run bp maths
```

```
first number? 17
second number? 5
sum      = 22
product  = 85
```

```
quotient = 3.4
remainder= 2
```

`mod()` is a function, not a statement — it is called inside an expression and returns a value. SDBasic has a full set of math functions: `abs`, `int`, `sqrt`, `sin`, `cos`, `tan`, `ln`, `exp`, `pwr`, and more. The full list is in SD Basic - Math Functions.

The redundant-looking type-forcing idiom. Because conversion happens on use, not on assignment, a string of digits used in a tight loop converts on every iteration. SDBasic programs often contain apparently redundant statements that force the conversion once:

```
a = a + 0 ;* Convert to numeric form s = s : "" ;* Convert to string form
```

The first makes `a` numeric so the next arithmetic use does not convert again; the second makes `s` a string. Neither changes the value — they change how it is held.

4. A program that reads a file

This is where SDBasic starts to earn its keep. The `customers` file from the Introduction holds records. Each record is a dynamic array — a string with field marks in it. Reading a record gives you the whole string; extracting a field gives you one piece of it.

```
program show.customer
  open 'customers' to f.cust else
    print 'cannot open customers'
    stop
  end

  print 'customer id?' :
  input id

  read rec from f.cust, id then
    print 'name   : ' : rec<1>
    print 'phone : ' : rec<2>
    print 'city  : ' : rec<3>
  end else
    print 'no such customer'
  end
end
```

```
run bp show.customer
```

```
customer id? 1001
name  : Acme Supplies
phone : 555-1234
city  : Springfield
```

WHAT JUST HAPPENED

`open` looks up `customers` in the VOC and gives you a file variable. The `else` branch fires if the file does not exist — and **it is compulsory**. A program that omits the `else` does not compile.

`read` fetches the whole record by id. The `then` branch fires if the record exists; the `else` branch fires if it does not.

`rec<1>` is **field extraction** — the angle brackets are the shorthand for `extract(rec, 1, 0, 0)`. Field 1, field 2, field 3. Dynamic arrays are covered in SD Basic - Dynamic Arrays.

5. A program that writes

```
program add.customer
  open 'customers' to f.cust else
    print 'cannot open customers'
    stop
  end

  print 'new customer id?' :
  input id

  print 'name?' :
  input name
  print 'phone?' :
  input phone
  print 'city?' :
  input city

  rec = name : @fm : phone : @fm : city
  write rec on f.cust, id
```

```
print 'saved'
end
```

```
run bp add.customer
```

```
new customer id? 1002
name? Widget Corp
phone? 555-5678
city? Shelbyville
saved
```

`@fm` is the **field mark constant** — character 254. Building a record is joining fields with it. `write` stores the whole record; the old record, if any, is replaced.

THE MARKS

Three mark characters divide a dynamic array:

	NAME	CONSTANT	CODE
field mark	<code>@fm</code>	254	
value mark	<code>@vm</code>	253	
subvalue mark	<code>@sm</code>	252	

A field contains values; a value contains subvalues. `rec<1>` gets the first field; `rec<1,2>` gets the second value in the first field; `rec<1,2,1>` gets the first subvalue in that value.

6. A program with a select list

Reading one record by id is useful. Reading every record in a file is how you produce reports. A select list is the mechanism:

```
program list.customers
  open 'customers' to f.cust else
    print 'cannot open customers'
  stop
end
```

```

select f.cust to 1
print 'id' : ' ' : 'name' : ' ' : 'city'
print '---' : ' ' : '----' : ' ' : '----'

loop
  readnext id from 1 else exit
  read rec from f.cust, id then
    print id : ' ' : rec<1> : ' ' : rec<3>
  end
repeat

clearselect 1
end

```

```
run bp list.customers
```

```

id name city
--- ----
1001 Acme Supplies Springfield
1002 Widget Corp Shelbyville

```

`select` builds a list of every record id in the file. `readnext` reads one id at a time from the list.

`readnext` is a **statement, not a function**, and its `else` branch is what ends the loop — it fires when the list is exhausted. `loop` / `readnext ... else exit` / `repeat` is the standard pattern for walking a select list. Writing it as `readnext(id)` does not fail helpfully: the compiler reads an unknown function call as a matrix reference and complains that `READNEXT` is not referenced in a `dim` statement.

`clearselect 1` clears list 1 when the program is done. A select list is **session state**: it survives the program that made it. Clearing it is good manners.

Continued in

SD Programming 101 - Routines and an Application — lessons 7 to 10 — subroutines, functions, error handling and a small application.

SD Programming 101 - Routines and an Application

Lessons 7 to 10: subroutines, functions, error handling, and a small application put together.

This page continues SD Programming 101.

7. A subroutine

A subroutine is a separate program, called by name, that takes arguments and returns values through them. You write it as its own source record and compile it:

```
subroutine get.field(result, file.var, id, field.no)
  read rec from file.var, id then
    result = rec<field.no>
  end else
    result = ''
  end
end
```

```
basic bp get.field
```

Notice: a subroutine starts with `subroutine` where a program starts with `program`. It does not `run` — you call it from another program.

```
program test.get.field
  open 'customers' to f.cust else stop 'no file'
  call get.field(name, f.cust, '1001', 1)
  print 'name is ' : name

  call get.field(phone, f.cust, '1001', 2)
  print 'phone is ' : phone
end
```

```
basic bp test.get.field
run bp test.get.field
```

```
name is Acme Supplies
phone is 555-1234
```

`call` runs the subroutine. Arguments are passed **by reference**: the subroutine sets `result` and the caller sees it in `name`. This is how subroutines return values — there is no `return value` statement for subroutines.

A subroutine must be compiled before the program that calls it. If `get.field` is not compiled, the calling program gets *Subroutine not found* at run time, not at compile time.

8. A function

A function is like a subroutine, but it returns a value in an expression rather than through an argument:

```
function fmt.phone(number)
  if len(number) = 7 then
    fmt.phone = number[1,3] : '-' : number[4,3]
  end else
    fmt.phone = number
  end
end
```

```
basic bp fmt.phone
```

A function assigns its return value to **its own name**. That is the `fmt.phone = ...` line — it is not a variable called `fmt.phone`, it is the function saying what to return.

A caller has to declare it with `deffun` before using it. Without that line, the compiler has no way to know `fmt.phone(raw)` is a call rather than a matrix reference, and says so in those terms: *Matrix FMT.PHONE is not referenced in a DIM statement*. Subroutines called with `call` need no declaration; functions do.

```
program test.fmt.phone
  deffun fmt.phone(number) calling 'FMT.PHONE'
  open 'customers' to f.cust else stop 'no file'
  read rec from f.cust, '1001' then
```

```

    raw = rec<2>
    print 'raw      : ' : raw
    print 'formatted: ' : fmt.phone(raw)
end
end

```

```

basic bp test.fmt.phone
run bp test.fmt.phone

```

```

raw      : 5551234
formatted: 555-1234

```

`number[1,3]` is a **substring**: start at position 1, take 3 characters. `number[4,3]` starts at position 4 and takes 3 more. The square brackets are the shorthand for the `substr()` function.

9. Error handling

File operations can fail. The `on error` clause catches failures that are not about the record being missing — disk full, file locked by another session, network error:

```

program safe.write
  open 'customers' to f.cust else stop 'no file'

  print 'id?' :
  input id
  print 'name?' :
  input name

  rec = name : @fm : ' ' : @fm : ' '

  write rec on f.cust, id on error
    print 'write failed, status ' : status()
    stop
  end
  print 'saved'
end

```

```
basic bp safe.write
run bp safe.write
```

```
id? 1003
name? Test Co
saved
```

The `on error` clause fires when the write fails for a system reason. The `then / else` on `write` is about the record — but `write` has no `then / else`, because writing always succeeds unless something is broken. The `on error` is where that broken-something arrives.

`status()` **carries the code**, and it is what to read inside an `on error` block. Read it on the line you care about rather than three lines later: the next operation that sets a status overwrites it.

There is no system variable holding the error *text*. To turn a code into a sentence, call the catalogued subroutine `!ERRTEXT`:

```
call "!ERRTEXT", message, status()
print 'write failed: ' : message
```

`@system.return.code` is a different thing and is worth not confusing with this one: it carries the result of the last `execute`d command, not of the last file operation.

10. Putting it together: a small application

This program ties everything together. It is a menu-driven customer manager: list, add, view and delete. It uses subroutines, select lists, file I/O, error handling, and a loop.

```
program customer.manager
  open 'customers' to f.cust else
    print 'cannot open customers - create it first'
    stop
  end

  loop
    print
    print 'l)list a)dd v)iew d)elele q)uit'
    print 'choice?' :
    input choice
```

```

begin case
    case choice = 'l'
        gosub list.all

    case choice = 'a'
        gosub add.one

    case choice = 'v'
        gosub view.one

    case choice = 'd'
        gosub delete.one

    case choice = 'q'
        exit

    case 1
        print 'not a choice'
end case
repeat

print 'bye'
stop

* ---- list ----
list.all:
    select f.cust to 2
    cnt = 0

    print 'id' : ' ' : 'name' : ' ' : 'city'
    print '---' : ' ' : '----' : ' ' : '----'

    loop
        readnext id from 2 else exit
        read rec from f.cust, id then
            print id : ' ' : rec<1> : ' ' : rec<3>
            cnt += 1
        end
    repeat

    clearselect 2
    print cnt : ' record(s)'
    return

```

```

* ---- add ----
add.one:
    print 'new id?' :
    input id
    print 'name?' :
    input name
    print 'phone?' :
    input phone
    print 'city?' :
    input city

    rec = name : @fm : phone : @fm : city
    write rec on f.cust, id on error
        print 'write failed, status ' : status()
        return
    end
    print 'saved'
    return

* ---- view ----
view.one:
    print 'id?' :
    input id
    read rec from f.cust, id then
        print 'name : ' : rec<1>
        print 'phone : ' : rec<2>
        print 'city : ' : rec<3>
    end else
        print 'not found'
    end
    return

* ---- delete ----
delete.one:
    print 'id?' :
    input id
    print 'are you sure? (y/n)' :
    input confirm
    if confirm # 'y' then
        print 'cancelled'
        return
    end

    delete f.cust, id on error
        print 'delete failed, status ' : status()

```

```

        return
    end
    print 'deleted'
    return
end

```

```

basic bp customer.manager
run bp customer.manager

```

```

l)ist a)dd v)iew d)elele q)uit
choice? l

id name city
--- ---- ----
1001 Acme Supplies Springfield
1002 Widget Corp Shelbyville
2 record(s)

l)ist a)dd v)iew d)elele q)uit
choice? v
id? 1001
name  : Acme Supplies
phone : 555-1234
city  : Springfield

l)ist a)dd v)iew d)elele q)uit
choice? q
bye

```

WHAT IS IN THIS PROGRAM

FEATURE	WHERE
<code>loop ... repeat</code>	the main menu loop
<code>begin case</code> / <code>case</code> / <code>end case</code>	dispatching on the menu choice
<code>gosub</code> / <code>return</code>	internal subroutines — labels inside the program
<code>select</code> / <code>readnext</code> / <code>clearselect</code>	walking the file for the list
<code>read</code> / <code>then</code> / <code>else</code>	reading a record

FEATURE	WHERE
<code>write</code> / <code>on error</code>	writing a record safely
<code>delete</code> / <code>on error</code>	deleting a record
<code>@fm</code>	building a dynamic array
<code>rec<1></code>	extracting fields
<code>exit</code>	leaving the main loop
<code>#</code>	not-equal comparison
<code>+=</code>	increment

The `gosub` subroutines are **internal** — they are labels in the same program, not separate programs. They share every variable in the program. For subroutines with their own scope, see *SD Basic - Modern Program Structure*.

Cataloguing

Running a program with `run bp name` works, but typing `run bp` every time is tedious. **Cataloguing** gives a program a name you can type directly:

```
basic bp customer.manager
catalog bp customer.manager
```

```
customer.manager
```

```
l)ist a)dd v)iew d)elele q)uit
choice?
```

A catalogued program can be called from any account that can reach it. The catalogue puts an entry in the VOC — the same file that maps `list`, `select` and the built-in verbs. See *SD TCL - Programs and the Catalogue*.

Where to go next

Each section above linked to a reference page. Here they are together, in the order the tutorial uses them:

TOPIC	REFERENCE PAGE
Program structure	SD Basic - Program Structure
Variables, loops, conditions	SD Basic - Program Control
Math functions	SD Basic - Math Functions
String functions	SD Basic - String Functions
Dynamic arrays	SD Basic - Dynamic Arrays
Data conversion	SD Basic - Data Conversion
File handling	SD Basic - File Handling
Select lists	SD Basic - Select Lists
Locks and transactions	SD Basic - Locks and Transactions
Modern program structure	SD Basic - Modern Program Structure
Compiling and cataloguing	SD TCL - Programs and the Catalogue
File system concepts	SD File System Concepts
Standard subroutines	SD Standard Subroutines

The syntax cards at the end of the set list every statement and function alphabetically:

SD Basic - Syntax	every SDBasic name, with its syntax
SD TCL - Syntax	every TCL verb, with its syntax

SD Basic - Syntax

Every statement and function, alphabetically, with its syntax and nothing else.

A lookup card, and nothing else. If you know what you want and have forgotten how to spell it, it is here. If you want to know what it *does*, the other seventeen documents in this set are where that lives.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part. SD folds case, so any of this may be written in either case.

This page is generated, and it is checked for completeness rather than proof-read for it. The roster comes from `BCOMP`'s own tables — the same extraction the rest of this set uses — and `tools/mksyntax.py` refuses to write the page if a single name that belongs on it has no line. Argument counts for functions are read out of `BCOMP`'s dispatch table, which is positional against the name list and carries each name in a comment; the script asserts the two agree before it uses either. The shapes that a count cannot express — every statement, and about twenty functions — come from documents 01 to 17, where each one is described in full.

What is not here, and where it went. Everything on this card is something an application may use. Names that an ordinary program **cannot** compile are in the administrator documentation, under *SD Basic - Restricted Commands*: the restricted statements, the internal-only functions, and the one name that is in the compiler's table with nothing behind it. **If you are looking for something and it is not here, that is where to look before concluding it does not exist.**

ONE THING IS MARKED, AND IT IS THE ONE THAT WASTES TIME.

(clause word) not a statement — a word that belongs inside another one's syntax

A

<code>abort</code>	<code>abort {message}</code>
--------------------	------------------------------

<code>abs</code>	<code>abs(a)</code>
------------------	-----------------------

<code>abss</code>	<code>abss(a)</code>
-------------------	------------------------

<code>accept.socket.connection</code>	<code>accept.socket.connection(<i>a</i>, <i>b</i>)</code>
<code>acos</code>	<code>acos(<i>a</i>)</code>
<code>alpha</code>	<code>alpha(<i>a</i>)</code>
<code>ands</code>	<code>ands(<i>a</i>, <i>b</i>)</code>
<code>append</code>	clause of <code>openseq</code> , and of <code>edit</code> (clause word)
<code>arg</code>	<code>arg(<i>a</i>)</code>
<code>arg.count</code>	<code>arg.count()</code>
<code>ascii</code>	<code>ascii(<i>a</i>)</code>
<code>asin</code>	<code>asin(<i>a</i>)</code>
<code>assigned</code>	<code>assigned(<i>a</i>)</code>
<code>atan</code>	<code>atan(<i>a</i>)</code>

B

<code>before</code>	clause of <code>ins</code> : <code>ins value</code> before <code>arr</code> <code>< f ></code> (clause word)
<code>begin</code>	<code>begin case ... end case</code> · <code>begin transaction ... end transaction</code>
<code>bindkey</code>	<code>bindkey(<i>a</i>, <i>b</i>)</code>
<code>bitand</code>	<code>bitand(<i>a</i>, <i>b</i>)</code>
<code>bitnot</code>	<code>bitnot(<i>a</i>)</code>
<code>bitor</code>	<code>bitor(<i>a</i>, <i>b</i>)</code>
<code>bitreset</code>	<code>bitreset(<i>a</i>, <i>b</i>)</code>
<code>bitset</code>	<code>bitset(<i>a</i>, <i>b</i>)</code>
<code>bittest</code>	<code>bittest(<i>a</i>, <i>b</i>)</code>

bitxor	<code>bitxor(<i>a</i>, <i>b</i>)</code>
break	<code>break on</code> · <code>break off</code> · <code>break</code> <i>n</i>
by	clause of <code>locate</code> and <code>sselect</code> — <code>by</code> 'AL', 'AR', 'DL', 'DR' (clause word)

C

call	<code>call</code> <i>name</i> (<i>arg</i> , ...) · <code>call @</code> <i>var</i> (<i>arg</i> , ...)
capturing	clause of <code>execute</code> and <code>os.execute</code> — take the output (clause word)
case	<code>case</code> <i>condition</i> , inside <code>begin case</code> ... <code>end case</code>
catalogued	<code>catalogued(<i>a</i>)</code>
cats	<code>cats(<i>a</i>, <i>b</i>)</code>
ccall	<code>ccall(<i>a</i>, <i>b</i>)</code>
chain	<code>chain</code> <i>command</i>
change	<code>change(<i>string</i>, <i>from</i>, <i>to</i> {, <i>occurrences</i> {, <i>start</i>} })</code>
char	<code>char(<i>a</i>)</code>
checksum	<code>checksum(<i>a</i>)</code>
chgphant	<code>chgphant()</code>
class	<code>class</code> <i>name</i>
clear	<code>clear</code> — zeroes every variable
clearcommon	<code>clearcommon</code> { <i>name</i> }
cleardata	<code>cleardata</code>
clearfile	<code>clearfile</code> <i>file.var</i> { <code>on error</code> ... }
clearinput	<code>clearinput</code>

<code>clearselect</code>	<code>clearselect</code> { <i>list.no</i> }
<code>close</code>	<code>close</code> <i>file.var</i> { <code>on error</code> ...}
<code>close.socket</code>	<code>close.socket</code> <i>socket</i>
<code>closeseq</code>	<code>closeseq</code> <i>file.var</i>
<code>col1</code>	<code>col1()</code>
<code>col2</code>	<code>col2()</code>
<code>com</code>	<code>com</code> / <i>name</i> / <i>var</i> , ... — synonym of <code>common</code>
<code>commit</code>	<code>commit</code> , inside <code>begin transaction</code> ... <code>end transaction</code>
<code>common</code>	<code>common</code> / <i>name</i> / <i>var</i> , <code>matrix</code> (<i>n</i>), ...
<code>compare</code>	<code>compare</code> (<i>a</i> , <i>b</i> { <i>justification</i> })
<code>config</code>	<code>config</code> (<i>a</i>)
<code>continue</code>	<code>continue</code> — next iteration of the enclosing <code>loop</code> or <code>for</code>
<code>convert</code>	<code>convert</code> (<i>a</i> , <i>b</i> , <i>c</i>)
<code>cos</code>	<code>cos</code> (<i>a</i>)
<code>count</code>	<code>count</code> (<i>a</i> , <i>b</i>)
<code>counts</code>	<code>counts</code> (<i>a</i> , <i>b</i>)
<code>create</code>	<code>create</code> <i>file.var</i> <code>then</code> ... <code>else</code> ...
<code>create.file</code>	<code>create.file</code> <i>name</i> { <code>directory</code> } { <code>on error</code> ...}
<code>create.server.socket</code>	<code>create.server.socket</code> (<i>addr</i> , <i>port</i> { <i>flags</i> })
<code>crop</code>	<code>crop</code> (<i>a</i>)
<code>crt</code>	<code>crt</code> <i>expr</i> { <code>:</code> ...} { <code>:</code> }
<code>csvdq</code>	<code>csvdq</code> (<i>line</i> { <i>delimiter</i> })
<code>current.level</code>	clause of <code>enter</code> — run at the current command level (clause word)

D

data	<code>data</code> <i>expr</i> , ...
date	<code>date()</code>
dcount	<code>dcount(<i>a</i>, <i>b</i>)</code>
debug	<code>debug</code> — needs <code>basic</code> <i>file record</i> <code>debugging</code>
deffun	<code>deffun</code> <i>name</i> (<i>arg</i> , ...) { <code>calling</code> "cat.name" } { <code>local</code> }
del	<code>del</code> <i>arr</i> < <i>f</i> { <i>v</i> { <i>sv</i> } } >
delete	<code>delete</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... } · <code>delete(</code> <i>arr</i> , <i>f</i> { <i>v</i> { <i>sv</i> } })
deletelist	<code>deletelist</code> <i>name</i> <code>then</code> ... <code>else</code> ...
deleteseq	<code>deleteseq</code> <i>path</i> { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
deleteu	<code>deleteu</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... }
dim	<code>dim</code> <i>name</i> (<i>rows</i> { <i>cols</i> }) , ...
dimension	<code>dimension</code> <i>name</i> (<i>rows</i> { <i>cols</i> }) , ... — synonym of <code>dim</code>
dir	<code>dir(</code> <i>a</i>)
disinherit	<code>disinherit</code> <i>object</i>
display	<code>display</code> <i>expr</i> { <code>:</code> ... } { <code>:</code> }
div	<code>div(</code> <i>a</i> , <i>b</i>)
do	<code>loop</code> ... <code>do</code> ... <code>repeat</code>
downcase	<code>downcase(</code> <i>a</i>)
dparse	<code>dparse</code> <i>string</i> , <i>delimiter</i> , <i>var</i> , ...
dparse.csv	<code>dparse.csv</code> <i>line</i> , <i>delimiter</i> , <i>var</i> , ...
dquote	<code>dquote(</code> <i>a</i>)
dtx	<code>dtx(</code> <i>n</i> { <i>digits</i> })

E

ebcdic	<code>ebcdic(<i>a</i>)</code>
echo	<code>echo on</code> · <code>echo off</code> · <code>echo</code> <i>expr</i>
edit	clause of <code>keyedit</code> , and the <code>edit</code> verb at the command prompt (clause word)
else	clause of <code>if</code> , and of every statement that has a <code>then</code>
end	<code>end</code> — closes a program, a <code>then</code> , an <code>else</code> , a <code>case</code> , a transaction
enter	<code>enter</code> <i>name</i> · <code>enter @</code> <i>var</i>
env	<code>env(<i>a</i>)</code>
eqs	<code>eqs(<i>a</i>, <i>b</i>)</code>
equ	<code>equ</code> <i>name</i> <code>to</code> <i>value</i> — synonym of <code>equate</code>
equate	<code>equate</code> <i>name</i> <code>to</code> <i>value</i> {, <i>name</i> <code>to</code> <i>value</i> } ...
execute	<code>execute</code> <i>cmd</i> { <code>capturing</code> <i>v</i> } { <code>returning</code> <i>v</i> } { <code>passlist</code> { <i>l</i> } } { <code>rtnlist</code> { <i>v</i> } }
exit	<code>exit</code> — leave the enclosing <code>loop</code> or <code>for</code>
exp	<code>exp(<i>a</i>)</code>
extract	<code>extract(<i>arr</i>, <i>f</i> {, <i>v</i> {, <i>sv</i>} })</code>

F

field	<code>field(<i>string</i>, <i>delimiter</i>, <i>occurrence</i> {, <i>count</i>})</code>
fields	<code>fields(<i>arr</i>, <i>delimiter</i>, <i>occurrence</i> {, <i>count</i>})</code>
fieldstore	<code>fieldstore(<i>a</i>, <i>b</i>, <i>c</i>, <i>d</i>, <i>e</i>)</code>
file	<code>file</code> <i>name</i> — names a file at compile time
fileinfo	<code>fileinfo(<i>file.var</i>, <i>key</i>)</code>

filelock	<code>filelock file.var { locked ... } { on error ... } — no then / else</code>
fileunlock	<code>fileunlock file.var { on error ... }</code>
findstr	<code>findstr x in arr { occ } setting f { v { sv } } then ... else ...</code>
flush	<code>flush file.var then ... else ...</code>
fmt	<code>fmt(a, b)</code>
fmts	<code>fmts(a, b)</code>
fold	<code>fold(string, width {, delimiter})</code>
folds	<code>folds(arr, width {, delimiter})</code>
footing	<code>footing expr</code>
for	<code>for var = start to end { step n } ... next var</code>
formlist	<code>formlist arr {, delimiter} { to list.no }</code>
from	clause of <code>read</code> , <code>readnext</code> , <code>matbuild</code> , <code>matparse</code> , <code>remove</code> , <code>readcsv</code> (clause word)
function	<code>function name (arg, ...)</code>

G

ges	<code>ges(a, b)</code>
get	<code>get var from object</code>
get.messages	<code>get.messages()</code>
get.port.params	<code>get.port.params(a)</code>
getlist	<code>getlist name { to list.no } then ... else — the then / else is not optional</code>
getnls	<code>getnls(a)</code>
getpu	<code>getpu(a, b)</code>

<code>getrem</code>	<code>getrem(<i>a</i>)</code>
---------------------	---------------------------------

<code>go</code>	<code>go</code> <i>label</i> — synonym of <code>goto</code>
-----------------	---

<code>gosub</code>	<code>gosub</code> <i>label</i> { <i>arg</i> , ... }
--------------------	--

<code>goto</code>	<code>goto</code> <i>label</i>
-------------------	--------------------------------

<code>gts</code>	<code>gts(<i>a</i>, <i>b</i>)</code>
------------------	--

H

<code>heading</code>	<code>heading</code> <i>expr</i>
----------------------	----------------------------------

<code>hidden</code>	clause of <code>input</code> — do not echo what is typed (<i>clause word</i>)
---------------------	--

<code>hush</code>	<code>hush on</code> · <code>hush off</code> · <code>hush</code> <i>expr</i>
-------------------	--

I

<code>iconv</code>	<code>iconv(<i>a</i>, <i>b</i>)</code>
--------------------	--

<code>iconvs</code>	<code>iconvs(<i>a</i>, <i>b</i>)</code>
---------------------	---

<code>idiv</code>	<code>idiv(<i>a</i>, <i>b</i>)</code>
-------------------	---

<code>if</code>	<code>if</code> <i>condition</i> <code>then</code> ... <code>else</code> ...
-----------------	--

<code>ifs</code>	<code>ifs(<i>a</i>, <i>b</i>, <i>c</i>)</code>
------------------	--

<code>in</code>	clause of <code>find</code> , <code>findstr</code> and <code>locate</code>
-----------------	--

<code>include</code>	<code>\$include</code> <i>record</i> · <code>\$include</code> <i>file record</i>
----------------------	--

<code>index</code>	<code>index(<i>a</i>, <i>b</i>, <i>c</i>)</code>
--------------------	--

<code>indexs</code>	<code>indexs(<i>a</i>, <i>b</i>, <i>c</i>)</code>
---------------------	---

indices	<code>indices(file.var {, index})</code>
inherit	<code>inherit object</code>
inmat	<code>inmat() · inmat(matrix)</code>
input	<code>input var {, length} { : } { _ } { hidden } { overlay } { upcase } { waiting n }</code>
inputblk	<code>inputblk(a)</code>
inputclear	<code>inputclear</code>
inputcsv	<code>inputcsv var, ...</code>
inputerr	<code>inputerr message</code>
inputfield	<code>inputfield var, length {, mask}</code>
ins	<code>ins value before arr < f {, v {, sv}} ></code>
insert	<code>insert(arr, f {, v {, sv}}, value)</code>
int	<code>int(a)</code>
itype	<code>itype(compiled.i.type)</code>

K

keycode	<code>keycode({timeout})</code>
keyedit	<code>keyedit key, action</code>
keyexit	<code>keyexit key, action</code>
keyin	<code>keyin({timeout})</code>
keyinc	<code>keyinc({timeout})</code>
keyinr	<code>keyinr({timeout})</code>
keyready	<code>keyready()</code>

keytrap	keytrap <i>key, action</i>
---------	----------------------------

L

len	len(<i>a</i>)
-----	-----------------

lens	lens(<i>a</i>)
------	------------------

les	les(<i>a</i> , <i>b</i>)
-----	----------------------------

listindex	listindex(<i>a</i> , <i>b</i> , <i>c</i>)
-----------	---

ln	ln(<i>a</i>)
----	----------------

local	local <i>var</i> , ... — inside a <code>deffun</code> declared <code>local</code>
-------	---

locate	locate <i>x</i> in <i>arr</i> < <i>f</i> { <i>v</i> } > {by <i>order</i> } setting <i>pos</i> then ... else ...
--------	---

lock	lock <i>n</i> then ... else ... — with no else it retries for ever
------	---

locked	clause of <code>readu</code> , <code>readl</code> , <code>readvu</code> , <code>readvl</code> , <code>matreadu</code> , <code>matreadl</code> , <code>filelock</code> , <code>recordlocku</code> , <code>recordlockl</code> (clause word)
--------	--

logmsg	logmsg <i>text</i>
--------	--------------------

loop	loop ... { while <i>c</i> } { until <i>c</i> } ... repeat
------	---

lower	lower(<i>a</i>)
-------	-------------------

lts	lts(<i>a</i> , <i>b</i>)
-----	----------------------------

M

mark.mapping	mark.mapping <i>file.var, state</i>
--------------	-------------------------------------

mat	mat <i>matrix</i> = <i>value</i> · mat <i>a</i> = mat <i>b</i>
-----	--

matbuild	<code>matbuild</code> <i>var</i> <code>from</code> <i>matrix</i> { <i>start</i> { <i>end</i> } } — no <code>using</code>
matchfield	<code>matchfield</code> (<i>a</i> , <i>b</i> , <i>c</i>)
matparse	<code>matparse</code> <i>matrix</i> <code>from</code> <i>string</i> { <i>delimiter</i> }
matread	<code>matread</code> <i>matrix</i> <code>from</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
matreadcsv	<code>matreadcsv</code> <i>matrix</i> <code>from</code> <i>file.var</i> <code>then</code> ... <code>else</code> ...
matreadl	<code>matreadl</code> <i>matrix</i> <code>from</code> <i>file.var</i> , <i>id</i> { <code>locked</code> ... } <code>then</code> ... <code>else</code> ...
matreadu	<code>matreadu</code> <i>matrix</i> <code>from</code> <i>file.var</i> , <i>id</i> { <code>locked</code> ... } <code>then</code> ... <code>else</code> ...
matwrite	<code>matwrite</code> <i>matrix</i> <code>to</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... }
matwriteu	<code>matwriteu</code> <i>matrix</i> <code>to</code> <i>file.var</i> , <i>id</i> { <code>on error</code> ... }
max	<code>max</code> (<i>a</i> , <i>b</i>)
maximum	<code>maximum</code> (<i>a</i>)
min	<code>min</code> (<i>a</i> , <i>b</i>)
minimum	<code>minimum</code> (<i>a</i>)
mod	<code>mod</code> (<i>a</i> , <i>b</i>)
mods	<code>mods</code> (<i>a</i> , <i>b</i>)

N

nap	<code>nap</code> <i>milliseconds</i>
neg	<code>neg</code> (<i>a</i>)
negs	<code>negs</code> (<i>a</i>)
nes	<code>nes</code> (<i>a</i> , <i>b</i>)
next	<code>next</code> <i>var</i> — closes a <code>for</code>

nobuf	<code>nobuf</code> <i>file.var</i> <code>then</code> ... <code>else</code> ...
nocaseinvert	clause of <code>input</code> — do not invert the case of typed letters (<i>clause word</i>)
not	<code>not</code> (<i>a</i>)
nots	<code>nots</code> (<i>a</i>)
null	<code>null</code> — does nothing; the empty statement
num	<code>num</code> (<i>a</i>)
nums	<code>nums</code> (<i>a</i>)

O

object	<code>object</code> (<i>class</i> { <i>arg</i> , ... })
objinfo	<code>objinfo</code> (<i>a</i> , <i>b</i>)
oconv	<code>oconv</code> (<i>a</i> , <i>b</i>)
oconvs	<code>oconvs</code> (<i>a</i> , <i>b</i>)
on	<code>on</code> <i>n</i> <code>goto</code> <i>label</i> , ... — clamps, it does not fall through
open	<code>open</code> { <i>dict</i> , } <i>name</i> <code>to</code> <i>file.var</i> { <code>readonly</code> } { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
open.socket	<code>open.socket</code> (<i>addr</i> , <i>port</i> , <i>flags</i> { <i>context</i> })
openpath	<code>openpath</code> <i>path</i> <code>to</code> <i>file.var</i> { <code>readonly</code> } { <code>on error</code> ... } <code>then</code> ... <code>else</code> ...
openseq	<code>openseq</code> <i>path</i> <code>to</code> <i>file.var</i> { <code>append</code> } { <code>overwrite</code> } { <code>readonly</code> } <code>then</code> ... <code>else</code> ... — <i>then</i> = it existed, <i>else</i> = it was created
ors	<code>ors</code> (<i>a</i> , <i>b</i>)
os.error	<code>os.error</code> ()
os.execute	<code>os.execute</code> <i>command</i> { <code>capturing</code> <i>var</i> }
outerjoin	<code>outerjoin</code> (<i>a</i> , <i>b</i> , <i>c</i>)

<code>overlay</code>	clause of <code>input</code> — type over what is already there (clause word)
----------------------	---

<code>overwrite</code>	clause of <code>openseq</code> (clause word)
------------------------	---

P

<code>page</code>	<code>page {n}</code>
-------------------	-----------------------

<code>panning</code>	clause of <code>input</code> — pan a field wider than the screen (clause word)
----------------------	---

<code>passlist</code>	clause of <code>execute</code> — hand the active select list to the command (clause word)
-----------------------	--

<code>pause</code>	<code>pause {seconds}</code>
--------------------	------------------------------

<code>perform</code>	<code>perform command</code>
----------------------	------------------------------

<code>precision</code>	<code>precision n</code>
------------------------	--------------------------

<code>print</code>	<code>print expr {[: ...] [:]} · print on unit expr</code>
--------------------	--

<code>printcsv</code>	<code>printcsv {on unit} value, ... {[:]}</code>
-----------------------	--

<code>printer</code>	<code>printer on · printer off · printer close · printer file on unit file, record</code>
----------------------	---

<code>printer.setting</code>	<code>printer.setting(a, b, c)</code>
------------------------------	---

<code>printerr</code>	<code>printerr message</code>
-----------------------	-------------------------------

<code>private</code>	<code>private var, ...</code>
----------------------	-------------------------------

<code>procread</code>	<code>procread var then ... else ...</code>
-----------------------	---

<code>procwrite</code>	<code>procwrite var</code>
------------------------	----------------------------

<code>program</code>	<code>program name</code>
----------------------	---------------------------

<code>public</code>	<code>public var, ...</code>
---------------------	------------------------------

<code>pwr</code>	<code>pwr(a, b)</code>
------------------	--------------------------

Q

<code>quote</code>	<code>quote(<i>a</i>)</code>
--------------------	--------------------------------

R

<code>raise</code>	<code>raise(<i>a</i>)</code>
--------------------	--------------------------------

<code>randomize</code>	<code>randomize · randomize(<i>seed</i>)</code>
------------------------	---

<code>rdiv</code>	<code>rdiv(<i>a</i>, <i>b</i>)</code>
-------------------	---

<code>read</code>	<code>read var from <i>file.var</i>, <i>id</i> { on error ... } then ... else ...</code>
-------------------	--

<code>read.socket</code>	<code>read.socket(<i>a</i>, <i>b</i>, <i>c</i>, <i>d</i>)</code>
--------------------------	--

<code>readblk</code>	<code>readblk var from <i>file.var</i>, <i>length</i> then ... else ...</code>
----------------------	--

<code>readcsv</code>	<code>readcsv from <i>file.var</i> to <i>var</i>, ... then ... else ...</code>
----------------------	--

<code>readl</code>	<code>readl var from <i>file.var</i>, <i>id</i> { locked ... } { on error ... } then ... else ...</code>
--------------------	--

<code>readlist</code>	<code>readlist var { from <i>list.no</i> } then ... else ...</code>
-----------------------	---

<code>readnext</code>	<code>readnext <i>id</i> {, <i>vm</i>} { from <i>list.no</i> } { on error ... } then ... else ...</code>
-----------------------	--

<code>readonly</code>	clause of <code>open</code> , <code>openpath</code> and <code>openseq</code> (clause word)
-----------------------	---

<code>readseq</code>	<code>readseq var from <i>file.var</i> { on error ... } then ... else ...</code> — <code>else</code> is end of file
----------------------	---

<code>readu</code>	<code>readu var from <i>file.var</i>, <i>id</i> { locked ... } { on error ... } then ... else ...</code>
--------------------	--

<code>readv</code>	<code>readv var from <i>file.var</i>, <i>id</i>, <i>field</i> { on error ... } then ... else ...</code>
--------------------	---

<code>readvl</code>	<code>readvl var from <i>file.var</i>, <i>id</i>, <i>field</i> { locked ... } then ... else ...</code>
---------------------	--

<code>readvu</code>	<code>readvu var from <i>file.var</i>, <i>id</i>, <i>field</i> { locked ... } then ... else ...</code>
---------------------	--

<code>recordlocked</code>	<code>recordlocked(<i>file.var</i>, <i>id</i>)</code> — -3 -2 -1 0 1 2 3
---------------------------	---

recordlock1	<code>recordlock1</code> <i>file.var, id</i> { <code>locked</code> ... } { <code>on error</code> ... }
recordlocku	<code>recordlocku</code> <i>file.var, id</i> { <code>locked</code> ... } { <code>on error</code> ... }
release	<code>release</code> { <i>file.var</i> { <i>id</i> } } { <code>on error</code> ... } — bare <code>release</code> releases everything
rem	<code>rem</code> (<i>a, b</i>)
remark	<code>remark</code> <i>text</i> — synonym of <code>rem</code>
remove	<code>remove</code> <i>value</i> <code>from</code> <i>arr</i> <code>setting</code> <i>delimiter.code</i>
remove.break.handler	<code>remove.break.handler</code>
repeat	<code>repeat</code> — closes a <code>loop</code>
replace	<code>replace</code> (<i>arr, f</i> { <i>v</i> { <i>sv</i> } }, <i>value</i>)
restore.screen	<code>restore.screen</code> <i>image, restore.cursor</i>
return	<code>return</code> { <code>to</code> <i>label</i> } · <code>return</code> (<i>value</i>) from a function
returning	clause of <code>execute</code> — take the command's error text (clause word)
reuse	<code>reuse</code> (<i>a</i>)
rnd	<code>rnd</code> (<i>a</i>)
rollback	<code>rollback</code> , inside <code>begin transaction</code> ... <code>end transaction</code>
rqm	<code>rqm</code> { <i>seconds</i> }
rtnlist	clause of <code>execute</code> — take the list the command left (clause word)
rtrans	<code>rtrans</code> (<i>file.var, id, field, action</i>)

S

save.screen	<code>save.screen</code> (<i>a, b, c, d</i>)
savelist	<code>savelist</code> <i>name</i> { <code>from</code> <i>list.no</i> } <code>then</code> ... <code>else</code> — the <i>then/else</i> is not optional

sddecrypt	<code>sddecrypt(<i>a</i>, <i>b</i>, <i>c</i>)</code>
sdencrypt	<code>sdencrypt(<i>a</i>, <i>b</i>, <i>c</i>)</code>
seek	<code>seek <i>file.var</i>, <i>offset</i> {, <i>relative.to</i>} then ... else ...</code>
select	<code>select {<i>file.var</i>} {to <i>list.no</i>}</code>
selecte	<code>selecte {to <i>var</i>}</code>
selectindex	<code>selectindex <i>index</i> {, <i>value</i>} from <i>file.var</i> {to <i>list.no</i>}</code>
selectinfo	<code>selectinfo(<i>a</i>, <i>b</i>)</code>
selectleft	<code>selectleft <i>index</i> from <i>file.var</i> setting <i>var</i> to <i>list.no</i> — no then / else</code>
selectn	<code>selectn {<i>file.var</i>} to <i>list.no</i></code>
selectright	<code>selectright <i>index</i> from <i>file.var</i> setting <i>var</i> to <i>list.no</i> — no then / else</code>
selectv	<code>selectv {<i>file.var</i>} to <i>var</i> — read it back with readnext ... from</code>
sendmail	<code>sendmail <i>to</i>, <i>from</i>, <i>subject</i>, <i>body</i> {, <i>attachments</i>} then ... else ...</code>
sentence	<code>sentence()</code> — the same value as <code>@sentence</code>
seq	<code>seq(<i>a</i>)</code>
server.addr	<code>server.addr(<i>a</i>)</code>
set	<code>set <i>var</i> on <i>object</i> to <i>value</i></code>
set.arg	<code>set.arg <i>n</i>, <i>value</i></code>
set.break.handler	<code>set.break.handler <i>subroutine.name</i></code>
set.exit.status	<code>set.exit.status <i>n</i></code>
set.port.params	<code>set.port.params(<i>a</i>, <i>b</i>)</code>
set.socket.mode	<code>set.socket.mode(<i>a</i>, <i>b</i>, <i>c</i>)</code>
setleft	<code>setleft <i>index</i> from <i>file.var</i> — no then / else</code>
setnls	<code>setnls <i>key</i>, <i>value</i></code>
setpu	<code>setpu on <i>unit</i>, <i>key</i>, <i>value</i></code>

setrem	<code>setrem</code> <i>position</i> <code>on</code> <i>var</i>
setright	<code>setright</code> <i>index</i> <code>from</code> <i>file.var</i> — <i>no</i> <code>then</code> / <code>else</code>
setting	clause of <code>find</code> , <code>findstr</code> , <code>locate</code> , <code>remove</code> , <code>selectleft</code> , <code>selectright</code> (clause word)
shift	<code>shift(a, b)</code>
sin	<code>sin(a)</code>
sleep	<code>sleep {seconds}</code> · <code>sleep</code> <i>hh:mm:ss</i>
socket.info	<code>socket.info(a, b)</code>
soundex	<code>soundex(a)</code>
soundexs	<code>soundexs(a)</code>
space	<code>space(a)</code>
spaces	<code>spaces(a)</code>
splice	<code>splice(a, b, c)</code>
sqrt	<code>sqrt(a)</code>
squote	<code>squote(a)</code>
sselect	<code>sselect {file.var} {by field} {to list.no}</code>
status	<code>status()</code>
step	clause of <code>for</code> : <code>for</code> <i>v</i> = <i>a</i> <code>to</code> <i>b</i> <code>step</code> <i>n</i> (clause word)
stop	<code>stop {message}</code>
str	<code>str(a, b)</code>
strs	<code>strs(a, b)</code>
sub	<code>sub</code> <i>name</i> (<i>arg</i> , ...) — synonym of <code>subroutine</code>
subr	<code>subr("name" { arg, ... })</code>
subroutine	<code>subroutine</code> <i>name</i> (<i>arg</i> , ...)

substitute	<code>substitute(<i>arr</i>, <i>from</i>, <i>to</i>, <i>delimiter</i>)</code>
substrings	<code>substrings(<i>a</i>, <i>b</i>, <i>c</i>)</code>
sum	<code>sum(<i>a</i>)</code>
summation	<code>summation(<i>a</i>)</code>
swap	<code>swap(<i>string</i>, <i>from</i>, <i>to</i> {, <i>occurrences</i> {, <i>start</i>}})</code>
swapcase	<code>swapcase(<i>a</i>)</code>
sysmsg	<code>sysmsg(<i>number</i> {, <i>substitution</i>, ...})</code>
system	<code>system(<i>a</i>)</code>

T

tan	<code>tan(<i>a</i>)</code>
tclread	<code>tclread <i>var</i></code>
terminfo	<code>terminfo(<i>capability</i> {, <i>arg</i>, ...})</code>
then	clause of <code>if</code> , and of every statement that has an <code>else</code> (clause word)
time	<code>time()</code>
timedate	<code>timedate()</code>
timeout	<code>timeout <i>file.var</i>, <i>seconds</i></code>
to	clause of <code>open</code> , <code>for</code> , <code>equate</code> , <code>readcsv</code> , <code>select</code> , <code>savelist</code> , <code>writcsv</code> (clause word)
trans	<code>trans({<i>dict</i>, } <i>file.name</i>, <i>id</i>, <i>field</i>, <i>action</i>)</code>
transaction	<code>transaction start</code> · <code>transaction commit</code> · <code>transaction abort</code>
trapping	clause of <code>execute</code> — trap the break key (clause word)
trim	<code>trim(<i>string</i> {, <i>character</i> {, <i>option</i>}})</code>

<code>trimb</code>	<code>trimb(<i>a</i>)</code>
<code>trimbs</code>	<code>trimbs(<i>a</i>)</code>
<code>trimf</code>	<code>trimf(<i>a</i>)</code>
<code>trimfs</code>	<code>trimfs(<i>a</i>)</code>
<code>trims</code>	<code>trims(<i>arr</i> {, <i>character</i> {, <i>option</i>}})</code>

U

<code>umask</code>	<code>umask(<i>a</i>)</code>
<code>unassigned</code>	<code>unassigned(<i>a</i>)</code>
<code>unlock</code>	<code>unlock {<i>n</i>}</code>
<code>until</code>	clause of <code>loop</code> : <code>loop ... until <i>condition</i> ... repeat</code>
<code>upcase</code>	<code>upcase(<i>a</i>)</code>

V

<code>vartype</code>	<code>vartype(<i>a</i>)</code>
<code>void</code>	<code>void</code> <i>function.call</i> — call a function and discard its value
<code>vslice</code>	<code>vslice(<i>a</i>, <i>b</i>)</code>

W

<code>waiting</code>	clause of <code>input</code> : <code>input var waiting seconds (clause word)</code>
----------------------	--

wake	wake <i>user.no</i>
weofseq	weofseq <i>file.var</i> { on error ... }
while	clause of loop: loop ... while <i>condition</i> ... repeat
write	write <i>var</i> to <i>file.var</i> , <i>id</i> { on error ... } — <i>inside a transaction the lock must already be held</i>
write.socket	write.socket(<i>a</i> , <i>b</i> , <i>c</i> , <i>d</i>)
writeblk	writeblk <i>var</i> to <i>file.var</i> { on error ... }
writcsv	writcsv <i>value</i> , ... to <i>file.var</i> then ... else ...
writeseq	writeseq <i>var</i> to <i>file.var</i> { on error ... } then ... else ...
writeseqf	writeseqf <i>var</i> to <i>file.var</i> { on error ... } then ... else ...
writeu	writeu <i>var</i> to <i>file.var</i> , <i>id</i> { on error ... }
writev	writev <i>var</i> to <i>file.var</i> , <i>id</i> , <i>field</i> { on error ... }
writevu	writevu <i>var</i> to <i>file.var</i> , <i>id</i> , <i>field</i> { on error ... }

X

xlate	xlate({dict, } <i>file.name</i> , <i>id</i> , <i>field</i> , <i>action</i>)
xtd	xtd(<i>a</i>)

See also

SD Basic - Program Structure · SD Basic - Program Control · SD Basic - File Handling · SD Basic - System and Environment.

SD TCL - Syntax

Every verb you can type, alphabetically, with its syntax and nothing else.

A lookup card, and nothing else. If you know which verb you want and have forgotten its arguments, it is here. If you want to know what it *does*, the subject documents are where that lives.

Italics mark something you supply, **bold** a word typed as it stands, braces an optional part, and a vertical bar separates alternatives. SD folds case, so any of this may be typed in either case.

This page is generated, and it is checked for completeness rather than proof-read for it. The roster is computed from SD's own VOC: the verb records in `newvoc` plus the ones an administrator account adds, which is **147** verbs, and `tools/mktc1syntax.py` refuses to write the page if any of them has no line. The shapes come from the subject documents, where each verb is described in full.

The tier column is the VOC, not an opinion. It is read from `TIER.OMIT.STANDARD` and `TIER.ADD.ADMINISTRATOR`, the same two lists the account-creation code uses, so it cannot drift from what an account actually gets. **A verb your account does not have is not refused — the name is simply not recognised.**

standard	82 verbs	every account has these
programmer	42 more	withheld from a standard account
administrator	23 more	and several need an elevated session as well

The verbs

	SYNTAX	TIER
<code>!</code>	<code>! command</code>	A
<code>abort</code>	<code>abort {message}</code>	
<code>alias</code>	<code>alias command target</code>	

	SYNTAX	TIER
<code>analyse.file</code>	<code>analyse.file</code> { <code>dict</code> } <i>file</i> { <code>statistics</code> } { <code>lptr</code> }	P
<code>analyze.file</code>	<code>analyze.file</code> — the same verb as <code>analyse.file</code>	P
<code>append.sd.path</code>	<code>append.sd.path</code> { <code>on</code> <code>off</code> }	
<code>autologout</code>	<code>autologout</code> { <i>minutes</i> }	
<code>basic</code>	<code>basic</code> { <i>file</i> } <i>record</i> { <i>record ...</i> }	P
<code>bell</code>	<code>bell on</code> <code>off</code>	
<code>break</code>	<code>break on</code> <code>off</code>	
<code>build.index</code>	<code>build.index</code> <i>file field</i> ...	<code>all</code>
<code>catalog</code>	<code>catalog</code> { <i>file</i> { <i>call.name</i> } } { <i>program</i> } { <code>local</code>	<code>global</code>
<code>catalogue</code>	<code>catalogue</code> — the same verb as <code>catalog</code>	P
<code>cd</code>	<code>cd</code> { <code>dict</code> <code>data</code> } <i>file</i> { <i>i-type ...</i> } { <code>no.query</code> } { <code>no.page</code> } · <code>cd</code> <code>local</code> · <code>cd all</code>	
<code>clean.account</code>	<code>clean.account</code>	A
<code>clear.abort</code>	<code>clear.abort</code>	
<code>clear.data</code>	<code>clear.data</code>	
<code>clear.file</code>	<code>clear.file</code> { <code>data</code> <code>dict</code> } <i>file</i>	
<code>clear.input</code>	<code>clear.input</code>	
<code>clear.locks</code>	<code>clear.locks</code> { <i>n</i> }	A

	SYNTAX	TIER
<code>clear.prompts</code>	<code>clear.prompts</code>	
<code>clear.select</code>	<code>clear.select</code> { <i>list.no</i> }	
<code>clear.stack</code>	<code>clear.stack</code>	
<code>cleardata</code>	<code>cleardata</code> — the same verb as <code>clear.data</code>	
<code>clearinput</code>	<code>clearinput</code> — the same verb as <code>clear.input</code>	
<code>clearprompts</code>	<code>clearprompts</code> — the same verb as <code>clear.prompts</code>	
<code>clearselect</code>	<code>clearselect</code> — the same verb as <code>clear.select</code>	
<code>clr</code>	<code>clr</code>	
<code>cname</code>	<code>cname</code> <i>old.file</i> <code>to</code> <i>new.file</i>	P
<code>como</code>	<code>como on</code> { <i>record</i> } <code>off</code>	
<code>compile.dict</code>	<code>compile.dict</code> — the same verb as <code>cd</code>	P
<code>config</code>	<code>config</code> { <i>lptr</i> } <code>gpl</code>	
<code>configure.file</code>	<code>configure.file</code> { <code>dict</code> } <i>voc.name</i> { <i>parameters</i> }	<code>default</code>
<code>copy</code>	<code>copy from</code> { <code>dict</code> } <i>src</i> { <code>to</code> { <code>dict</code> } <i>tgt</i> } { <i>id ...</i> } { <code>options</code> }	P

	SYNTAX	TIER
copy.list	copy.list <i>list</i> { , <i>new</i> } { from <i>src</i> } { to <i>tgt</i> } { <i>options</i> }	
copyp	copyp { dict } <i>file id ...</i> { (<i>options</i>) }	P
count	count { dict } <i>file</i> { <i>selection</i> }	
create.account	create.account <i>user</i> <i>name</i> { administrator }	programmer } < ssh
create.file	create.file { dict } <i>voc.name</i> { directory } { <i>parameters</i> }	P
create.index	create.index <i>file field</i> ... { no.nulls } { pathname <i>path</i> }	P
cs	cs — the same verb as clr	
ct	ct { dict } <i>file {id ...</i> * } { (<i>options</i>) }	
date	date	internal
date.format	date.format on	off
debug	debug	P
delete	delete { dict } <i>file {id</i> ... } { no.query }	P
delete.account	delete.account <i>account</i>	A
delete.catalog	delete.catalog <i>name</i> ... { global	local }
delete.catalogue	delete.catalogue — the same verb as delete.catalog	P

	SYNTAX	TIER
<code>delete.common</code>	<code>delete.common</code> <i>name</i>	P
<code>delete.file</code>	<code>delete.file</code> { <code>dict</code> <code>data</code> } <i>voc.name</i> { <code>force</code> } { <code>no.query</code> }	
<code>delete.index</code>	<code>delete.index</code> <i>file field</i> <code>all</code> ...	
<code>delete.list</code>	<code>delete.list</code> <i>list</i>	
<code>display</code>	<code>display</code> <i>text</i>	
<code>dump</code>	<code>dump</code> { <code>dict</code> } <i>file</i> { <i>id ...</i> } * { <code>(options)</code> }	
<code>echo</code>	<code>echo</code> <code>on</code> <code>off</code>	
<code>ed</code>	<code>ed</code> { <code>dict</code> } <i>file</i> { <i>id ...</i> }	P
<code>edit</code>	<code>edit</code> { <code>dict</code> } <i>file</i> <i>record</i>	P
<code>form.list</code>	<code>form.list</code> { <code>dict</code> } <i>file</i> { <i>list.no</i> }	
<code>format</code>	<code>format</code> { <i>file</i> } { <i>record</i> } { <code>case</code> }	
<code>fstat</code>	<code>fstat</code> <i>file</i> <code>on</code> <code>off</code>	
<code>generate</code>	<code>generate</code> { <code>dict</code> } <i>file</i> <i>record ...</i>	P
<code>get.list</code>	<code>get.list</code> <i>list</i> { <code>to</code> <i>list.no</i> }	
<code>get.stack</code>	<code>get.stack</code> { <i>name</i> }	
<code>go</code>	<code>go</code> <i>label</i>	
<code>grant</code>	<code>grant</code> <i>account</i> <code>to</code> <i>user</i>	A
<code>hsm</code>	<code>hsm</code> <code>on</code> <code>off</code>	
<code>hush</code>	<code>hush</code> <code>on</code> <code>off</code>	

	SYNTAX	TIER
<code>if</code>	<code>if</code> <i>condition</i> <i>command</i>	
<code>list</code>	<code>list</code> { <code>dict</code> } <i>file</i> { <i>selection</i> } { <i>fields</i> } { <i>options</i> }	
<code>list.common</code>	<code>list.common</code> { <code>all</code> }	
<code>list.diff</code>	<code>list.diff</code> <i>list.1</i> { <i>list.2</i> { <i>tgt</i> }} { <code>count.sup</code> }	
<code>list.files</code>	<code>list.files</code>	
<code>list.grants</code>	<code>list.grants</code> <i>account</i>	A
<code>list.index</code>	<code>list.index</code> <i>file</i> { <i>field</i> ...}	P
<code>list.inter</code>	<code>list.inter</code> <i>list.1</i> { <i>list.2</i> { <i>tgt</i> }} { <code>count.sup</code> }	
<code>list.item</code>	<code>list.item</code> { <code>dict</code> } <i>file</i> { <i>selection</i> }	
<code>list.label</code>	<code>list.label</code> { <code>dict</code> } <i>file</i> { <i>selection</i> } { <i>fields</i> }	
<code>list.locks</code>	<code>list.locks</code>	A
<code>list.readu</code>	<code>list.readu</code> { <i>user.no</i> } { <code>detail</code> } { <code>wait</code> } { <code>no.page</code> } { <code>lptr</code> } { <i>n</i> }	A
<code>list.union</code>	<code>list.union</code> <i>list.1</i> { <i>list.2</i> { <i>tgt</i> }} { <code>count.sup</code> }	
<code>list.vars</code>	<code>list.vars</code> { <i>pattern</i> }	
<code>listu</code>	<code>listu</code> { <code>no.page</code> } { <code>lptr</code> } { <i>n</i> }	A
<code>lock</code>	<code>lock</code> <i>n</i> { <code>no.wait</code> }	A

	SYNTAX	TIER
<code>logmsg</code>	<code>logmsg</code> <i>text</i>	
<code>logout</code>	<code>logout</code>	<i>n ...</i>
<code>logto</code>	<code>logto</code> <i>account</i>	
<code>make.index</code>	<code>make.index</code> <i>file field ...</i> { <code>no.nulls</code> } { <code>pathname</code> <i>path</i> }	P
<code>map</code>	<code>map</code> { <code>all</code> } { <code>lptr</code> { <i>n</i> } } { <code>file</code> { <i>name</i> } }	P
<code>merge.list</code>	<code>merge.list</code> <i>list.no</i> <i>rel.op list.no</i> { <code>to</code> <i>list.no</i> } { <code>count.sup</code> }	
<code>message</code>	<code>message</code> <i>user.no</i> { <code>immediate</code> } { <i>text</i> }	
<code>micro</code>	<code>micro</code> { <code>dict</code> } <i>file</i> <i>record</i>	P
<code>modify.account</code>	<code>modify.account</code> <i>account</i> <code>standard</code>	<code>programmer</code>
<code>modify.password</code>	<code>modify.password</code> { <i>account</i> }	A
<code>nselect</code>	<code>nselect</code> <i>file</i> { <i>list.no</i> }	
<code>off</code>	<code>off</code>	
<code>option</code>	<code>option</code> <i>name</i> { <code>on</code>	<code>off</code>
<code>pause</code>	<code>pause</code> { <i>seconds</i> }	
<code>pdebug</code>	<code>pdebug</code> { <i>command</i> }	P
<code>pdump</code>	<code>pdump</code> <i>n</i>	P
<code>phantom</code>	<code>phantom</code> <i>command</i>	P
<code>printer</code>	<code>printer</code> { <i>unit</i> } <code>query</code>	<code>at</code> <i>name</i>

	SYNTAX	TIER
pstat	pstat { user <i>n</i> } { level <i>n</i> } { no.page } { lptr { <i>n</i> } }	P
pterm	pterm display	lptr
qselect	qselect { dict } <i>file</i> <i>{id ...</i>	* { (options) }
quit	quit — the same verb as off	
reformat	reformat { dict } <i>file</i> <i>{selection} {fields}</i>	P
release	release <i>file id ...</i> · release filelock <i>file</i>	
remote.api	remote.api { on	local
remote.ssh	remote.ssh { on	off }
rename	rename — the same verb as cname	P
report.src	report.src on	off
report.style	report.style { <i>name</i>	off }
revoke	revoke <i>account</i> from <i>user</i>	A
run	run { <i>file</i> } <i>record</i> <i>{arguments}</i>	P
save.list	save.list <i>list</i> { from <i>list.no</i> }	
save.stack	save.stack { <i>name</i> }	
search	search { dict } <i>file</i> <i>{selection}</i>	

	SYNTAX	TIER
select	select { dict } <i>file</i> { <i>selection</i> } { <i>list.no</i> }	
set	set <i>name value</i>	
set.date	set.date <i>date</i>	A
set.exit.status	set.exit.status <i>n</i>	
set.file	set.file <i>account file</i> <i>pointer</i>	
set.trigger	set.trigger <i>file name</i> { <i>modes</i> }	P
setptr	setptr <i>unit</i> default , width , depth , top , bottom , mode { , options } · setptr display · setptr <i>unit</i> , display	
sh	sh <i>command</i>	A
show	show { dict } <i>file</i> { <i>selection</i> }	
sleep	sleep <i>n</i> <i>hh</i> : <i>mm</i> { : <i>ss</i> }	
sort	sort { dict } <i>file</i> { <i>selection</i> } { <i>fields</i> } { <i>options</i> }	
sort.item	sort.item { dict } <i>file</i> { <i>selection</i> }	
sort.label	sort.label { dict } <i>file</i> { <i>selection</i> } { <i>fields</i> }	
sp.close	sp.close	
sp.open	sp.open	
sp.view	sp.view	
spool	spool <i>file id ...</i> { lines <i>n m</i> } { lnum } { lptr <i>n</i> }	

	SYNTAX	TIER
sreformat	sreformat {dict} file {selection} {fields}	P
sselect	sselect {dict} file {selection} {list.no}	
ssh.server	ssh.server {install remove}	
status	status	
stop	stop	
sum	sum {dict} file {selection} field	
term	term	width{, lines} {type}
time	time	internal
unlock	unlock file n {user all} n} id ...	
update.accounts	update.accounts {all}	A
who	who	
who.am.i	who.am.i	

Blank in the tier column means every account has it; ☐ P is a programmer verb and ☐ A an administrator one.